Chapter 2

Mathematical Foundations: Frames, Tensors and Coordinate Systems

> *A velocity of* $300\,\mathrm{m/s}$ *means nothing until you say: relative to what, and measured along which axes?*

Every number inside a flight simulator is one of three things: a plain number (mass, time), a vector (position, velocity, force), or a matrix-like object (an inertia tensor, a rotation). And every vector and matrix only has numbers once we have chosen *which frame* we are talking about and *which axes* we are measuring along. Most bugs in real flight software are not deep physics mistakes. They are a vector written in the wrong axes, a rotation applied in the wrong order, or a derivative taken in the wrong frame.

This chapter gives you the tools to never make those mistakes. We start from the very basics and build up:

1. vectors and matrices, and what they really mean (Section 2.1);
2. the linear algebra the whole book uses: eigenvalues, symmetric matrices, the matrix exponential, norms, and the complex version used in quantum computing (Section 2.2);
3. the difference between a *frame* and a *coordinate system*, and the frames a launch vehicle needs (Section 2.3);
4. how to convert a vector from one set of axes to another (Section 2.4);
5. Euler angles and why they fail on the launch pad, and quaternions, which do not (Sections 2.5 and 2.6);
6. how to take a time derivative when the axes themselves are rotating, which is where the Coriolis force comes from (Section 2.7);
7. tensors, with the inertia tensor as the main example (Section 2.8);
8. the shape of the Earth and how to turn latitude and longitude into a position vector (Section 2.9);
9. how numbers behave inside a computer (Section 2.10);

10. how a computer makes random numbers with any distribution we need, the starting point of every Monte Carlo simulation (Section 2.11).

As in Chapter 1, every idea comes with a worked example that uses real numbers, so you can check each step with a calculator.

2.1 Vectors and matrices: what the numbers mean

2.1.1 A vector is an arrow, not a list of numbers

A *scalar* is a single number with a unit: a mass of 500 t, a time of 12 s. A *vector* has a size *and* a direction: the wind blowing east at 10 m/s, the thrust pushing the rocket upward, the position of the vehicle relative to the launch pad.

The important point is this. **The vector itself is a physical arrow. The list of numbers we write for it depends on the axes we choose.** The same wind can be written in many different ways, as Fig. 2.1 shows.

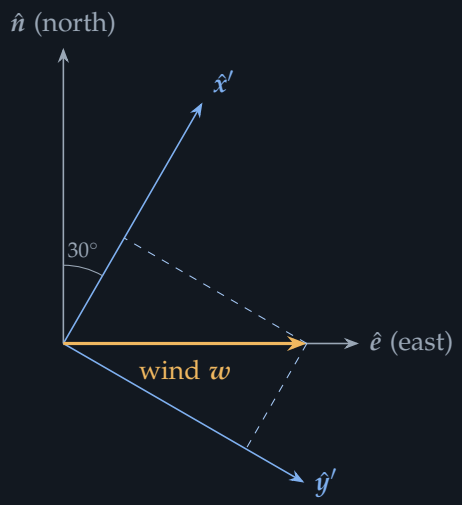


Figure 2.1: One wind vector $\boldsymbol{w}$, two sets of axes. Along east and north it is $(10,\ 0)$ m/s (east, north). Along the rotated axes $\hat{\boldsymbol{x}}'$ (pointing 30° east of north) and $\hat{\boldsymbol{y}}'$ it is $(5,\ 8.66)$ m/s. The arrow has not changed; only its description has.

Worked example: One wind, two descriptions. The wind blows toward the east at 10 m/s. A rocket on the pad is pointed with its x'-axis at a heading of 30° (that is, 30° east of north), and its y'-axis 90° to the right of that. The component of the wind along an axis is the wind speed times the cosine of the angle between them:

$$w_{x'} = 10\cos 60^\circ = 5.00\,\text{m/s}, \qquad w_{y'} = 10\cos 30^\circ = 8.66\,\text{m/s}.$$

Check: $\sqrt{5^2 + 8.66^2} = \sqrt{25 + 75} = 10$. The length is the same in both descriptions, as it must be.

We write the physical vector in bold, $\boldsymbol{v}$, and its list of components in a particular coordinate

system A as $[\boldsymbol{v}]^A$, a column of three numbers:

$$[\boldsymbol{v}]^A = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix}, \qquad \boldsymbol{v} = v_1\,\hat{\boldsymbol{a}}_1 + v_2\,\hat{\boldsymbol{a}}_2 + v_3\,\hat{\boldsymbol{a}}_3, \tag{2.1}$$

where $\hat{\boldsymbol{a}}_1, \hat{\boldsymbol{a}}_2, \hat{\boldsymbol{a}}_3$ are the unit vectors along the axes of A. The hat means "length one".

2.1.2 The operations you need

Adding and scaling. Vectors add tip-to-tail, and in components you simply add matching entries. Multiplying by a number stretches the arrow. These are what let us write, for example, total force = thrust + drag + weight.

The dot product. For two vectors in the same coordinate system,

$$\boldsymbol{a} \cdot \boldsymbol{b} = a_1 b_1 + a_2 b_2 + a_3 b_3 = \|\boldsymbol{a}\|\ \|\boldsymbol{b}\| \cos\gamma, \tag{2.2}$$

where γ is the angle between them and $\|\boldsymbol{a}\| = \sqrt{\boldsymbol{a} \cdot \boldsymbol{a}}$ is the length. The dot product answers two everyday questions:

- *What is the angle between two directions?* $\cos\gamma = \dfrac{\boldsymbol{a} \cdot \boldsymbol{b}}{\|\boldsymbol{a}\|\,\|\boldsymbol{b}\|}$. If $\boldsymbol{a} \cdot \boldsymbol{b} = 0$ the vectors are perpendicular.
- *How much of $\boldsymbol{a}$ points along a unit direction $\hat{\boldsymbol{u}}$?* The answer is $\boldsymbol{a} \cdot \hat{\boldsymbol{u}}$. This is exactly what we did with the wind: each component is a dot product with a unit axis vector.

In matrix notation, with column vectors, $\boldsymbol{a} \cdot \boldsymbol{b} = \boldsymbol{a}^{\mathsf{T}} \boldsymbol{b}$.

The cross product. This one only exists in three dimensions:

$$\boldsymbol{a} \times \boldsymbol{b} = \begin{bmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{bmatrix}, \qquad \|\boldsymbol{a} \times \boldsymbol{b}\| = \|\boldsymbol{a}\|\ \|\boldsymbol{b}\| \sin\gamma. \tag{2.3}$$

The result is perpendicular to both $\boldsymbol{a}$ and $\boldsymbol{b}$, with direction given by the *right-hand rule*: point the fingers of your right hand along $\boldsymbol{a}$, curl them toward $\boldsymbol{b}$, and your thumb points along $\boldsymbol{a} \times \boldsymbol{b}$. Note that the order matters: $\boldsymbol{b} \times \boldsymbol{a} = -\boldsymbol{a} \times \boldsymbol{b}$.

The cross product is how we compute a *moment* (a twisting effect): a force $\boldsymbol{F}$ applied at a point $\boldsymbol{r}$ from the centre of mass produces the moment $\boldsymbol{M} = \boldsymbol{r} \times \boldsymbol{F}$.

> **Worked example**: The moment from a gimballed engine. Use the body axes of Fig. 2.2. The engine is $\ell = 15\,\text{m}$ behind the centre of gravity, so $[\boldsymbol{r}]^B = (-15,\ 0,\ 0)\,\text{m}$. The thrust is $T = 900\,\text{kN}$, tilted by $\delta = 2^\circ$ so that part of it pushes the tail toward $+z_B$:

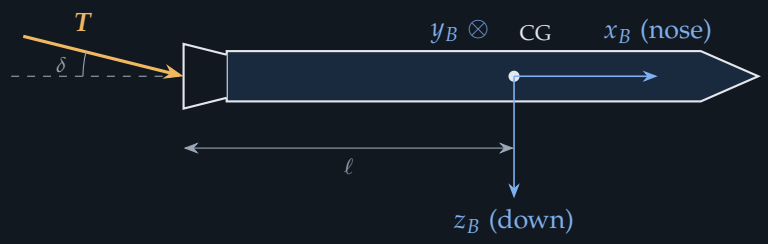


Figure 2.2: Body axes: x_B out of the nose, y_B out of the right side (into the page, shown $\otimes$), z_B completing a right-handed set, pointing "down" in normal flight. The engine sits a distance ℓ behind the centre of gravity, and the gimbal tilts the thrust by δ. (The angle is exaggerated.)

$[\boldsymbol{T}]^B = T\,(\cos\delta,\ 0,\ \sin\delta)$. Using (2.3) with $\boldsymbol{a} = \boldsymbol{r}$, $\boldsymbol{b} = \boldsymbol{T}$:

$$\boldsymbol{M} = \boldsymbol{r} \times \boldsymbol{T} = \begin{bmatrix} 0 \cdot T\sin\delta - 0 \cdot 0 \\ 0 \cdot T\cos\delta - (-\ell)\,T\sin\delta \\ (-\ell)\cdot 0 - 0 \cdot T\cos\delta \end{bmatrix} = \begin{bmatrix} 0 \\ \ell\,T\sin\delta \\ 0 \end{bmatrix}.$$

Numerically, $M_y = 15 \times 900\,000 \times \sin 2^\circ = 15 \times 900\,000 \times 0.0349 \approx 471\,\text{kN m}$. The moment is about the y_B-axis (pitch), and positive, which by the right-hand rule about y_B means *nose up*: pushing the tail down tips the nose up. This is the same relation we wrote in Chapter 1 as $M \approx T\ell\delta$, now with the direction worked out properly.

The cross product as a matrix. It will be very useful to write "$\boldsymbol{a}\times$" as a matrix that multiplies $\boldsymbol{b}$. Reading off (2.3):

$$\boldsymbol{a} \times \boldsymbol{b} = [\boldsymbol{a}\times]\,\boldsymbol{b}, \qquad [\boldsymbol{a}\times] = \begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix}. \tag{2.4}$$

This matrix is *skew-symmetric*: $[\boldsymbol{a}\times]^{\mathsf{T}} = -[\boldsymbol{a}\times]$. We used exactly this property in Chapter 1 to show that quaternions keep their length.

2.1.3 Matrices as machines that transform vectors

A 3×3 matrix $\mathbf{A}$ takes a vector $\boldsymbol{x}$ and produces a new vector $\boldsymbol{y} = \mathbf{A}\boldsymbol{x}$, with $y_i = \sum_j A_{ij}x_j$. In this book a matrix almost always means one of three things:

- a **change of axes**: the same vector, rewritten in different coordinates (Section 2.4);
- a **physical relation** between two vectors: angular momentum = inertia tensor $\times$ angular velocity (Section 2.8);
- a **system matrix**: $\dot{\boldsymbol{x}} = \mathbf{A}\boldsymbol{x}$ describing how a state evolves (Section 2.2).

The rules you need: matrix products are done row-times-column; $(\mathbf{AB})^{\mathsf{T}} = \mathbf{B}^{\mathsf{T}}\mathbf{A}^{\mathsf{T}}$; the identity $\mathbf{I}$ has ones on the diagonal and leaves every vector unchanged; and the inverse $\mathbf{A}^{-1}$, if it exists,

undoes $\mathbf{A}$: $\mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$. The inverse exists exactly when the *determinant* is not zero. For a 2×2 matrix,

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc, \qquad \begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}. \tag{2.5}$$

Geometrically, $|\det \mathbf{A}|$ is how much $\mathbf{A}$ stretches volumes; a determinant of zero means $\mathbf{A}$ squashes space flat and cannot be undone.

> **Key idea.** Matrix multiplication is *not* commutative: in general $\mathbf{AB} \neq \mathbf{BA}$. For rotations this has a physical meaning. Turning a rocket 90° in pitch and then 90° in roll leaves it pointing somewhere different from roll-then-pitch. We will see this with numbers in Section 2.4.

2.2 The linear algebra the whole book uses

2.2.1 Eigenvalues and eigenvectors

Most vectors change direction when a matrix acts on them. A few special ones do not; they only get stretched.

Definition 2.1. A non-zero vector $\boldsymbol{v}$ is an *eigenvector* of a square matrix $\mathbf{A}$, with *eigenvalue* λ, if

$$\mathbf{A}\boldsymbol{v} = \lambda \boldsymbol{v}. \tag{2.6}$$

How to find them. Rewrite the definition as $(\mathbf{A} - \lambda \mathbf{I})\boldsymbol{v} = \mathbf{0}$. A non-zero $\boldsymbol{v}$ can solve this only if $\mathbf{A} - \lambda \mathbf{I}$ squashes space flat, that is,

$$\det(\mathbf{A} - \lambda \mathbf{I}) = 0. \tag{2.7}$$

This is a polynomial in λ, the *characteristic polynomial*. Its roots are the eigenvalues.

Why we care: eigenvalues are the poles. Write the uncontrolled pitch dynamics from Chapter 1, $\ddot{\theta} = \mu_\alpha \theta$, as two first-order equations using the state $\boldsymbol{x} = (\theta,\ \dot{\theta})$:

$$\dot{\boldsymbol{x}} = \mathbf{A}\boldsymbol{x}, \qquad \mathbf{A} = \begin{bmatrix} 0 & 1 \\ \mu_\alpha & 0 \end{bmatrix}. \tag{2.8}$$

Then

$$\det(\mathbf{A} - \lambda \mathbf{I}) = \det \begin{bmatrix} -\lambda & 1 \\ \mu_\alpha & -\lambda \end{bmatrix} = \lambda^2 - \mu_\alpha = 0 \quad \Longrightarrow \quad \lambda = \pm\sqrt{\mu_\alpha}.$$

With $\mu_\alpha = 4$ these are ± 2: exactly the poles we found with the transfer function in Chapter 1. With the PD controller of that chapter, the closed-loop matrix becomes

$$\mathbf{A}_{\text{cl}} = \begin{bmatrix} 0 & 1 \\ -9 & -4.2 \end{bmatrix}, \qquad \lambda^2 + 4.2\lambda + 9 = 0 \quad \Longrightarrow \quad \lambda = -2.1 \pm 2.14i.$$

> **Key idea.** For a system $\dot{\boldsymbol{x}} = \mathbf{A}\boldsymbol{x}$, the eigenvalues of $\mathbf{A}$ are the poles. The system is stable when every eigenvalue has a negative real part. Transfer functions and state-space matrices are two views of the same thing.

2.2.2 Symmetric matrices

A matrix is *symmetric* if $\mathbf{A}^{\mathsf{T}} = \mathbf{A}$. Two of the most important matrices in this book are symmetric: the inertia tensor (Section 2.8) and the covariance matrix of a Kalman filter (Chapter 7). Symmetric matrices are the best-behaved matrices there are.

Theorem 2.2 (Spectral theorem, real symmetric case)**.** *If $\mathbf{A}$ is real and symmetric, then all its eigenvalues are real, and eigenvectors belonging to different eigenvalues are perpendicular. Moreover $\mathbf{A}$ can be written as*

$$\mathbf{A} = \mathbf{Q}\,\blacksquare\,\mathbf{Q}^{\mathsf{T}}, \tag{2.9}$$

where the columns of $\mathbf{Q}$ are orthonormal eigenvectors and $\blacksquare$ is diagonal with the eigenvalues.

Proof of the perpendicular part. Let $\mathbf{A}\boldsymbol{u} = \lambda\boldsymbol{u}$ and $\mathbf{A}\boldsymbol{v} = \mu\boldsymbol{v}$ with $\lambda \neq \mu$. Then

$$\lambda\,\boldsymbol{v}^{\mathsf{T}}\boldsymbol{u} = \boldsymbol{v}^{\mathsf{T}}(\mathbf{A}\boldsymbol{u}) = (\mathbf{A}^{\mathsf{T}}\boldsymbol{v})^{\mathsf{T}}\boldsymbol{u} = (\mathbf{A}\boldsymbol{v})^{\mathsf{T}}\boldsymbol{u} = \mu\,\boldsymbol{v}^{\mathsf{T}}\boldsymbol{u}.$$

So $(\lambda - \mu)\,\boldsymbol{v}^{\mathsf{T}}\boldsymbol{u} = 0$, and since $\lambda \neq \mu$, $\boldsymbol{v}^{\mathsf{T}}\boldsymbol{u} = 0$. □

In words: *every symmetric matrix is just a stretch along some set of perpendicular axes.* Find those axes and the matrix becomes diagonal. For the inertia tensor, those axes are the *principal axes* of the body.

Definition 2.3. A symmetric matrix $\mathbf{A}$ is *positive definite* if $\boldsymbol{x}^{\mathsf{T}}\mathbf{A}\boldsymbol{x} > 0$ for every non-zero $\boldsymbol{x}$. Equivalently, all its eigenvalues are positive.

Inertia tensors and covariance matrices are always positive definite (or at least semi-definite): a body cannot have negative rotational inertia, and an uncertainty cannot have negative variance. If your Kalman filter ever produces a covariance matrix with a negative eigenvalue, something has gone numerically wrong.

2.2.3 The matrix exponential

For one number, $\dot{x} = ax$ has the solution $x(t) = e^{at}x(0)$. For a system $\dot{\boldsymbol{x}} = \mathbf{A}\boldsymbol{x}$ the solution looks the same:

$$\boldsymbol{x}(t) = e^{\mathbf{A}t}\,\boldsymbol{x}(0), \qquad e^{\mathbf{A}t} = \mathbf{I} + \mathbf{A}t + \frac{(\mathbf{A}t)^2}{2!} + \frac{(\mathbf{A}t)^3}{3!} + \cdots \tag{2.10}$$

The series is the same as for the ordinary exponential, with $\mathbf{A}$ in place of a. Differentiating term by term gives $\frac{\mathrm{d}}{\mathrm{d}t}e^{\mathbf{A}t} = \mathbf{A}e^{\mathbf{A}t}$, so (2.10) really does solve the equation.

Worked example: Where the Kalman filter's prediction comes from. A vehicle moving in a straight line at constant speed has state $\boldsymbol{x} = (\text{position},\ \text{velocity})$ and

$$\mathbf{A} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \qquad \mathbf{A}^2 = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}.$$

Because $\mathbf{A}^2 = \mathbf{0}$, every term after the second in (2.10) vanishes, and over a time step Δt

$$e^{\mathbf{A}\Delta t} = \mathbf{I} + \mathbf{A}\Delta t = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}.$$

So the new position is old position $+\ \Delta t \times$ velocity, and the velocity stays the same. This is exactly the "predict" step of the g–h filter in Chapter 1, and it is the matrix $\mathbf{F}$ of the Kalman filter for this model.

2.2.4 Norms and conditioning

A *norm* measures the size of a vector. Three are common:

$$\|\boldsymbol{x}\|_2 = \sqrt{\textstyle\sum_i x_i^2}, \qquad \|\boldsymbol{x}\|_1 = \textstyle\sum_i |x_i|, \qquad \|\boldsymbol{x}\|_\infty = \max_i |x_i|. \tag{2.11}$$

The Euclidean norm $\|\cdot\|_2$ is ordinary length. The other two appear in optimization: $\|\cdot\|_1$ for fuel use (the sum of thrust magnitudes), $\|\cdot\|_\infty$ for "the largest error anywhere". One inequality is used constantly, the *Cauchy–Schwarz inequality*:

$$\left|\boldsymbol{a}^{\mathsf{T}}\boldsymbol{b}\right| \le \|\boldsymbol{a}\|_2\ \|\boldsymbol{b}\|_2, \tag{2.12}$$

which is just $|\cos\gamma| \le 1$ from (2.2).

Every matrix can be written as a rotation, a stretch along perpendicular axes, and another rotation. This is the *singular value decomposition* $\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^{\mathsf{T}}$, with the stretches $\sigma_1 \ge \sigma_2 \ge \cdots \ge 0$ on the diagonal of $\boldsymbol{\Sigma}$. The ratio

$$\kappa(\mathbf{A}) = \frac{\sigma_{\max}}{\sigma_{\min}} \tag{2.13}$$

is the *condition number*. When we solve $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ on a computer, a small relative error in $\boldsymbol{b}$ can grow by up to a factor of κ in $\boldsymbol{x}$.

Worked example: An almost-singular system.

$$\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 1 & 1.0001 \end{bmatrix}.$$

This matrix is symmetric and positive definite, so its singular values equal its eigenvalues, about 2.00005 and 0.00005. The condition number is about 40 000. An error in the fifth significant figure of $\boldsymbol{b}$ can wipe out the first significant figure of $\boldsymbol{x}$. Matrices like this appear when two sensors measure almost the same thing, which is why navigation filters need care (Chapter 7).

2.2.5 Complex vectors: the same ideas in quantum notation

Quantum computing (Chapter 8) uses vectors with *complex* entries. Almost everything above carries over if you replace "transpose" by "conjugate transpose", written with a dagger: $\mathbf{A}^\dagger = \overline{\mathbf{A}}^\mathsf{T}$. The table is worth memorising, because it lets you reuse everything you know.

Idea	Real (flight dynamics)	Complex (quantum)
inner product	$\boldsymbol{a}^\mathsf{T}\boldsymbol{b}$	$\langle a\|b\rangle = \boldsymbol{a}^\dagger\boldsymbol{b}$
length-preserving matrix	orthogonal: $\mathbf{R}^\mathsf{T}\mathbf{R} = \mathbf{I}$	unitary: $U^\dagger U = I$
"nice" matrix with real eigenvalues	symmetric: $\mathbf{A}^\mathsf{T} = \mathbf{A}$	Hermitian: $H^\dagger = H$
example	rotation matrix, inertia tensor	quantum gate, Hamiltonian

A rotation matrix for a rocket and a quantum gate for a qubit are the same kind of object: a matrix that changes the description without changing the length.

The tensor (Kronecker) product. To describe two qubits together we combine their vectors with the Kronecker product. For a 2×2 matrix $\mathbf{A}$ and any matrix $\mathbf{B}$,

$$\mathbf{A} \otimes \mathbf{B} = \begin{bmatrix} a_{11}\mathbf{B} & a_{12}\mathbf{B} \\ a_{21}\mathbf{B} & a_{22}\mathbf{B} \end{bmatrix}, \qquad \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} \otimes \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} = \begin{bmatrix} a_1 b_1 \\ a_1 b_2 \\ a_2 b_1 \\ a_2 b_2 \end{bmatrix}. \tag{2.14}$$

Sizes multiply: two 2-vectors give a 4-vector, and n qubits give a 2^n-vector. A useful rule is $(\mathbf{A} \otimes \mathbf{B})(\mathbf{C} \otimes \mathbf{D}) = (\mathbf{AC}) \otimes (\mathbf{BD})$: applying a gate to each qubit separately is the same as applying their Kronecker product to the pair.

2.3 Frames and coordinate systems

2.3.1 Two words that are not the same

Everyday language uses "frame" and "coordinate system" as if they meant the same thing. In flight dynamics, following Zipfel, we keep them apart, because mixing them up causes real errors.

A frame is a rigid body of reference: a set of points that do not move relative to each other. "The Earth", "the rocket" and "the distant stars" are frames. A frame tells you *what counts as standing still*. Velocity, acceleration and time derivatives only make sense *relative to a frame*.

A coordinate system is a choice of origin and three perpendicular unit axes, fixed in some frame, used to write vectors as lists of numbers. A coordinate system tells you *which numbers to write down*.

Worked example: A point on the launch pad. Take a point on the launch pad at Cape Canaveral, latitude 28.6°. Relative to the *Earth frame* it is not moving: its velocity is zero. Relative to an *inertial frame* (one that does not rotate with the Earth) it is carried around by the Earth's rotation at

$$v = \omega_E R \cos\varphi \approx 7.292 \times 10^{-5} \times 6\,383\,000 \times \cos 28.6^\circ \approx 408\,\mathrm{m/s}$$

toward the east. Same point, two frames, two completely different velocities. Neither is wrong. This is free speed for a rocket launched eastward, and one reason launch sites are placed close to the equator, where the figure is 465 m/s.

The general rule: **the frame decides the physics (what the velocity is); the coordinate system decides the bookkeeping (which numbers describe it).** We can write the inertial velocity of the rocket in Earth-fixed axes, or its Earth-relative velocity in body axes. Both are legitimate, and a simulator does both constantly.

2.3.2 The frames of a launch vehicle

Frame (symbol)	Origin and axes	What we use it for
Inertial, ECI (I)	Earth's centre; z along the spin axis, x toward a fixed direction in the sky; does not rotate	Newton's law holds here; orbital mechanics; navigation integrates here
Earth-fixed, ECEF (E)	Earth's centre; z along the spin axis, x through latitude 0°, longitude 0°; rotates with the Earth	Launch site and landing site positions; GPS outputs
Local level, NED (G)	A point on the Earth's surface; x north, y east, z down	Altitude, heading, flight-path angle; what a human understands
Body (B)	Vehicle centre of mass; x out of the nose, y right, z down (Fig. 2.2)	Engine and fin forces; IMU measurements; inertia tensor
Velocity, or wind (V)	Vehicle centre of mass; x along the velocity relative to the air	Lift and drag; angle of attack

Table 2.1: The main frames and coordinate systems in this book. In practice each frame comes with its own natural coordinate system, so we usually use the same letter for both.

Reality check: Is the ECI frame really inertial?. Not exactly. The Earth's centre accelerates toward the Sun at about $(29.8 \times 10^3)^2 / 1.496 \times 10^{11} \approx 5.9 \times 10^{-3}\,\mathrm{m/s^2}$. But the Sun pulls on the rocket almost exactly as hard, so the rocket and the Earth fall toward the Sun together and the difference is tiny. For the few minutes of an ascent or a landing, treating ECI as inertial is an excellent approximation.

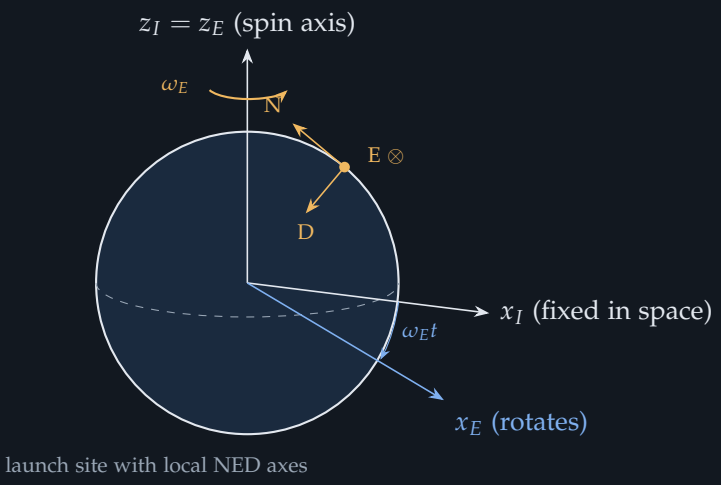


Figure 2.3: Inertial axes stay fixed in space; Earth-fixed axes turn with the Earth at $\omega_E = 7.292\,115\,9 \times 10^{-5}\,\mathrm{rad/s}$ (one turn per sidereal day); local north–east–down axes sit on the surface at the launch site.

2.4 Changing coordinates: the direction cosine matrix

2.4.1 Deriving the transformation

Suppose we know the components of a vector in coordinate system A and want them in coordinate system B. Start from (2.1): $\boldsymbol{v} = v_1^A \hat{\boldsymbol{a}}_1 + v_2^A \hat{\boldsymbol{a}}_2 + v_3^A \hat{\boldsymbol{a}}_3$. The i-th component in B is the dot product with the i-th axis of B:

$$v_i^B = \hat{\boldsymbol{b}}_i \cdot \boldsymbol{v} = \sum_{j=1}^{3} (\hat{\boldsymbol{b}}_i \cdot \hat{\boldsymbol{a}}_j)\, v_j^A. \tag{2.15}$$

Collect the dot products into a matrix and this becomes one matrix multiplication:

$$\boxed{[\boldsymbol{v}]^B = \mathbf{T}^{BA}\,[\boldsymbol{v}]^A, \qquad T_{ij}^{BA} = \hat{\boldsymbol{b}}_i \cdot \hat{\boldsymbol{a}}_j = \cos(\text{angle between } \hat{\boldsymbol{b}}_i \text{ and } \hat{\boldsymbol{a}}_j).} \tag{2.16}$$

Each entry is a cosine of an angle between two axes, which is why $\mathbf{T}^{BA}$ is called the *direction cosine matrix* (DCM). Read the superscript from right to left: $\mathbf{T}^{BA}$ takes A coordinates and produces B coordinates.

2.4.2 Properties that make life easy

Proposition 2.4. *Every direction cosine matrix is* orthogonal, $\mathbf{T}^{\mathsf{T}}\mathbf{T} = \mathbf{I}$, *and has* $\det \mathbf{T} = +1$. *Hence its inverse is just its transpose:*

$$\mathbf{T}^{AB} = \left(\mathbf{T}^{BA}\right)^{-1} = \left(\mathbf{T}^{BA}\right)^{\mathsf{T}}. \tag{2.17}$$

Proof. The i-th row of $\mathbf{T}^{BA}$ is $\hat{\boldsymbol{b}}_i$ written in A coordinates. The rows are therefore three perpendicular unit vectors, so the dot product of row i with row k is 1 if $i = k$ and 0 otherwise. That is exactly $\mathbf{T}\mathbf{T}^{\mathsf{T}} = \mathbf{I}$, and for square matrices this implies $\mathbf{T}^{\mathsf{T}}\mathbf{T} = \mathbf{I}$ too. The determinant is $+1$ rather than -1 because both coordinate systems are right-handed. □

This is a gift: inverting a general 3×3 matrix takes work, but inverting a rotation is just a transpose. Orthogonal matrices also preserve lengths and angles: $\|\mathbf{T}v\|^2 = v^\mathsf{T}\mathbf{T}^\mathsf{T}\mathbf{T}v = v^\mathsf{T}v$.

Chaining. To go from A to C through B, apply one after the other:

$$\mathbf{T}^{CA} = \mathbf{T}^{CB}\,\mathbf{T}^{BA}. \tag{2.18}$$

The inner letters match and "cancel", which makes it easy to check you have the order right.

2.4.3 The three elementary rotations

Any change of axes can be built from rotations about one axis at a time. If the new axes are the old ones turned by an angle α about the old x-, y- or z-axis, the coordinate transformations are

$$\mathbf{R}_1(\alpha) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\alpha & \sin\alpha \\ 0 & -\sin\alpha & \cos\alpha \end{bmatrix},\ \mathbf{R}_2(\alpha) = \begin{bmatrix} \cos\alpha & 0 & -\sin\alpha \\ 0 & 1 & 0 \\ \sin\alpha & 0 & \cos\alpha \end{bmatrix},\ \mathbf{R}_3(\alpha) = \begin{bmatrix} \cos\alpha & \sin\alpha & 0 \\ -\sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{2.19}$$

The axis you rotate about keeps its component (the row and column with the 1). A check on the signs: the wind example of Fig. 2.1 is a rotation by $\psi = 30^\circ$ about the vertical. With (north, east) components $(0, 10)$, the top-left 2×2 block of $\mathbf{R}_3(30^\circ)$ gives $(10\sin 30^\circ,\ 10\cos 30^\circ) = (5,\ 8.66)$, as before.

> **Worked example**: Order matters. Take 90° about x followed by 90° about the new y, and the other way round. Rounding $\cos 90^\circ = 0$ and $\sin 90^\circ = 1$:
>
> $$\mathbf{R}_2(90^\circ)\mathbf{R}_1(90^\circ) = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}, \qquad \mathbf{R}_1(90^\circ)\mathbf{R}_2(90^\circ) = \begin{bmatrix} 0 & 0 & -1 \\ 1 & 0 & 0 \\ 0 & -1 & 0 \end{bmatrix}.$$
>
> Different matrices, so different final orientations. Large rotations do not add up like vectors. (Small rotations almost do, which is why linearised models work.)

2.5 Euler angles, and why they fail on the launch pad

2.5.1 Yaw, pitch and roll

The most intuitive way to describe the attitude of a vehicle is with three angles, applied in a fixed order to go from the local-level axes G (north, east, down) to the body axes B. The aerospace standard is the "3-2-1" sequence:

1. turn by the **yaw** (heading) angle ψ about the down axis (z);
2. then by the **pitch** angle θ about the new right axis (y);
3. then by the **roll** angle ϕ about the new nose axis (x).

Chaining the elementary rotations, with the first rotation on the right:

$$\mathbf{T}^{BG} = \mathbf{R}_1(\phi)\,\mathbf{R}_2(\theta)\,\mathbf{R}_3(\psi). \tag{2.20}$$

Multiplying out, and writing c for cosine and s for sine,

$$\mathbf{T}^{BG} = \begin{bmatrix} c\theta\,c\psi & c\theta\,s\psi & -s\theta \\ s\phi\,s\theta\,c\psi - c\phi\,s\psi & s\phi\,s\theta\,s\psi + c\phi\,c\psi & s\phi\,c\theta \\ c\phi\,s\theta\,c\psi + s\phi\,s\psi & c\phi\,s\theta\,s\psi - s\phi\,c\psi & c\phi\,c\theta \end{bmatrix}. \tag{2.21}$$

Going back from the matrix to the angles uses three of its entries:

$$\theta = -\arcsin T_{13}, \qquad \psi = \operatorname{atan2}(T_{12},\, T_{11}), \qquad \phi = \operatorname{atan2}(T_{23},\, T_{33}). \tag{2.22}$$

(The two-argument arctangent $\operatorname{atan2}(y, x)$ returns the angle in the correct quadrant; always use it instead of $\arctan(y/x)$.)

Worked example: A rocket shortly after lift-off. Heading $\psi = 30°$, pitch $\theta = 80°$ (nearly vertical), roll $\phi = 5°$. Evaluating (2.21) to four decimals:

$$\mathbf{T}^{BG} \approx \begin{bmatrix} 0.1504 & 0.0868 & -0.9848 \\ -0.4238 & 0.9056 & 0.0151 \\ 0.8932 & 0.4151 & 0.1730 \end{bmatrix}.$$

The first row is the nose direction in (north, east, down) coordinates: mostly "up" (-0.98 along down), tilted a little north and east. Recovering the angles with (2.22): $\theta = -\arcsin(-0.9848) = 80°$, $\psi = \operatorname{atan2}(0.0868, 0.1504) = 30°$, $\phi = \operatorname{atan2}(0.0151, 0.1730) = 5°$. Everything is recovered.

2.5.2 Gimbal lock

Now stand the rocket exactly upright: $\theta = 90°$, so $c\theta = 0$ and $s\theta = 1$. Look at rows 2 and 3 of (2.21) and use the angle-difference formulas $\sin(\phi - \psi) = s\phi\,c\psi - c\phi\,s\psi$ and $\cos(\phi - \psi) = c\phi\,c\psi + s\phi\,s\psi$:

$$\mathbf{T}^{BG}\big|_{\theta=90°} = \begin{bmatrix} 0 & 0 & -1 \\ \sin(\phi-\psi) & \cos(\phi-\psi) & 0 \\ \cos(\phi-\psi) & -\sin(\phi-\psi) & 0 \end{bmatrix}. \tag{2.23}$$

The matrix only depends on the *difference* $\phi - \psi$. Yaw and roll have become the same motion (both spin the rocket about its vertical axis), and no amount of looking at the matrix can separate them. In (2.22), ψ and ϕ become $\operatorname{atan2}(0, 0)$, which is undefined. This is *gimbal lock*.

The problem also appears in the equations that turn body angular rates (p, q, r) (what the gyroscopes measure) into Euler angle rates:

$$\begin{aligned} \dot\phi &= p + (q\sin\phi + r\cos\phi)\tan\theta, \\ \dot\theta &= q\cos\phi - r\sin\phi, \\ \dot\psi &= \frac{q\sin\phi + r\cos\phi}{\cos\theta}. \end{aligned} \tag{2.24}$$

At $\theta = 90°$, $\tan\theta$ and $1/\cos\theta$ are infinite. A simulator integrating these equations will blow up as the rocket passes through vertical — which every rocket does at lift-off.

> **Reality check.** Gimbal lock is not a physical problem; the rocket is perfectly happy pointing straight up. It is a problem with the *description*. Any description of attitude using only three numbers has a singularity somewhere; this is a mathematical fact about rotations. The cure is to use four numbers with one constraint: the quaternion.

2.6 Quaternions

In Section 2.5 we saw the problem with Euler angles: when the rocket points straight up, two of the three angles become the same motion and the equations divide by zero. Every rocket starts pointing straight up, so we need something better.

The fix is the *quaternion*. The name sounds exotic, but the idea is simple:

> **Key idea.** A quaternion is four numbers that store *one axis* and *one angle*. That is all. Everything else in this section is about how to store them cleverly, how to combine two of them, and how to use them to turn vectors.

We take it one step at a time.

2.6.1 Step 1: any turn is one turn about one axis

Pick up a book from a table. Turn it over, spin it, tilt it — do anything you like. However complicated your movement was, there is always a single straight line (an *axis*) such that one turn about that line, by one angle, would take the book from where it started to where it ended. This is *Euler's rotation theorem*.

So every orientation change can be described by just two things:

- an axis, written as a unit vector $\hat{\boldsymbol{n}}$ (an arrow of length 1), and
- an angle α: how far to turn about that axis, using the right-hand rule (thumb along $\hat{\boldsymbol{n}}$, fingers curl in the direction of turning).

2.6.2 Step 2: storing the axis and angle as four numbers

The obvious idea would be to store the three numbers of the axis and the one number of the angle. That almost works, but it has two annoying problems: when the angle is zero, the axis can be anything (so the description is not unique), and an angle of $0°$ and an angle of $360°$ look different even though they are the same orientation.

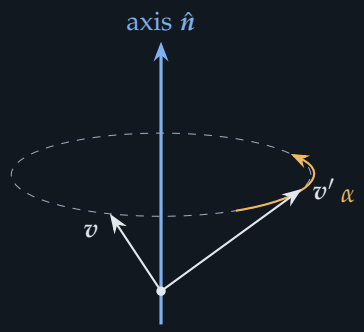


Figure 2.4: One axis, one angle. Turning about the axis $\hat{n}$ by the angle α carries the arrow v around the dashed circle to v'. Any change of orientation can be described this way.

The quaternion stores the same information in a cleverer way. It uses *half* the angle:

$$q = \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \end{bmatrix} = \begin{bmatrix} \cos(\alpha/2) \\ \hat{n}\,\sin(\alpha/2) \end{bmatrix} = \begin{bmatrix} \cos(\alpha/2) \\ n_1 \sin(\alpha/2) \\ n_2 \sin(\alpha/2) \\ n_3 \sin(\alpha/2) \end{bmatrix}. \tag{2.25}$$

The first number, q_0, is called the *scalar part*. The other three, $q_v = (q_1, q_2, q_3)$, form the *vector part*; they point along the axis. We explain why the angle is halved in Section 2.6.5. For now, just accept it.

The length is always 1. Add up the squares:

$$q_0^2 + q_1^2 + q_2^2 + q_3^2 = \cos^2\tfrac{\alpha}{2} + \left(n_1^2 + n_2^2 + n_3^2\right)\sin^2\tfrac{\alpha}{2} = \cos^2\tfrac{\alpha}{2} + 1 \cdot \sin^2\tfrac{\alpha}{2} = 1,$$

because $\hat{n}$ has length 1 and $\cos^2 + \sin^2 = 1$. That is why these are called *unit* quaternions. Four numbers with one rule connecting them leaves three free numbers — the same as three angles, but with no point where the description breaks down.

Some quaternions you should recognise.

Rotation	**Half angle**	**Quaternion** (q_0, q_1, q_2, q_3)
no rotation at all	0°	(1, 0, 0, 0)
90° about z	45°	(0.7071, 0, 0, 0.7071)
180° about z	90°	(0, 0, 0, 1)
180° about x	90°	(0, 1, 0, 0)
90° about y (pitch up by 90°)	45°	(0.7071, 0, 0.7071, 0)

Notice the pattern: q_0 is large when the turn is small, and zero for a half turn.

Reading a quaternion back. To get the axis and angle out again, reverse (2.25):

$$\alpha = 2\arccos q_0, \qquad \hat{n} = \frac{q_v}{\sin(\alpha/2)} \quad (\text{when } \alpha \neq 0). \tag{2.26}$$

For example, $\boldsymbol{q} = (0.5,\ 0.5,\ 0.5,\ 0.5)$ gives

$$\alpha = 2\arccos 0.5 = 120^\circ, \qquad \hat{\boldsymbol{n}} = \frac{(0.5,\ 0.5,\ 0.5)}{\sin 60^\circ} = \frac{(1,\ 1,\ 1)}{\sqrt{3}},$$

a 120° turn about the diagonal of a cube. We will meet this one again in a moment.

2.6.3 Step 3: combining two turns (quaternion multiplication)

If a rocket turns one way and then another, the result is a single new turn. With rotation matrices we would multiply the matrices. With quaternions we use the *quaternion product* $\odot$. For two quaternions $\boldsymbol{p} = (p_0, \boldsymbol{p}_v)$ and $\boldsymbol{q} = (q_0, \boldsymbol{q}_v)$:

$$\boldsymbol{p} \odot \boldsymbol{q} = \begin{bmatrix} p_0 q_0 - \boldsymbol{p}_v \cdot \boldsymbol{q}_v \\ p_0\, \boldsymbol{q}_v + q_0\, \boldsymbol{p}_v + \boldsymbol{p}_v \times \boldsymbol{q}_v \end{bmatrix}. \tag{2.27}$$

Read it in words:

- **new scalar part** = (scalar times scalar) minus (dot product of the two vector parts);
- **new vector part** = each scalar times the other's vector part, plus the cross product of the two vector parts.

That is it — just the dot and cross products from Section 2.1.

The order rule. If you turn by $\boldsymbol{q}_1$ first and then by $\boldsymbol{q}_2$, the combined turn is $\boldsymbol{q}_2 \odot \boldsymbol{q}_1$: the first turn goes on the *right*. This is the same as for matrices, where "first $\mathbf{R}_1$, then $\mathbf{R}_2$" is $\mathbf{R}_2\mathbf{R}_1$.

Worked example: Two quarter turns make a half turn. Turn by 90° about z, twice. Both quaternions are $\boldsymbol{q} = (c,\ 0,\ 0,\ s)$ with $c = s = 0.7071$, so $\boldsymbol{q}_v = (0, 0, s)$. Using (2.27):

- scalar part: $c \cdot c - (0,0,s) \cdot (0,0,s) = c^2 - s^2 = 0.5 - 0.5 = 0$;
- vector part: $c\,(0,0,s) + c\,(0,0,s) + (0,0,s) \times (0,0,s) = (0,\ 0,\ 2cs) + \mathbf{0} = (0,\ 0,\ 1)$. (A vector crossed with itself is always zero.)

The result is $(0, 0, 0, 1)$, which our table says is a 180° turn about z. Two quarter turns make a half turn, as they should.

Worked example: Order matters. Now turn 90° about x first, $\boldsymbol{p} = (c,\ s,\ 0,\ 0)$, and then 90° about z, $\boldsymbol{q} = (c,\ 0,\ 0,\ s)$. The combined turn is $\boldsymbol{q} \odot \boldsymbol{p}$:

- scalar part: $c \cdot c - (0,0,s) \cdot (s,0,0) = 0.5 - 0 = 0.5$;
- vector part: $c\,(s,0,0) + c\,(0,0,s) + (0,0,s) \times (s,0,0)$. The cross product is $(0,\ s^2,\ 0) = (0,\ 0.5,\ 0)$, so the vector part is $(0.5,\ 0.5,\ 0.5)$.

So $\boldsymbol{q} \odot \boldsymbol{p} = (0.5,\ 0.5,\ 0.5,\ 0.5)$: the 120° turn about $(1,1,1)/\sqrt{3}$ we decoded earlier. Doing it the other way round gives $\boldsymbol{p} \odot \boldsymbol{q} = (0.5,\ 0.5,\ -0.5,\ 0.5)$, because the cross product changes sign. A different turn: order matters, exactly as we saw with matrices in Section 2.4.

Undoing a turn: the conjugate. To undo a turn, turn back about the same axis by the same angle in the opposite direction. For a quaternion, that just means flipping the sign of the vector part. This is called the *conjugate*:

$$\boldsymbol{q}^* = (q_0,\ -\boldsymbol{q}_v). \tag{2.28}$$

Check with (2.27): the scalar part of $\boldsymbol{q} \odot \boldsymbol{q}^*$ is $q_0^2 + \boldsymbol{q}_v \cdot \boldsymbol{q}_v = 1$, and the vector part is $-q_0\boldsymbol{q}_v + q_0\boldsymbol{q}_v - \boldsymbol{q}_v \times \boldsymbol{q}_v = \mathbf{0}$. So $\boldsymbol{q} \odot \boldsymbol{q}^* = (1, 0, 0, 0)$: no rotation at all.

2.6.4 Step 4: using a quaternion to turn a vector

Now the useful part: given a quaternion $\boldsymbol{q}$ and a vector $\boldsymbol{v}$, where does $\boldsymbol{v}$ end up after the turn? The recipe has three steps.

1. Put the vector into a quaternion with a zero in front: $(0,\ \boldsymbol{v})$.
2. "Sandwich" it: multiply by $\boldsymbol{q}$ on the left and by $\boldsymbol{q}^*$ on the right.
3. The answer is the vector part of the result.

In symbols:

$$\begin{bmatrix} 0 \\ \boldsymbol{v}' \end{bmatrix} = \boldsymbol{q} \odot \begin{bmatrix} 0 \\ \boldsymbol{v} \end{bmatrix} \odot \boldsymbol{q}^*. \tag{2.29}$$

Worked example: Turning the x-axis by 90° about z, step by step. Take $\boldsymbol{q} = (c, 0, 0, s)$ with $c = s = 0.7071$, and $\boldsymbol{v} = (1, 0, 0)$.
First multiplication, $\boldsymbol{q} \odot (0, \boldsymbol{v})$:

- scalar: $c \cdot 0 - (0, 0, s) \cdot (1, 0, 0) = 0$;
- vector: $c\,(1, 0, 0) + 0 + (0, 0, s) \times (1, 0, 0) = (c, 0, 0) + (0, s, 0) = (c,\ s,\ 0)$.

So the middle result is $(0,\ c,\ s,\ 0)$.
Second multiplication, by $\boldsymbol{q}^* = (c,\ 0,\ 0,\ -s)$:

- scalar: $0 \cdot c - (c, s, 0) \cdot (0, 0, -s) = 0$;
- vector: $0 + c\,(c, s, 0) + (c, s, 0) \times (0, 0, -s) = (c^2, cs, 0) + (-s^2, cs, 0) = (c^2 - s^2,\ 2cs,\ 0) = (0,\ 1,\ 0)$.

The x-axis has been turned onto the y-axis: exactly a 90° turn about z.

The shortcut: a rotation matrix. Doing two quaternion products for every vector is slow. If we carry out the sandwich (2.29) once with symbols and collect the terms, we get an ordinary 3×3 matrix that does the same job, $\boldsymbol{v}' = \mathbf{R}(\boldsymbol{q})\,\boldsymbol{v}$:

$$\mathbf{R}(\boldsymbol{q}) = \begin{bmatrix} q_0^2 + q_1^2 - q_2^2 - q_3^2 & 2(q_1q_2 - q_0q_3) & 2(q_1q_3 + q_0q_2) \\ 2(q_1q_2 + q_0q_3) & q_0^2 - q_1^2 + q_2^2 - q_3^2 & 2(q_2q_3 - q_0q_1) \\ 2(q_1q_3 - q_0q_2) & 2(q_2q_3 + q_0q_1) & q_0^2 - q_1^2 - q_2^2 + q_3^2 \end{bmatrix}. \tag{2.30}$$

You do not need to memorise it. Notice the pattern instead: the diagonal has all four squares with different signs, and the off-diagonal entries are pairs $2(q_iq_j \pm q_0q_k)$.

For our example $\boldsymbol{q} = (0.7071, 0, 0, 0.7071)$: $q_0^2 - q_3^2 = 0$, $q_0^2 + q_3^2 = 1$ and $2q_0q_3 = 1$, so

$$\mathbf{R}(\boldsymbol{q}) = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \qquad \mathbf{R}(\boldsymbol{q}) \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix},$$

the same answer as the step-by-step sandwich.

Turning arrows versus relabelling axes. $\mathbf{R}(\boldsymbol{q})$ *moves* an arrow. A direction cosine matrix from Section 2.4 does the opposite job: it keeps the arrow still and *rewrites its numbers* in turned axes. Turning the axes one way looks, from the axes' point of view, like the arrow turning the other way. So if $\boldsymbol{q}$ describes how the body axes are turned relative to the reference axes, the coordinate transformation is the reverse turn, which for a rotation matrix is the transpose:

$$\mathbf{T}^{BA} = \mathbf{R}(\boldsymbol{q})^{\mathsf{T}}. \tag{2.31}$$

> **Reality check**: Check conventions with a known example. Books and software libraries do not all agree on quaternion conventions. Some put the scalar first, some last, and some define the product in the reverse order. None is wrong, but mixing two of them silently turns every rotation backwards. Before trusting a new library, run the 90°-about-z example above through it and check that you get $(0, 1, 0)$.

2.6.5 Why the half angle?

Here is a strange but useful fact. Replace every number in $\boldsymbol{q}$ by its negative. In (2.30), every entry is a product of *two* components, and $(-a)(-b) = ab$, so the matrix does not change at all:

$$\mathbf{R}(-\boldsymbol{q}) = \mathbf{R}(\boldsymbol{q}). \tag{2.32}$$

So $\boldsymbol{q}$ and $-\boldsymbol{q}$ describe exactly the same orientation.

The half angle explains why. Look at what happens to $q_0 = \cos(\alpha/2)$ as you keep turning about one axis:

Turn α	Half angle	q_0	Orientation
0°	0°	$+1$	where we started
180°	90°	0	half turn
360°	180°	-1	back where we started, but $\boldsymbol{q}$ has become $-\boldsymbol{q}$
720°	360°	$+1$	back where we started, and $\boldsymbol{q}$ is back too

Because of the half angle, the quaternion needs *two* full turns to come back to itself, while the orientation needs only one. So every orientation has two quaternions, $\boldsymbol{q}$ and $-\boldsymbol{q}$. Mathematicians call this a *double cover*.

You have seen this before. The qubit state of Chapter 1, $\cos\frac{\vartheta}{2}|0\rangle + e^{i\varphi}\sin\frac{\vartheta}{2}|1\rangle$, also uses half angles, for the same mathematical reason. Unit quaternions and single-qubit gates are two

faces of the same group, which is one more reason the rocket side and the quantum side of this book fit together.

Why this matters in software. When a filter compares two attitude quaternions, say the estimate and a measurement, they might be the same orientation written with opposite signs. Compared naively, they look as if the vehicle has turned by almost 360°. The simple fix: if the scalar part of $q_1^* \odot q_2$ is negative, flip the sign of q_2 before comparing.

2.6.6 How a quaternion changes over time

In flight, the gyroscopes measure the body's angular velocity ω (in body axes, in rad/s). The simulator and the navigation software must turn this into an updated quaternion at every time step. Here is where the formula comes from.

A tiny turn. In a very short time Δt, the body turns about the direction of ω by the small angle $\|\omega\|\,\Delta t$. For a small angle, $\cos(\text{small}/2) \approx 1$ and $\sin(\text{small}/2) \approx \text{small}/2$, so by (2.25) this tiny turn is the quaternion

$$\delta q \approx \begin{bmatrix} 1 \\ \frac{1}{2}\,\omega\,\Delta t \end{bmatrix}. \tag{2.33}$$

Add it on. Because ω is measured in body axes, the tiny turn is applied *after* the current attitude, relative to the body, which puts it on the right: $q(t+\Delta t) = q(t) \odot \delta q$. Then

$$\frac{q(t+\Delta t) - q(t)}{\Delta t} = \frac{q \odot \left[(1,\,\mathbf{0}) + (0,\,\tfrac{1}{2}\omega\Delta t)\right] - q}{\Delta t} = \tfrac{1}{2}\, q \odot \begin{bmatrix} 0 \\ \omega \end{bmatrix},$$

using $q \odot (1, \mathbf{0}) = q$. Letting $\Delta t \to 0$:

$$\boxed{\dot{q} = \tfrac{1}{2}\, q \odot \begin{bmatrix} 0 \\ \omega \end{bmatrix}.} \tag{2.34}$$

Written out in full. Expanding with (2.27) gives four ordinary equations that you can type straight into code:

$$\begin{aligned} \dot{q}_0 &= -\tfrac{1}{2}\,(q_1\omega_1 + q_2\omega_2 + q_3\omega_3), \\ \dot{q}_1 &= \tfrac{1}{2}\,(q_0\omega_1 + q_2\omega_3 - q_3\omega_2), \\ \dot{q}_2 &= \tfrac{1}{2}\,(q_0\omega_2 + q_3\omega_1 - q_1\omega_3), \\ \dot{q}_3 &= \tfrac{1}{2}\,(q_0\omega_3 + q_1\omega_2 - q_2\omega_1). \end{aligned} \tag{2.35}$$

This is exactly the matrix equation $\dot{q} = \frac{1}{2}\blacksquare(\omega)q$ we used in Chapter 1.

> **Worked example**: One time step. The vehicle starts level, $q = (1,0,0,0)$, and pitches up at $\omega = (0,\ 0.1,\ 0)$ rad/s. Then $\dot{q} = (0,\ 0,\ 0.05,\ 0)$. After one step of $\Delta t = 0.01$ s:
>
> $$q \approx (1,\ 0,\ 0.0005,\ 0).$$

The exact answer is a turn of $0.1 \times 0.01 = 0.001$ rad about y, which is

$$(\cos 0.0005,\ 0,\ \sin 0.0005,\ 0) = (0.99999988,\ 0,\ 0.0005,\ 0).$$

The step is very close. The small difference in q_0 is why simulators re-normalise $\boldsymbol{q}$ to length 1 after each step.

Look at the four equations again: there is no division and no tangent anywhere. Nothing can blow up, whichever way the rocket points. That is the whole reason flight software uses quaternions.

2.6.7 Which description to use

Description	Numbers	Good at	Weak at
Euler angles	3	easy for people to read	breaks down at $\theta = \pm 90^\circ$ (gimbal lock)
Direction cosine matrix	9	changing axes quickly	9 numbers for 3 pieces of information; slowly drifts away from a true rotation in a simulation
Quaternion	4	integrating in time; no singularity	less intuitive; $\boldsymbol{q}$ and $-\boldsymbol{q}$ mean the same thing

Table 2.2: Three ways to describe attitude. Flight software usually stores and integrates a quaternion, converts it to a direction cosine matrix when it needs to change axes, and converts it to Euler angles only for display.

2.7 Derivatives when the axes are rotating

2.7.1 The problem

The derivative of a vector depends on the frame you watch it from. The launch-pad example of Section 2.3 showed this: the pad's position does not change as seen from the Earth, but changes at 408 m/s as seen from space. Newton's law $\boldsymbol{F} = m\boldsymbol{a}$ holds only when $\boldsymbol{a}$ is the acceleration seen from an inertial frame. But we usually know the motion relative to the rotating Earth. We need a formula that connects the two.

2.7.2 The transport theorem

Let frame B rotate relative to frame A with angular velocity $\boldsymbol{\omega}^{BA}$. Write a vector using B's axes, $\boldsymbol{v} = \sum_i v_i \hat{\boldsymbol{b}}_i$, and differentiate as seen from A. The product rule gives two kinds of terms:

$$\frac{\mathrm{d}^A \boldsymbol{v}}{\mathrm{d}t} = \underbrace{\sum_i \dot{v}_i \, \hat{\boldsymbol{b}}_i}_{\text{change seen from } B} + \underbrace{\sum_i v_i \, \frac{\mathrm{d}^A \hat{\boldsymbol{b}}_i}{\mathrm{d}t}}_{\text{the axes themselves turning}} . \tag{2.36}$$

The first sum is the derivative an observer riding on B would see, $\mathrm{d}^B \boldsymbol{v}/\,\mathrm{d}t$. For the second, each axis $\hat{\boldsymbol{b}}_i$ is a unit vector fixed in B, so seen from A its tip moves in a circle about $\boldsymbol{\omega}^{BA}$, with velocity $\boldsymbol{\omega}^{BA} \times \hat{\boldsymbol{b}}_i$. (A point at $\boldsymbol{r}$ on a body rotating with $\boldsymbol{\omega}$ moves with velocity $\boldsymbol{\omega} \times \boldsymbol{r}$.) So the second sum is $\sum_i v_i \, \boldsymbol{\omega}^{BA} \times \hat{\boldsymbol{b}}_i = \boldsymbol{\omega}^{BA} \times \boldsymbol{v}$, and

$$\boxed{\frac{\mathrm{d}^A \boldsymbol{v}}{\mathrm{d}t} = \frac{\mathrm{d}^B \boldsymbol{v}}{\mathrm{d}t} + \boldsymbol{\omega}^{BA} \times \boldsymbol{v}.} \tag{2.37}$$

This is the *transport theorem* (Zipfel calls the frame-specific derivative the *rotational time derivative* and builds it as a tensor operator; Appendix D of his book gives the full treatment). It holds for any vector: position, velocity, angular momentum.

2.7.3 Coriolis and centripetal accelerations

Apply (2.37) to the position $\boldsymbol{r}$ of a vehicle, from the inertial frame I and the Earth frame E, with the Earth's constant rotation $\boldsymbol{\omega}_E$:

$$\boldsymbol{v}^I = \boldsymbol{v}^E + \boldsymbol{\omega}_E \times \boldsymbol{r}, \tag{2.38}$$

where $\boldsymbol{v}^I = \mathrm{d}^I \boldsymbol{r}/\,\mathrm{d}t$ and $\boldsymbol{v}^E = \mathrm{d}^E \boldsymbol{r}/\,\mathrm{d}t$. Apply it again to $\boldsymbol{v}^I$, and use $\mathrm{d}^E(\boldsymbol{\omega}_E \times \boldsymbol{r})/\,\mathrm{d}t = \boldsymbol{\omega}_E \times \boldsymbol{v}^E$ because $\boldsymbol{\omega}_E$ is constant:

$$\begin{aligned} \boldsymbol{a}^I &= \frac{\mathrm{d}^E \boldsymbol{v}^I}{\mathrm{d}t} + \boldsymbol{\omega}_E \times \boldsymbol{v}^I && (2.39)\\ &= \left(\boldsymbol{a}^E + \boldsymbol{\omega}_E \times \boldsymbol{v}^E\right) + \boldsymbol{\omega}_E \times \left(\boldsymbol{v}^E + \boldsymbol{\omega}_E \times \boldsymbol{r}\right) && (2.40)\\ &= \boldsymbol{a}^E + \underbrace{2\,\boldsymbol{\omega}_E \times \boldsymbol{v}^E}_{\text{Coriolis}} + \underbrace{\boldsymbol{\omega}_E \times (\boldsymbol{\omega}_E \times \boldsymbol{r})}_{\text{centripetal}} . && (2.41) \end{aligned}$$

Newton's law uses $\boldsymbol{a}^I$. So if we want to simulate in Earth-relative terms, we must write

$$m\,\boldsymbol{a}^E = \boldsymbol{F} - 2m\,\boldsymbol{\omega}_E \times \boldsymbol{v}^E - m\,\boldsymbol{\omega}_E \times (\boldsymbol{\omega}_E \times \boldsymbol{r}). \tag{2.42}$$

The two extra terms behave like forces, but nothing is pushing; they appear because our reference is turning.

> **Worked example**: How big are they?. *Centripetal*, at the equator: $\omega_E^2 R = (7.292 \times 10^{-5})^2 \times 6.378 \times 10^6 \approx 0.034\,\mathrm{m/s^2}$, about 0.35% of g. Small, but it is part of why measured gravity is weaker at the equator.

> *Coriolis*, for a vehicle moving at 2000 m/s perpendicular to the Earth's axis: $2\omega_E v = 2 \times 7.292 \times 10^{-5} \times 2000 \approx 0.29\,\mathrm{m/s^2}$. That also looks small. But over 100 s it would push the vehicle sideways by about $\tfrac{1}{2}at^2 = \tfrac{1}{2} \times 0.29 \times 100^2 \approx 1.46\,\mathrm{km}$. For a vehicle trying to land on a pad a few tens of metres wide, leaving out the Coriolis term is not an option.

2.8 Tensors and the inertia tensor

2.8.1 What a tensor is

The word *tensor* sounds frightening, but for this book it means something simple: a physical quantity that, like a vector, exists independently of the axes, but whose components are written as a matrix and change in a definite way when the axes change.

- A scalar (mass, temperature) is a tensor of *rank 0*: its value does not depend on the axes at all.
- A vector is a tensor of *rank 1*: three components that transform as $[v]^B = \mathbf{T}^{BA}[v]^A$.
- A *rank-2* tensor relates one vector to another, $\boldsymbol{y} = \mathcal{J}\boldsymbol{x}$, and is written as a 3×3 matrix in each coordinate system.

How do the components of a rank-2 tensor transform? Write the relation in both systems, $[\boldsymbol{y}]^A = [\mathcal{J}]^A[\boldsymbol{x}]^A$ and $[\boldsymbol{y}]^B = [\mathcal{J}]^B[\boldsymbol{x}]^B$. Substitute $[\boldsymbol{y}]^B = \mathbf{T}^{BA}[\boldsymbol{y}]^A$ and $[\boldsymbol{x}]^A = \mathbf{T}^{AB}[\boldsymbol{x}]^B = (\mathbf{T}^{BA})^{\mathsf{T}}[\boldsymbol{x}]^B$:

$$[\boldsymbol{y}]^B = \mathbf{T}^{BA}[\mathcal{J}]^A(\mathbf{T}^{BA})^{\mathsf{T}}[\boldsymbol{x}]^B \quad \Longrightarrow \quad \boxed{[\mathcal{J}]^B = \mathbf{T}^{BA}\,[\mathcal{J}]^A\,(\mathbf{T}^{BA})^{\mathsf{T}}.} \tag{2.43}$$

A transformation matrix on the left and its transpose on the right: that is the signature of a rank-2 tensor.

2.8.2 The inertia tensor

The most important rank-2 tensor in flight dynamics links angular velocity to angular momentum, $\boldsymbol{H} = \mathbf{J}\boldsymbol{\omega}$. For a body made of small masses $\mathrm{d}m$ at positions $\boldsymbol{r} = (x, y, z)$ from the centre of mass,

$$\mathbf{J} = \int \left(\|\boldsymbol{r}\|^2 \mathbf{I} - \boldsymbol{r}\boldsymbol{r}^{\mathsf{T}} \right) \mathrm{d}m = \begin{bmatrix} \int (y^2+z^2)\,\mathrm{d}m & -\int xy\,\mathrm{d}m & -\int xz\,\mathrm{d}m \\ -\int xy\,\mathrm{d}m & \int (x^2+z^2)\,\mathrm{d}m & -\int yz\,\mathrm{d}m \\ -\int xz\,\mathrm{d}m & -\int yz\,\mathrm{d}m & \int (x^2+y^2)\,\mathrm{d}m \end{bmatrix}. \tag{2.44}$$

The diagonal entries are the *moments of inertia*: how hard it is to spin the body about each axis. Mass far from an axis counts a lot (it enters squared). The off-diagonal entries are the *products of inertia*; they are zero when the body is symmetric about the axes.

> **Worked example**: A launch vehicle as a cylinder. Model a fully fuelled vehicle as a uniform solid cylinder of mass $m = 500\,\mathrm{t}$, radius $R = 1.85\,\mathrm{m}$ and length $L = 70\,\mathrm{m}$, with

x along its length. Standard results for a cylinder give

$$J_{xx} = \tfrac{1}{2}mR^2 = \tfrac{1}{2}\times 5\times 10^5 \times 1.85^2 \approx 8.6\times 10^5\,\mathrm{kg\,m^2},$$

$$J_{yy} = J_{zz} = \tfrac{1}{12}m\left(3R^2 + L^2\right) = \tfrac{1}{12}\times 5\times 10^5\times(10.27 + 4900) \approx 2.05\times 10^8\,\mathrm{kg\,m^2}.$$

Pitching the vehicle is about 240 times harder than rolling it. This is why roll can be controlled with small thrusters or by differential gimballing, while pitch and yaw need the main engine's full thrust and a long lever arm.

Principal axes. $\mathbf{J}$ is symmetric and positive definite, so by the spectral theorem there is a set of perpendicular axes in which it is diagonal. These are the *principal axes*, and the diagonal entries are the *principal moments*.

Worked example: Finding principal axes. In a planar problem, suppose $\mathbf{J} = \begin{bmatrix} 10 & 2 \\ 2 & 7 \end{bmatrix}$ (in some unit). The characteristic polynomial is $(10-\lambda)(7-\lambda) - 4 = \lambda^2 - 17\lambda + 66 = 0$, so $\lambda = (17 \pm \sqrt{289 - 264})/2 = (17 \pm 5)/2$, giving principal moments 11 and 6. For $\lambda = 11$: $(10-11)x + 2y = 0$, so $y = x/2$ and the axis is $(2,1)/\sqrt{5}$, at an angle of $\arctan(1/2) \approx 26.6^\circ$. For $\lambda = 6$ the axis is $(-1,2)/\sqrt{5}$, perpendicular to the first, as the theorem promised. Rotating the axes by 26.6° using (2.43) turns $\mathbf{J}$ into $\mathrm{diag}(11, 6)$.

The parallel-axis theorem. As propellant burns, the centre of mass moves, and we need the inertia about a new point. If $\mathbf{J}_C$ is the inertia about the centre of mass and the new reference point is displaced by $\boldsymbol{d}$,

$$\mathbf{J}_O = \mathbf{J}_C + m\left(\|\boldsymbol{d}\|^2\,\mathbf{I} - \boldsymbol{d}\,\boldsymbol{d}^{\mathsf{T}}\right). \tag{2.45}$$

It has the same shape as (2.44), as if all the mass sat at the centre of mass. In Chapter 3 we use this to update $\mathbf{J}(t)$ at every time step.

2.9 Where on Earth? Geodetic coordinates

2.9.1 The Earth is not a sphere

The Earth's rotation makes it bulge at the equator. Navigation systems, including GPS, use the WGS-84 ellipsoid:

$$a = 6\,378\,137\,\mathrm{m}\ \text{(equatorial radius)}, \qquad f = \frac{1}{298.257223563}\ \text{(flattening)}, \qquad e^2 = f(2-f) \approx 0.00669438. \tag{2.46}$$

The polar radius is $a(1-f) \approx 6\,356\,752\,\mathrm{m}$, about 21 km less than the equatorial radius.

A position is given by *geodetic latitude* φ (the angle between the local vertical and the equator plane), longitude λ and height h above the ellipsoid. Because the Earth is flattened, the local vertical does not pass through the Earth's centre. The *geocentric* latitude φ_c (the angle of the

line from the centre) is slightly smaller: $\tan\varphi_c = (1-e^2)\tan\varphi$. The largest difference is about 0.19°, near 45° latitude — roughly 21 km on the ground. Mix the two up and your landing site moves by a city's width.

2.9.2 From latitude, longitude and height to ECEF

Define the *prime vertical radius* $N(\varphi) = a/\sqrt{1-e^2\sin^2\varphi}$. Then the Earth-fixed coordinates are

$$\begin{aligned} x &= (N+h)\cos\varphi\cos\lambda, \\ y &= (N+h)\cos\varphi\sin\lambda, \\ z &= \big(N(1-e^2)+h\big)\sin\varphi. \end{aligned} \tag{2.47}$$

Worked example: Locating a launch pad. Take $\varphi = 28.6083^\circ$, $\lambda = -80.6041^\circ$ (west), $h = 3\,\text{m}$, close to a pad at Cape Canaveral. Then $\sin^2\varphi \approx 0.2293$ and

$$N = \frac{6\,378\,137}{\sqrt{1-0.00669438\times 0.2293}} \approx 6\,383\,037\,\text{m}.$$

Substituting into (2.47):

$$[\boldsymbol{r}]^E \approx (914\,844,\ -5\,528\,577,\ 3\,035\,861)\ \text{m}, \qquad \|\boldsymbol{r}\| \approx 6\,373\,269\,\text{m}.$$

The pad is about 4.9 km closer to the Earth's centre than the equatorial radius, because of the flattening.

2.9.3 From Earth-fixed to local north–east–down

The local-level axes at latitude φ and longitude λ are reached from ECEF by a rotation λ about z, then a rotation of $-(90^\circ+\varphi)$ about the new y. The result is

$$\mathbf{T}^{GE} = \begin{bmatrix} -\sin\varphi\cos\lambda & -\sin\varphi\sin\lambda & \cos\varphi \\ -\sin\lambda & \cos\lambda & 0 \\ -\cos\varphi\cos\lambda & -\cos\varphi\sin\lambda & -\sin\varphi \end{bmatrix}. \tag{2.48}$$

Each row is a local axis written in ECEF coordinates: north, east and down. As a check, transform the Earth's rotation vector $[\boldsymbol{\omega}_E]^E = (0,0,\omega_E)$: the result is $(\omega_E\cos\varphi,\ 0,\ -\omega_E\sin\varphi)$. At the Cape that is $(6.40, 0, -3.49)\times 10^{-5}\,\text{rad/s}$: the Earth's spin seen locally has a northward part and an upward part (negative "down"), exactly as it should in the northern hemisphere.

2.9.4 Gravity, to first order

For now we use the simplest gravity model, a point mass:

$$\boldsymbol{g}(\boldsymbol{r}) = -\frac{\mu}{\|\boldsymbol{r}\|^3}\,\boldsymbol{r}, \qquad \mu = 3.986\,004\,418\times 10^{14}\,\text{m}^3/\text{s}^2. \tag{2.49}$$

At the equatorial radius this gives $9.80\,\text{m/s}^2$; at 200 km altitude, $9.21\,\text{m/s}^2$. Gravity in low orbit is only about 6% weaker than on the ground; astronauts float because they are falling,

not because gravity has gone. In Chapter 4 we add the main correction for the Earth's bulge, the J_2 term.

2.10 Numbers inside a computer

2.10.1 Floating point

Computers store real numbers in *floating point*: a fixed number of significant binary digits and an exponent. Double precision (`float64`, the default in Python and NumPy) keeps about 16 significant decimal digits, with relative spacing $\epsilon \approx 2.2 \times 10^{-16}$. Single precision (`float32`) keeps only about 7.

> **Worked example**: Never store positions in single precision. An ECEF position is around 6.4×10^6 m. In `float32`, numbers between 2^{22} and 2^{23} (about 4.2 and 8.4 million) are spaced exactly 0.5 m apart. So a position stored in single precision can only take values on a half-metre grid — and velocities computed by differencing such positions are garbage. In `float64` the spacing at the same size is about 1×10^{-9} m. Use double precision for every navigation quantity.

2.10.2 Solving linear systems

When you need $\boldsymbol{x}$ from $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, do not compute $\mathbf{A}^{-1}$ and then multiply. Use a solver (`numpy.linalg.solve`), which factorises $\mathbf{A}$ and is both faster and more accurate. For symmetric positive definite matrices such as covariances, the Cholesky factorisation $\mathbf{A} = \mathbf{L}\mathbf{L}^{\mathsf{T}}$ is the standard tool. And keep the condition number in mind: with $\kappa \approx 10^4$ in double precision you lose about 4 of your 16 digits, which is usually fine; with $\kappa \approx 10^{12}$ you have only 4 left.

2.10.3 Trying it in Python

The code below builds the 3-2-1 DCM, checks that it is orthogonal, and checks the quaternion example of Section 2.6 two ways. It needs only NumPy (`pip install numpy`); Labbe's book (Appendix A) explains a full scientific Python setup.

```
import numpy as np

def R1(a): # coordinate transformation, rotation about x
   c, s = np.cos(a), np.sin(a)
   return np.array([[1, 0, 0], [0, c, s], [0, -s, c]])

def R2(a): # about y
   c, s = np.cos(a), np.sin(a)
   return np.array([[c, 0, -s], [0, 1, 0], [s, 0, c]])

def R3(a): # about z
```

```
    c, s = np.cos(a), np.sin(a)
    return np.array([[c, s, 0], [-s, c, 0], [0, 0, 1]])

def dcm_321(psi, theta, phi):
    """T^{BG}: local-level (NED) coordinates -> body coordinates."""
    return R1(phi) @ R2(theta) @ R3(psi)

def quat_mult(p, q):
    p0, pv = p[0], p[1:]
    q0, qv = q[0], q[1:]
    return np.concatenate(([p0*q0 - pv @ qv],
                           p0*qv + q0*pv + np.cross(pv, qv)))

def quat_to_rot(q):
    q0, q1, q2, q3 = q
    return np.array([
        [q0**2+q1**2-q2**2-q3**2, 2*(q1*q2-q0*q3), 2*(q1*q3+q0*q2)],
        [2*(q1*q2+q0*q3), q0**2-q1**2+q2**2-q3**2, 2*(q2*q3-q0*q1)],
        [2*(q1*q3-q0*q2), 2*(q2*q3+q0*q1), q0**2-q1**2-q2**2+q3**2]])

T = dcm_321(np.radians(30), np.radians(80), np.radians(5))
print("orthogonal:", np.allclose(T.T @ T, np.eye(3)), " det =", round(np
    .linalg.det(T), 12))
print("pitch back:", np.degrees(-np.arcsin(T[0, 2])))

q = np.array([np.cos(np.pi/4), 0, 0, np.sin(np.pi/4)]) # 90 deg about z
print("x-axis rotated:", np.round(quat_to_rot(q) @ [1, 0, 0], 12))
v = np.array([0, 1, 0, 0]) # pure quaternion for x
q_conj = q * np.array([1, -1, -1, -1])
print("q * v * q^-1 :", np.round(quat_mult(quat_mult(q, v), q_conj)[1:],
     12))
```

Running it prints:

```
orthogonal: True det = 1.0
pitch back: 79.99999999999999
x-axis rotated: [0. 1. 0.]
q * v * q^-1 : [0. 1. 0.]
```

Note the last digits of "pitch back": 80 comes back as 79.99999999999999. That is floating point at work, an error of about 10^{-14} degrees, and nothing to worry about.

2.11 Random numbers for simulation

Every Monte Carlo run in this book — every random wind gust, every noisy gyroscope reading, every engine that comes off the production line slightly stronger or weaker than the drawing says — starts with the same thing: a computer producing random numbers. This section explains where those numbers come from and how to shape them into any distribution we need. It covers Sections 2.1–2.3 of Robert and Casella's *Monte Carlo Statistical Methods*, the

computational-statistics companion to this book, with code in both R and Python.

The whole section rests on one simple idea:

> **Key idea.** Computers are good at producing one kind of randomness: numbers spread evenly between 0 and 1, the *uniform distribution* $\mathcal{U}(0,1)$. Everything else — normal noise, wind speeds, random directions, random attitudes — is built from uniform numbers by a clever transformation. Learn a few transformations and you can simulate almost anything.

We will use the language of probability throughout. If it is new to you, three words are enough:

- a *random variable* X is a number whose value is decided by chance;
- its *density* $f(x)$ shows where values are likely (high) or unlikely (low), and the total area under it is 1;
- its *cumulative distribution function* (CDF), $F(x) = P(X \le x)$, is the probability of getting a value of x or less. It rises from 0 to 1 as x increases.

2.11.1 Uniform random numbers

A computer cannot flip a coin. What it does instead is run a fixed formula that produces a long sequence of numbers that *look* random and pass every statistical test we throw at them. These are *pseudo-random* numbers. The oldest and simplest formula is the *linear congruential generator*:

$$x_{k+1} = (a\,x_k + c) \bmod m, \qquad u_k = \frac{x_k}{m}, \tag{2.50}$$

where "mod m" means "the remainder after dividing by m". The starting value x_0 is the *seed*.

> **Worked example**: A tiny generator you can run by hand. Take $a = 5$, $c = 3$, $m = 16$ and seed $x_0 = 7$. Then $x_1 = (5 \times 7 + 3) \bmod 16 = 38 \bmod 16 = 6$, then $x_2 = (5 \times 6 + 3) \bmod 16 = 33 \bmod 16 = 1$, and so on:
>
> $$6,\ 1,\ 8,\ 11,\ 10,\ 5,\ 12,\ 15,\ 14,\ 9,\ 0,\ 3,\ 2,\ 13,\ 4,\ 7,\ 6,\ 1,\ \ldots$$
>
> Every number from 0 to 15 appears exactly once before the sequence repeats: the *period* is 16. Dividing by 16 gives sixteen evenly spread values in $[0,1)$. Real generators work the same way with enormous numbers; R's default (the Mersenne Twister) has a period of $2^{19937} - 1$, and NumPy's default (PCG64) has a period of 2^{128}.

The seed matters in engineering. *The same seed always gives the same sequence.* That is what makes a Monte Carlo study repeatable: if run number 4 127 of a dispersion analysis crashes the vehicle, you can run exactly that case again with the same seed and watch what happened.

R

```r
lcg <- function(n, seed, a = 5, c = 3, m =
     16) {
  x <- numeric(n)
  for (k in 1:n) {
    seed <- (a * seed + c) %% m
    x[k] <- seed
  }
  x
}
lcg(16, seed = 7)
set.seed(2026); runif(3)
set.seed(2026); runif(3) # same again
```

```
 [1] 6 1 8 11 10 5 12 15 14 9 0 3 2 13 4 7
[1] 0.6986735 0.5565305 0.1401400
[1] 0.6986735 0.5565305 0.1401400
```

Python

```python
import numpy as np

def lcg(n, seed, a=5, c=3, m=16):
    x = []
    for _ in range(n):
        seed = (a * seed + c) % m
        x.append(seed)
    return x

print(lcg(16, seed=7))
rng = np.random.default_rng(2026)
print(rng.random(3))
rng = np.random.default_rng(2026)
print(rng.random(3)) # same again
```

```
[6, 1, 8, 11, 10, 5, 12, 15, 14, 9, 0, 3,
    2, 13, 4, 7]
[0.17893481 0.63991317 0.4672684 ]
[0.17893481 0.63991317 0.4672684 ]
```

Both languages reproduce the hand calculation exactly, and both repeat themselves perfectly when the seed is reset. Notice that R and Python give *different* numbers for the same seed, because they use different generators. A seed is only meaningful together with the generator it was used with.

2.11.2 The inverse transform

Suppose we want random numbers with some CDF F, and we can produce uniform numbers U. The *inverse transform* says: feed the uniform number backwards through the CDF.

Theorem 2.5 (Inverse transform)**.** *Let F be a continuous, increasing CDF with inverse F^{-1}. If $U \sim \mathcal{U}(0,1)$, then $X = F^{-1}(U)$ has CDF F.*

Proof. We need $P(X \leq x) = F(x)$. Take it one step at a time:

$$P(X \leq x) = P\big(F^{-1}(U) \leq x\big) = P\big(U \leq F(x)\big) = F(x).$$

The middle step applies F to both sides of the inequality; because F is increasing, the inequality keeps its direction. The last step is the defining property of a uniform number: the chance that U is at most some value c between 0 and 1 is exactly c. □

The picture. The CDF climbs from 0 to 1. Where the density is high, the CDF climbs steeply, so a whole band of u values maps onto a narrow range of x — many samples land there. Where the density is low, the CDF is flat, and few u values map there. The inverse transform automatically puts the samples where the probability is.

When F has jumps or flat parts, the same idea works with the *generalised inverse* $F^{-}(u) = \min\{x : F(x) \geq u\}$, "the first x at which the CDF reaches u". Robert and Casella state the theorem in this general form.

Worked example: Time between gusts: the exponential distribution. Model the time between strong gusts as exponential with a mean of 4 s, rate $\lambda = 1/4$ per second: $F(x) = 1 - e^{-\lambda x}$. To invert, set $u = F(x)$ and solve for x:

$$u = 1 - e^{-\lambda x} \implies e^{-\lambda x} = 1 - u \implies x = -\frac{\ln(1-u)}{\lambda} = -4\ln(1-u).$$

A uniform draw $u = 0.3$ gives $x = -4\ln 0.7 \approx 1.43$ s. (Since $1 - U$ is also uniform, $-4\ln U$ works just as well.)

Worked example: Peak wind speed: the Rayleigh distribution. If the north and east components of the wind are independent and normal with mean 0 and standard deviation σ, the wind *speed* $V = \sqrt{w_N^2 + w_E^2}$ has the Rayleigh distribution, $F(v) = 1 - e^{-v^2/(2\sigma^2)}$ (we prove this in Section 2.11.3). Invert it:

$$u = 1 - e^{-v^2/(2\sigma^2)} \implies v^2 = -2\sigma^2 \ln(1-u) \implies v = \sigma\sqrt{-2\ln(1-u)}.$$

With $\sigma = 7$ m/s and $u = 0.5$, $v = 7\sqrt{2\ln 2} \approx 8.24$ m/s: half of all simulated peak winds are below this, so it is the median.

Worked example: Engine condition: a discrete distribution. For fault-management studies (Chapter 12) we might draw an engine condition: nominal with probability 0.95, low thrust with 0.04, engine-out with 0.01. The CDF is a staircase with steps at 0.95, 0.99 and 1. The inverse transform becomes "draw u, and see which step it falls under": $u < 0.95$ is nominal, $0.95 \le u < 0.99$ is low thrust, and $u \ge 0.99$ is engine-out.

R

```r
set.seed(1)
n <- 100000
u <- runif(n)
gap <- -4 * log(1 - u)  # Exp, mean 4 s
wind <- 7 * sqrt(-2 * log(1 - u))  # Rayleigh
c(mean(gap), median(wind))
```

```
[1] 4.003320 8.240947
```

```r
p <- c(nominal = 0.95, low_thrust = 0.04, engine_out = 0.01)
set.seed(3)
u <- runif(100000)
k <- findInterval(u, cumsum(p)) + 1
table(names(p)[k]) / length(u)
```

```
engine_out low_thrust nominal
   0.01014 0.03944 0.95042
```

Python

```python
import numpy as np
rng = np.random.default_rng(1)
n = 100_000
u = rng.random(n)
gap = -4 * np.log(1 - u)  # Exp
wind = 7 * np.sqrt(-2 * np.log(1 - u))
print(gap.mean(), np.median(wind))
```

```
3.9979275648349137 8.235971772142761
```

```python
import numpy as np
names = ["nominal", "low_thrust", "engine_out"]
p = np.array([0.95, 0.04, 0.01])
rng = np.random.default_rng(3)
u = rng.random(100_000)
cs = np.cumsum(p)
k = np.searchsorted(cs, u, side="right")
for nm, v in zip(names, np.bincount(k) / u.size):
    print(nm, v)
```

```
nominal 0.9503
low_thrust 0.03993
engine_out 0.00977
```

The simulated mean gap is within 0.1% of 4 s, the median wind is within 0.1% of the predicted 8.24 m/s, and the engine conditions come out at almost exactly 95%, 4% and 1%. With 10^5 samples, differences of this size are expected; we learn how to measure them properly in Chapter 6.

2.11.3 Transformation methods

Some distributions have no simple inverse CDF — the normal distribution is the famous example. But we can still build them from uniform numbers by a change of variables, which is exactly the kind of coordinate change this chapter is about.

Normal noise from polar coordinates (Box–Muller). Take two independent standard normal numbers Z_1, Z_2. Their joint density is

$$f(z_1, z_2) = \frac{1}{2\pi} e^{-(z_1^2 + z_2^2)/2}.$$

Now change to polar coordinates, $z_1 = r\cos\theta$, $z_2 = r\sin\theta$. Area elements change by the Jacobian, $\mathrm{d}z_1\,\mathrm{d}z_2 = r\,\mathrm{d}r\,\mathrm{d}\theta$, so the probability of a small patch becomes

$$\frac{1}{2\pi} e^{-r^2/2}\, r\,\mathrm{d}r\,\mathrm{d}\theta.$$

One more substitution, $s = r^2/2$ with $\mathrm{d}s = r\,\mathrm{d}r$, turns this into

$$\underbrace{e^{-s}\,\mathrm{d}s}_{\text{exponential, mean 1}} \times \underbrace{\frac{\mathrm{d}\theta}{2\pi}}_{\text{uniform on } [0,2\pi)}.$$

The patch probability has split into a product: s and θ are *independent*, s is exponential with mean 1 and θ is uniform on a full circle. Both are easy to generate: by the inverse transform, $s = -\ln U_1$, and $\theta = 2\pi U_2$. Going back through $r = \sqrt{2s}$:

$$Z_1 = \sqrt{-2\ln U_1}\,\cos(2\pi U_2), \qquad Z_2 = \sqrt{-2\ln U_1}\,\sin(2\pi U_2). \tag{2.51}$$

Two uniform numbers in, two independent standard normal numbers out. As a bonus, the derivation proves the Rayleigh claim of the previous example: the length $r = \sqrt{z_1^2 + z_2^2}$ of a two-dimensional normal vector satisfies $P(R \le r) = 1 - e^{-r^2/2}$.

Correlated noise from the Cholesky factor. Real sensors have *correlated* errors: an accelerometer whose x-axis reads high tends to read high on y as well. Correlated noise is described by a covariance matrix $\boldsymbol{\Sigma}$, which is symmetric and positive definite (Section 2.2). Write it as $\boldsymbol{\Sigma} = \mathbf{L}\mathbf{L}^{\mathsf{T}}$ with $\mathbf{L}$ lower triangular (the Cholesky factor of Section 2.10), draw a vector $\boldsymbol{z}$ of independent standard normals, and set

$$\boldsymbol{x} = \mathbf{L}\,\boldsymbol{z}. \tag{2.52}$$

Then $\boldsymbol{x}$ has exactly the covariance we wanted, in one line:

$$\operatorname{Cov}(\boldsymbol{x}) = \mathbb{E}\big[\mathbf{L}\boldsymbol{z}\boldsymbol{z}^{\mathsf{T}}\mathbf{L}^{\mathsf{T}}\big] = \mathbf{L}\,\mathbb{E}\big[\boldsymbol{z}\boldsymbol{z}^{\mathsf{T}}\big]\,\mathbf{L}^{\mathsf{T}} = \mathbf{L}\,\mathbf{I}\,\mathbf{L}^{\mathsf{T}} = \boldsymbol{\Sigma}.$$

Worked example: Correlated accelerometer noise. Take $\boldsymbol{\Sigma} = \begin{bmatrix} 4 & 1.2 \\ 1.2 & 1 \end{bmatrix}$ (in units of $(\text{mm/s}^2)^2$). Its Cholesky factor is $\mathbf{L} = \begin{bmatrix} 2 & 0 \\ 0.6 & 0.8 \end{bmatrix}$. Check: $\mathbf{L}\mathbf{L}^{\mathsf{T}} = \begin{bmatrix} 2\cdot 2 & 2\cdot 0.6 \\ 0.6\cdot 2 & 0.6^2 + 0.8^2 \end{bmatrix} = \begin{bmatrix} 4 & 1.2 \\ 1.2 & 1 \end{bmatrix}$.

R

```r
set.seed(4); n <- 100000
u1 <- runif(n); u2 <- runif(n)
r <- sqrt(-2 * log(u1))
z1 <- r * cos(2 * pi * u2)
z2 <- r * sin(2 * pi * u2)
round(c(mean(z1), sd(z1), cor(z1, z2)), 3)
Sigma <- matrix(c(4, 1.2, 1.2, 1), 2)
L <- t(chol(Sigma)) # lower triangular
x <- L %*% rbind(z1, z2)
round(cov(t(x)), 2)
```

```
[1] 0.000 0.999 0.000
    [,1] [,2]
[1,] 3.99 1.2
[2,] 1.20 1.0
```

Python

```python
import numpy as np
rng = np.random.default_rng(4); n = 100
    _000
u1, u2 = rng.random(n), rng.random(n)
r = np.sqrt(-2 * np.log(u1))
z1 = r * np.cos(2 * np.pi * u2)
z2 = r * np.sin(2 * np.pi * u2)
print(np.round([z1.mean(), z1.std(),
            np.corrcoef(z1, z2)[0, 1]], 3))

Sigma = np.array([[4, 1.2], [1.2, 1]])
L = np.linalg.cholesky(Sigma) # lower
x = L @ np.vstack([z1, z2])
print(np.round(np.cov(x), 2))
```

```
[ 0. 1.001 -0. ]
[[4.01 1.2 ]
 [1.2 1. ]]
```

Mean 0, standard deviation 1 and correlation 0 for the Box–Muller output, and the target covariance matrix recovered to two decimals. (R's `chol` returns the *upper* triangular factor, so we transpose it; NumPy's `cholesky` returns the lower one directly — a small difference worth remembering.)

Random directions and random attitudes. Monte Carlo studies also need random *directions* — the direction a gust comes from, the direction a thrust misalignment points — and random *attitudes*. Both follow from one property of the standard normal vector $\boldsymbol{z}$: its density depends only on the length $\|\boldsymbol{z}\|$. A rotation $\mathbf{Q}$ (Section 2.4) does not change lengths, so $\mathbf{Q}\boldsymbol{z}$ has exactly the same distribution as $\boldsymbol{z}$: no direction is preferred. Therefore

$$\hat{\boldsymbol{d}} = \frac{\boldsymbol{z}}{\|\boldsymbol{z}\|}, \quad \boldsymbol{z} \in \mathbb{R}^3 \qquad \text{is a uniformly random direction,} \tag{2.53}$$

and in the same way, normalising four independent standard normals gives a uniformly random unit quaternion — a uniformly random attitude (Section 2.6).

A beautiful check, known to Archimedes: for a uniformly random direction, the z-component is itself uniform on $[-1, 1]$. So the chance that $d_z > 0.5$ should be exactly 0.25.

R

```r
set.seed(5)
z <- matrix(rnorm(3 * 100000), ncol = 3)
d <- z / sqrt(rowSums(z^2)) # unit vectors
round(colMeans(d), 3) # about (0, 0, 0)
mean(d[, 3] > 0.5) # about 0.25
q <- rnorm(4)
q / sqrt(sum(q^2)) # random attitude
```

```
[1] -0.005 -0.002 0.002
[1] 0.25171
[1] -0.0290857 -0.4306299 0.4864511
    0.7596560
```

Python

```python
import numpy as np
rng = np.random.default_rng(5)
z = rng.standard_normal((100_000, 3))
d = z / np.linalg.norm(z, axis=1, keepdims
    =True)
print(np.round(d.mean(axis=0), 3)) # ~ 0
print(np.mean(d[:, 2] > 0.5)) # ~ 0.25
q = rng.standard_normal(4)
print(q / np.linalg.norm(q)) # attitude
```

```
[-0. 0.001 0. ]
0.24826
[0.87113229 0.1088347 0.3211577 0.35516372]
```

2.11.4 Accept–reject

Sometimes there is neither a simple inverse CDF nor a handy transformation. Then we use the most general method of all, which needs only one thing: the *shape* of the density.

The idea, in pictures. Draw the density curve $f(x)$ on a sheet of paper. Now throw darts uniformly at the region *under* the curve and record the x-position of each dart. Where the curve is tall, more darts land; where it is low, fewer do. The x-positions therefore follow f exactly. Robert and Casella call this the *fundamental theorem of simulation*: sampling from f is the same as sampling points uniformly under its graph.

We cannot throw darts only under the curve, because we do not know its shape well enough to aim. But we can throw them uniformly under a *bigger* curve that we can sample from, and simply throw away the darts that land above f.

The algorithm. Choose a *proposal* density g that we can sample from, and a constant M such that $f(x) \le M g(x)$ everywhere (the envelope Mg lies above f). Then repeat:

1. draw Y from g and U from $\mathcal{U}(0,1)$;
2. if $U \le \dfrac{f(Y)}{M g(Y)}$, *accept* $X = Y$; otherwise *reject* and try again.

Theorem 2.6 (Accept–reject)**.** *The accepted values have density f, and each try is accepted with probability $1/M$.*

Proof. For one try, the chance of drawing Y near y *and* accepting it is $g(y)\,\mathrm{d}y \times \dfrac{f(y)}{M g(y)} = \dfrac{f(y)}{M}\,\mathrm{d}y$. Adding this up over all y gives the overall acceptance probability:

$$P(\text{accept}) = \int \frac{f(y)}{M}\,\mathrm{d}y = \frac{1}{M},$$

because the area under any density is 1. The density of the accepted values is the chance of "near y and accepted" divided by the chance of "accepted":

$$\frac{f(y)/M}{1/M} = f(y). \qquad \square$$

So M measures the waste: on average it takes M tries to get one sample, and the closer the envelope hugs f, the closer M is to 1.

Worked example: A bounded thrust dispersion. An engine passes its acceptance test only if its thrust is within $\pm 3\%$ of nominal, and values near nominal are most likely. A natural model for the scale factor is $k = 0.97 + 0.06\,X$ with X from the Beta$(3,3)$ density on $[0,1]$,

$$f(x) = 30\,x^2(1-x)^2,$$

a hump that is zero at both ends. (The 30 makes the area equal to 1.) Use the uniform proposal $g(x) = 1$ on $[0,1]$. The hump is tallest at $x = 0.5$, where $f(0.5) = 30 \times 0.25 \times 0.25 = 1.875$, so $M = 1.875$ and the acceptance rate should be $1/1.875 \approx 0.533$: about 53 darts in 100 are kept (Fig. 2.5b).

Worked example: Normal numbers from the Laplace distribution. A classic example in Robert and Casella: the standard normal $f(y) = e^{-y^2/2}/\sqrt{2\pi}$ from the Laplace (double-exponential) proposal $g(y) = \frac{1}{2}e^{-|y|}$, which is easy to sample by the inverse transform. The ratio is

$$\frac{f(y)}{g(y)} = \sqrt{\frac{2}{\pi}}\,e^{|y| - y^2/2}.$$

The exponent $|y| - y^2/2$ is largest at $|y| = 1$, where it equals $\frac{1}{2}$, so the smallest valid M is $\sqrt{2/\pi}\,e^{1/2} = \sqrt{2e/\pi} \approx 1.3155$, and about 76% of proposals are accepted.

R

```r
f <- function(x) 30 * x^2 * (1 - x)^2 # Beta(3,3)
M <- 1.875 # height of f at x = 0.5
set.seed(6); n <- 100000
y <- runif(n); u <- runif(n)
keep <- u <= f(y) / M
mean(keep) # acceptance, ~ 1/M
k <- 0.97 + 0.06 * y[keep] # thrust scale
round(c(mean(k), sd(k), range(k)), 4)
```

```
[1] 0.53369
[1] 1.0000 0.0114 0.9705 1.0295
```

Python

```python
import numpy as np
f = lambda x: 30 * x**2 * (1 - x)**2 # Beta(3,3)
M = 1.875 # height of f at x = 0.5
rng = np.random.default_rng(6); n = 100_000
y, u = rng.random(n), rng.random(n)
keep = u <= f(y) / M
print(keep.mean()) # ~ 1/M
k = 0.97 + 0.06 * y[keep] # thrust scale
print(np.round([k.mean(), k.std(ddof=1),
                k.min(), k.max()], 4))
```

```
0.53303
[0.9999 0.0113 0.9704 1.0293]
```

Both acceptance rates are within 0.1% of the predicted 0.533, every thrust scale factor lies inside the $\pm 3\%$ band, and the standard deviation, about 0.0113, matches the exact value $0.06 \times \sqrt{9/252} \approx 0.01134$ for the scaled Beta$(3, 3)$.

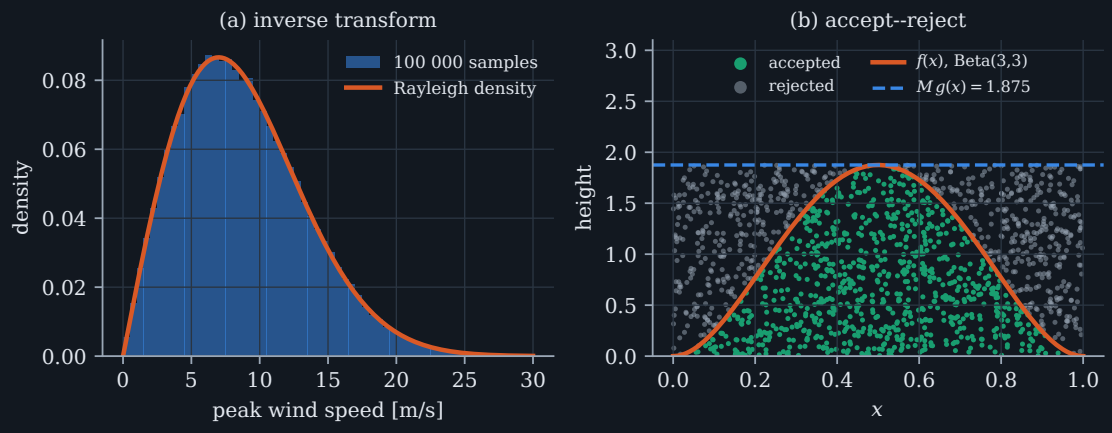


Figure 2.5: (a) Rayleigh wind speeds made by the inverse transform: the histogram of 100 000 samples sits on the exact density. (b) Accept–reject for the Beta(3,3) thrust model: darts thrown uniformly under the envelope $Mg(x) = 1.875$ are kept (green) if they land under $f(x)$ and thrown away (grey) otherwise. The kept darts fill the region under f evenly.

Method	Use it when	Example in this book
Inverse transform	the CDF can be inverted	gust timing, Rayleigh wind speed, engine condition
Transformation	a change of variables links the target to easy numbers	normal sensor noise, correlated noise, random directions and attitudes
Accept–reject	only the shape of the density is known	bounded thrust dispersion

Summary

- A vector is a physical arrow; its components $[v]^A$ depend on the axes. Dot products give angles and projections; cross products give moments, $\boldsymbol{M} = \boldsymbol{r} \times \boldsymbol{F}$.
- For $\dot{\boldsymbol{x}} = \mathbf{A}\boldsymbol{x}$, the eigenvalues of $\mathbf{A}$ are the poles, and $\boldsymbol{x}(t) = e^{\mathbf{A}t}\boldsymbol{x}(0)$. Symmetric matrices have real eigenvalues and perpendicular eigenvectors.
- A *frame* decides what is moving; a *coordinate system* decides which numbers we write. A point on the pad is still in the Earth frame but moves at 408 m/s in the inertial frame.
- Direction cosine matrices change axes, $[\boldsymbol{v}]^B = \mathbf{T}^{BA}[\boldsymbol{v}]^A$; they are orthogonal, so the inverse is the transpose, and they chain as $\mathbf{T}^{CA} = \mathbf{T}^{CB}\mathbf{T}^{BA}$. Order matters.
- Euler angles are intuitive but suffer gimbal lock at $\theta = \pm 90°$, which is exactly where a rocket starts. Quaternions have no singularity; $\boldsymbol{q}$ and $-\boldsymbol{q}$ are the same rotation.
- Derivatives depend on the frame: $\mathrm{d}^A\boldsymbol{v}/\,\mathrm{d}t = \mathrm{d}^B\boldsymbol{v}/\,\mathrm{d}t + \boldsymbol{\omega}^{BA} \times \boldsymbol{v}$. This produces the Coriolis and centripetal terms, which can move a vehicle by kilometres if ignored.
- The inertia tensor is a symmetric rank-2 tensor that transforms as $\mathbf{T}\mathbf{J}\mathbf{T}^{\mathsf{T}}$; its eigenvectors are the principal axes.
- Positions on Earth use the WGS-84 ellipsoid; geodetic and geocentric latitude differ by up to 0.19°.
- Use double precision for navigation, solvers instead of inverses, and watch the condition number.
- Computers make pseudo-random uniform numbers from a seed; the same seed gives the same run. The inverse transform $X = F^{-1}(U)$, transformations such as Box–Muller and the Cholesky factor, and accept–reject (acceptance rate $1/M$) turn uniform numbers into any distribution, including random directions and attitudes.

Exercises

2.1 Components. A wind of 15 m/s blows toward the north-east. Write it in (north, east) components, and then in the axes of a vehicle heading 60°. Check that the length is unchanged.

2.2 Cross product. Repeat the gimbal-moment example with the engine gimballed in yaw instead: $[\boldsymbol{T}]^B = T(\cos\delta, \sin\delta, 0)$. Which body axis is the moment about, and what is its sign?

2.3 Skew-symmetric matrix. Verify $[\boldsymbol{a}\times]\boldsymbol{b} = \boldsymbol{a} \times \boldsymbol{b}$ for $\boldsymbol{a} = (1,2,3)$ and $\boldsymbol{b} = (4,5,6)$. Then show that $[\boldsymbol{a}\times]\boldsymbol{a} = \mathbf{0}$ for any $\boldsymbol{a}$ and explain it geometrically.

2.4 Eigenvalues. Find the eigenvalues of $\begin{bmatrix} 0 & 1 \\ -6 & -5 \end{bmatrix}$. Is the system $\dot{\boldsymbol{x}} = \mathbf{A}\boldsymbol{x}$ stable?

2.5 Matrix exponential. For $\mathbf{A} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$ we found $e^{\mathbf{A}\Delta t}$. Now take a constant-acceleration model with state (position, velocity, acceleration) and $\mathbf{A}$ having ones just above the diagonal. Show that $\mathbf{A}^3 = \mathbf{0}$ and find $e^{\mathbf{A}\Delta t}$.

2.6 Orthogonality. Show that $\mathbf{R}_2(\alpha)^{\mathsf{T}}\mathbf{R}_2(\alpha) = \mathbf{I}$ and that $\mathbf{R}_2(\alpha)^{\mathsf{T}} = \mathbf{R}_2(-\alpha)$. Explain why the second fact makes sense.

2.7 Euler angles. Compute $\mathbf{T}^{BG}$ for $\psi = 90^\circ$, $\theta = 45^\circ$, $\phi = 0$. Where does the nose point, in (north, east, down)?

2.8 Gimbal lock in practice. Using (2.24), compute $\dot{\psi}$ when $\theta = 89^\circ$, $\phi = 0$, $q = 0$ and $r = 0.01\,\mathrm{rad/s}$. Repeat for $\theta = 89.9^\circ$. What happens to a simulator with a fixed time step?

2.9 Quaternions. Find the quaternion for a 180° rotation about x. Use (2.30) to find its rotation matrix, and check that it maps $(0, 1, 0)$ to $(0, -1, 0)$.

2.10 Double cover. Show directly from (2.30) that $\mathbf{R}(-\boldsymbol{q}) = \mathbf{R}(\boldsymbol{q})$.

2.11 Coriolis. A vehicle flies due north at 1500 m/s at latitude 45°. Using the NED form of $\boldsymbol{\omega}_E$, compute the Coriolis acceleration $-2\boldsymbol{\omega}_E \times \boldsymbol{v}$ in NED coordinates. Which way does it push the vehicle?

2.12 Inertia. After burning 80% of its propellant, the vehicle in the cylinder example has mass 150 t. Model it again as a uniform cylinder and recompute J_{yy}. By what factor did it drop, and what does that mean for the gimbal angles needed later in flight?

2.13 Geodesy. The Guiana Space Centre in Kourou is at about latitude 5.236°, longitude -52.769°, height 10 m. Convert this to ECEF. Then compute its geocentric latitude.

2.14 Floating point. In Python, compare `np.float32(6400000.3)` with `np.float64(6400000.3)`. Explain the result using the spacing argument of Section 2.10.

2.15 Generators. Run the generator (2.50) with $a = 5$, $c = 3$, $m = 16$ from seed 0. Then try $a = 3$. What period do you get, and why is a short period dangerous for a Monte Carlo study? (Do it in R and in Python.)

2.16 Inverse transform. The time to the first sensor dropout is modelled with the CDF $F(t) = 1 - e^{-(t/50)^2}$ (a Weibull distribution, t in seconds). Derive F^{-1}, generate 100 000 values in R and in Python, and check that about 63.2% are below 50 s.

2.17 Discrete inverse transform. Extend the engine-condition example with a fourth state, "gimbal stuck", probability 0.005, taking it out of the nominal case. Check the frequencies from 100 000 draws.

2.18 Box–Muller. Using (2.51), generate 100 000 pairs and estimate $P(|Z_1| > 2)$. Compare with the exact value 0.0455.

2.19 Cholesky. Factor $\boldsymbol{\Sigma} = \begin{bmatrix} 9 & 3 \\ 3 & 5 \end{bmatrix}$ by hand, check $\mathbf{L}\mathbf{L}^{\mathsf{T}} = \boldsymbol{\Sigma}$, and verify your answer with `chol` in R and `numpy.linalg.cholesky` in Python.

2.20 Random directions. Show that if you instead pick a random direction by drawing three *uniform* numbers on $[-1, 1]$ and normalising, the result is *not* uniform. Estimate $P(d_z > 0.5)$ by simulation and compare with 0.25.

2.21 Accept–reject. Code the normal-from-Laplace example in R and Python. Check that the acceptance rate is close to 0.760 and that the accepted values have mean 0 and standard deviation 1. What happens to the acceptance rate if you use $M = 2$ instead?

Reading for this chapter

- P. H. Zipfel, *Modeling and Simulation of Aerospace Vehicle Dynamics*, 2nd ed.: Chapter 2 (mathematical concepts and tensors), Chapter 3 (frames and coordinate systems), Appendix A (matrices) and Appendix D (the rotational time derivative).
- S. Boyd and L. Vandenberghe, *Convex Optimization*: Appendix A.1 (norms) and A.5 (linear algebra), and Appendix C.1 (algorithm complexity).
- M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*: Section 2.1 (linear algebra in quantum notation, including tensor products).
- R. R. Labbe, *Kalman and Bayesian Filters in Python*: Appendix A (installing Python, NumPy and FilterPy) and Appendix B (symbology).
- K. J. Åström and R. M. Murray, *Feedback Systems*: Sections 2.1–2.2 (state-space models), as preparation for Chapter 3.
- C. P. Robert and G. Casella, *Monte Carlo Statistical Methods*, 2nd ed.: Sections 2.1 (uniform simulation and the inverse transform), 2.2 (general transformation methods) and 2.3 (accept–reject methods and the fundamental theorem of simulation). STAT 3585 core.

Chapter 3

Kinematics and Rigid-Body Dynamics with Variable Mass and Inertia

> *A rocket is a body that is constantly throwing part of itself away. Every equation in this chapter has to remember that.*

Chapter 2 gave us the tools: vectors, frames, rotation matrices, quaternions, the transport theorem and the inertia tensor. This chapter uses those tools to build the equations that make a simulator move. By the end we will have written down, and understood, every one of the fourteen equations in the state equation $\dot{x} = f(x, u, t)$ that Chapter 1 promised.

We split the work into two kinds of question.

Kinematics asks *how things move*, without asking why. If we know the angular velocity, how does the attitude change? If we know the motion of the centre of mass, how fast is a sensor mounted near the nose moving?

Dynamics asks *why things move*: how forces and moments change the motion. This is where Newton's law, Euler's law and — for a rocket — the loss of mass come in.

The plan is:

1. the rigid-body idea and what it leaves out (Section 3.1);
2. kinematics of points on a moving body, and what an accelerometer really measures (Section 3.2);
3. attitude kinematics with direction cosine matrices (Section 3.3);
4. Newton's law for a body of constant mass, in inertial and in body axes (Section 3.4);
5. what changes when the mass changes: the thrust equation and the velocity budget (Section 3.5);
6. angular momentum and Euler's law for a rigid body, and what spin does to a rocket (Section 3.6);
7. Euler's law for a body that loses mass, and jet damping (Section 3.7);

8. the complete six-degree-of-freedom equations, and their linearisation (Sections 3.8 and 3.9);
9. a short Python example (Section 3.10).

Throughout the chapter we use one notional vehicle for the worked examples, so the numbers can be compared from section to section.

> **Key idea**: The example vehicle, LV-N1. LV-N1 is an invented single-stage vehicle, 40 m long and 3 m in diameter. It weighs 120 t at lift-off, of which 102 t is propellant. Its engine produces 2.2 MN in vacuum with a specific impulse of 310 s (280 s at sea level) and burns 723.67 kg/s for 140.95 s. Positions along the vehicle are given as *stations*, measured in metres back from the nose. The engine gimbal is at station 39 m and the nozzle exit at station 40 m. LV-N1 does not exist; it is sized only to make the numbers realistic.

3.1 Rigid bodies and degrees of freedom

A *rigid body* is a body in which the distance between any two points never changes. It can move in only six independent ways, its six *degrees of freedom*:

- three *translations*: the centre of mass can move along x, y and z;
- three *rotations*: the body can turn about three axes.

Everything else about its position follows from these six. That is why we can describe a whole rocket, made of millions of parts, with a position vector, a velocity vector, an attitude and an angular velocity.

A real rocket is not rigid. Propellant sloshes in the tanks, the long thin structure bends, and the engine nozzle swings on its gimbal. Treating the vehicle as rigid is a modelling decision, not a fact:

Left out	What it does	When it matters
Propellant slosh	liquid moves in the tanks and pushes on the walls	control design near resonance; large, partly empty tanks
Structural bending	the body flexes like a beam	when the flight-control loop is fast enough to excite the first bending mode
Engine motion	the nozzle's own mass swings when gimballed	large engines with fast actuators (the "tail-wags-dog" effect)

For trajectory simulation, a rigid body is the standard starting point. The one property of a rocket we *cannot* ignore, even in the simplest model, is that it loses mass. Most of this chapter is about doing that correctly.

3.2 How points on a moving body move

3.2.1 Velocity of a point on a rotating body

Let C be the centre of mass and P any point fixed in the body, at position $\boldsymbol{r}$ from C. Its position in an inertial frame is $\boldsymbol{r}_P = \boldsymbol{r}_C + \boldsymbol{r}$. Differentiate in the inertial frame I using the transport theorem of Chapter 2: $\boldsymbol{r}$ is fixed in the body B, so its derivative seen from B is zero, and

$$\frac{\mathrm{d}^I \boldsymbol{r}}{\mathrm{d}t} = \underbrace{\frac{\mathrm{d}^B \boldsymbol{r}}{\mathrm{d}t}}_{=\,\boldsymbol{0}} + \boldsymbol{\omega} \times \boldsymbol{r}.$$

Therefore

$$\boxed{\boldsymbol{v}_P = \boldsymbol{v}_C + \boldsymbol{\omega} \times \boldsymbol{r}.} \tag{3.1}$$

Every point moves with the centre of mass, plus a rotational part that grows with the distance from the centre and with the turning rate.

3.2.2 Acceleration of a point

We now do the same thing one level up: start from the velocity of P and differentiate once more, as seen from the inertial frame I. We go slowly, one step at a time.

Step 1: start from the velocity. From (3.1),

$$\boldsymbol{v}_P = \boldsymbol{v}_C + \boldsymbol{\omega} \times \boldsymbol{r}.$$

Step 2: differentiate both sides in I. Acceleration is the rate of change of velocity seen from the inertial frame, and the derivative of a sum is the sum of the derivatives:

$$\boldsymbol{a}_P = \frac{\mathrm{d}^I \boldsymbol{v}_P}{\mathrm{d}t} = \underbrace{\frac{\mathrm{d}^I \boldsymbol{v}_C}{\mathrm{d}t}}_{=\,\boldsymbol{a}_C} + \frac{\mathrm{d}^I}{\mathrm{d}t}\left(\boldsymbol{\omega} \times \boldsymbol{r}\right).$$

The first term is just the acceleration of the centre of mass, $\boldsymbol{a}_C$. All the work is in the second term.

Step 3: use the product rule on the cross product. A cross product is differentiated like an ordinary product, as long as we keep the order of the two vectors (because $\boldsymbol{a} \times \boldsymbol{b} = -\boldsymbol{b} \times \boldsymbol{a}$, swapping them would flip the sign):

$$\frac{\mathrm{d}^I}{\mathrm{d}t}\left(\boldsymbol{\omega} \times \boldsymbol{r}\right) = \frac{\mathrm{d}^I \boldsymbol{\omega}}{\mathrm{d}t} \times \boldsymbol{r} \;+\; \boldsymbol{\omega} \times \frac{\mathrm{d}^I \boldsymbol{r}}{\mathrm{d}t}.$$

So we need two derivatives: the rate of change of $\boldsymbol{\omega}$, and the rate of change of $\boldsymbol{r}$.

Step 4: the derivative of ω. By the transport theorem,

$$\frac{\mathrm{d}^I \boldsymbol{\omega}}{\mathrm{d}t} = \frac{\mathrm{d}^B \boldsymbol{\omega}}{\mathrm{d}t} + \boldsymbol{\omega} \times \boldsymbol{\omega} = \frac{\mathrm{d}^B \boldsymbol{\omega}}{\mathrm{d}t},$$

because any vector crossed with itself is zero. So the angular acceleration is the same whether we watch from I or from the body. We simply call it $\dot{\boldsymbol{\omega}}$ — it is what the change in gyroscope readings tells us.

Step 5: the derivative of $\boldsymbol{r}$. This is exactly what we worked out for the velocity: $\boldsymbol{r}$ is fixed in the body, so seen from I it changes only because the body turns,

$$\frac{\mathrm{d}^I \boldsymbol{r}}{\mathrm{d}t} = \underbrace{\frac{\mathrm{d}^B \boldsymbol{r}}{\mathrm{d}t}}_{=\,\mathbf{0}} + \boldsymbol{\omega} \times \boldsymbol{r} = \boldsymbol{\omega} \times \boldsymbol{r}.$$

Step 6: put the pieces back together. Substituting Steps 4 and 5 into Step 3, and then into Step 2:

$$\boxed{\boldsymbol{a}_P = \boldsymbol{a}_C + \underbrace{\dot{\boldsymbol{\omega}} \times \boldsymbol{r}}_{\text{tangential}} + \underbrace{\boldsymbol{\omega} \times (\boldsymbol{\omega} \times \boldsymbol{r})}_{\text{centripetal}}.} \tag{3.2}$$

What the two new terms mean. It helps to look at them one at a time.

The tangential term, $\dot{\boldsymbol{\omega}} \times \boldsymbol{r}$, is only there when the rotation is speeding up or slowing down ($\dot{\boldsymbol{\omega}} \neq \mathbf{0}$). It points along the direction in which P is moving around the axis, and its size is $\|\dot{\boldsymbol{\omega}}\|\, d$, where d is the distance of P from the axis. Think of pressing the accelerator on a merry-go-round: the riders are pushed along the direction of travel.

The centripetal term, $\boldsymbol{\omega} \times (\boldsymbol{\omega} \times \boldsymbol{r})$, is there whenever the body turns, even at a steady rate. That can seem strange at first: if nothing speeds up, why is there an acceleration? Think of swinging a ball on a string in a circle at a constant speed. The *speed* never changes, but the *direction* of the velocity changes all the time, and a change of direction is an acceleration too. The string has to keep pulling the ball toward the centre. The centripetal term is exactly that pull. Below we show, in small steps, that it always points toward the rotation axis.

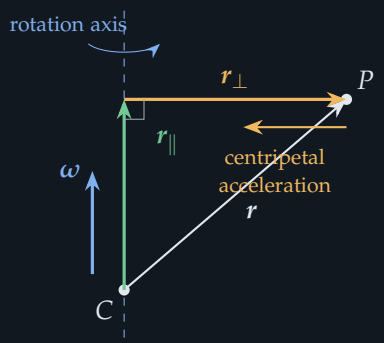


Figure 3.1: Splitting $\boldsymbol{r}$ into a part along the rotation axis, $\boldsymbol{r}_\parallel$, and a part at right angles to it, $\boldsymbol{r}_\perp$. The point P circles the axis at distance $d = \|\boldsymbol{r}_\perp\|$, and its centripetal acceleration points from P straight back toward the axis.

Step A: a rule for a cross product inside a cross product. Working out $\omega \times (\omega \times r)$ with components is possible but messy. A standard identity does it in one line:

$$a \times (b \times c) = b\,(a \cdot c) - c\,(a \cdot b). \tag{3.3}$$

It is called the "BAC–CAB" rule because the right-hand side reads $b(a \cdot c)$ minus $c(a \cdot b)$: "b-a-c minus c-a-b". The cross products disappear; only dot products (plain numbers) multiplying vectors are left.

Step B: substitute. Our expression has $a = \omega$, $b = \omega$ and $c = r$. Putting these into (3.3) one letter at a time:

$$\omega \times (\omega \times r) = \underbrace{\omega}_{b}\,(\underbrace{\omega}_{a} \cdot \underbrace{r}_{c}) - \underbrace{r}_{c}\,(\underbrace{\omega}_{a} \cdot \underbrace{\omega}_{b}) = \omega\,(\omega \cdot r) - \|\omega\|^2\, r,$$

using $\omega \cdot \omega = \|\omega\|^2$.

Step C: split r into two pieces. Any arrow r can be written as a piece along the axis plus a piece at right angles to it (Fig. 3.1):

$$r = r_{\parallel} + r_{\perp}, \qquad r_{\parallel} = \frac{\omega \cdot r}{\|\omega\|^2}\,\omega. \tag{3.4}$$

The formula for $r_{\parallel}$ is the projection from Chapter 2: $\omega \cdot r / \|\omega\|$ is how far r reaches along the axis, and $\omega / \|\omega\|$ is the unit arrow along the axis. Whatever is left over, $r_{\perp} = r - r_{\parallel}$, is the arrow from the axis out to P. Its length $d = \|r_{\perp}\|$ is the radius of the circle that P travels around.

Step D: simplify. After Step B we have two terms:

$$\omega \times (\omega \times r) = \underbrace{\omega\,(\omega \cdot r)}_{\text{first term}} - \underbrace{\|\omega\|^2\, r}_{\text{second term}}.$$

Step D simply renames both terms using the two pieces of r from Step C, so that we can see part of them cancel. We follow along with numbers, using $\omega = (0, 0, 2)$ rad/s and $r = (3, 0, 4)$ m, for which $\|\omega\|^2 = 4$, $\omega \cdot r = 8$, $r_{\parallel} = (0, 0, 4)$ and $r_{\perp} = (3, 0, 0)$.

Rename the first term. Step C defined $r_{\parallel} = \dfrac{\omega \cdot r}{\|\omega\|^2}\,\omega$. Multiply both sides by $\|\omega\|^2$ and the fraction disappears:

$$\|\omega\|^2\, r_{\parallel} = (\omega \cdot r)\,\omega.$$

The right-hand side is exactly the first term. So the first term is just $\|\omega\|^2\, r_{\parallel}$ written another way. *Numbers:* the first term is $8 \times (0, 0, 2) = (0, 0, 16)$, and $4 \times r_{\parallel} = 4 \times (0, 0, 4) = (0, 0, 16)$. The same.

Split the second term. Since $r = r_{\parallel} + r_{\perp}$,

$$\|\omega\|^2\, r = \|\omega\|^2\, r_{\parallel} + \|\omega\|^2\, r_{\perp}.$$

Numbers: $4 \times (3, 0, 4) = (12, 0, 16)$, which is $(0, 0, 16) + (12, 0, 0)$.

Subtract. The $\boldsymbol{r}_{\parallel}$ part is in both terms, so it cancels:

$$\boldsymbol{\omega} \times (\boldsymbol{\omega} \times \boldsymbol{r}) = \|\boldsymbol{\omega}\|^2 \boldsymbol{r}_{\parallel} - \left(\|\boldsymbol{\omega}\|^2 \boldsymbol{r}_{\parallel} + \|\boldsymbol{\omega}\|^2 \boldsymbol{r}_{\perp}\right) = -\|\boldsymbol{\omega}\|^2 \boldsymbol{r}_{\perp}.$$

It is like working out $5 - (5 + 3) = -3$: the fives cancel and only the three is left, with a minus sign. *Numbers:* $(0,0,16) - (12,0,16) = (-12,0,0)$, and $-4 \times (3,0,0) = (-12,0,0)$. So

$$\boxed{\boldsymbol{\omega} \times (\boldsymbol{\omega} \times \boldsymbol{r}) = -\|\boldsymbol{\omega}\|^2 \boldsymbol{r}_{\perp}.} \tag{3.5}$$

Step E: read the answer. This short result tells us three things.

1. *Direction: always toward the axis.* $\boldsymbol{r}_{\perp}$ is the arrow from the axis out to the point, and the minus sign turns it round to point back in. In the example, $\boldsymbol{r}_{\perp} = (3,0,0)$ points out along x, and the acceleration $(-12,0,0)$ points back toward the axis.
2. *Size:* $\omega^2 d$, where $d = \|\boldsymbol{r}_{\perp}\|$ is the distance from the axis. In the example $d = 3\,\mathrm{m}$ and $\omega = 2\,\mathrm{rad/s}$, so the size is $2^2 \times 3 = 12\,\mathrm{m/s^2}$. Spin twice as fast and the acceleration is *four* times larger, because ω is squared; move twice as far from the axis and it is twice as large.
3. *Height along the axis does not matter.* The $\boldsymbol{r}_{\parallel}$ part cancelled: the point's 4 m along the axis played no part, only its 3 m out from the axis. A point exactly on the axis ($d = 0$) feels no centripetal acceleration however fast the body spins — which is why the centre of a merry-go-round is the calm spot.

The link to v^2/r. A point d metres from the axis travels round a circle at speed $v = \omega d$, so

$$\frac{v^2}{d} = \frac{(\omega d)^2}{d} = \omega^2 d.$$

Numbers: $v = 2 \times 3 = 6\,\mathrm{m/s}$ and $v^2/d = 36/3 = 12\,\mathrm{m/s^2}$: the same 12 as before. The vector formula and the school formula agree.

Worked example: Checking the shortcut on a point that is not at right angles. Let the body spin about z at $\boldsymbol{\omega} = (0,0,2)\,\mathrm{rad/s}$, and take the point $\boldsymbol{r} = (3,0,4)\,\mathrm{m}$, which is partly along the axis.
The long way, two cross products with the component formula:

$$\boldsymbol{\omega} \times \boldsymbol{r} = (0 \cdot 4 - 2 \cdot 0,\ \ 2 \cdot 3 - 0 \cdot 4,\ \ 0 \cdot 0 - 0 \cdot 3) = (0,\ 6,\ 0),$$

$$\boldsymbol{\omega} \times (0,6,0) = (0 \cdot 0 - 2 \cdot 6,\ \ 2 \cdot 0 - 0 \cdot 0,\ \ 0 \cdot 6 - 0 \cdot 0) = (-12,\ 0,\ 0).$$

The shortcut: $\|\boldsymbol{\omega}\|^2 = 4$ and $\boldsymbol{\omega} \cdot \boldsymbol{r} = 8$, so $\boldsymbol{r}_{\parallel} = (8/4)(0,0,2) = (0,0,4)$ and $\boldsymbol{r}_{\perp} = (3,0,4) - (0,0,4) = (3,0,0)$. Then $-\|\boldsymbol{\omega}\|^2 \boldsymbol{r}_{\perp} = -4\,(3,0,0) = (-12,0,0)\,\mathrm{m/s^2}$.
Both ways agree. The point is 3 m from the axis, and the acceleration of $12\,\mathrm{m/s^2}$ points straight back toward the axis; the 4 m along the axis played no part.

If the point also moves inside the body. So far P was bolted to the body. Now let it move *within* the body: propellant flowing down a feed pipe, or liquid sloshing in a tank. Two motions now add up: the body carries P around, and P also travels relative to the body with velocity $\boldsymbol{v}_{\text{rel}}$ and acceleration $\boldsymbol{a}_{\text{rel}}$ (both as seen by someone riding on the body). We repeat the derivation, keeping the new pieces.

Velocity. Seen from the body, $\boldsymbol{r}$ now changes: $\mathrm{d}^B\boldsymbol{r}/\,\mathrm{d}t = \boldsymbol{v}_{\text{rel}}$. The transport theorem gives $\mathrm{d}^I\boldsymbol{r}/\,\mathrm{d}t = \boldsymbol{v}_{\text{rel}} + \boldsymbol{\omega}\times\boldsymbol{r}$, so

$$\boldsymbol{v}_P = \boldsymbol{v}_C + \boldsymbol{\omega}\times\boldsymbol{r} + \boldsymbol{v}_{\text{rel}}.$$

Acceleration. Differentiate each of the three terms in I:

- $\boldsymbol{v}_C$ gives $\boldsymbol{a}_C$, as before.
- $\boldsymbol{\omega}\times\boldsymbol{r}$ gives, by the product rule, $\dot{\boldsymbol{\omega}}\times\boldsymbol{r} + \boldsymbol{\omega}\times\dfrac{\mathrm{d}^I\boldsymbol{r}}{\mathrm{d}t} = \dot{\boldsymbol{\omega}}\times\boldsymbol{r} + \boldsymbol{\omega}\times\boldsymbol{v}_{\text{rel}} + \boldsymbol{\omega}\times(\boldsymbol{\omega}\times\boldsymbol{r})$.
- $\boldsymbol{v}_{\text{rel}}$ gives, by the transport theorem, $\dfrac{\mathrm{d}^I\boldsymbol{v}_{\text{rel}}}{\mathrm{d}t} = \boldsymbol{a}_{\text{rel}} + \boldsymbol{\omega}\times\boldsymbol{v}_{\text{rel}}$.

Add them up. The term $\boldsymbol{\omega}\times\boldsymbol{v}_{\text{rel}}$ appears *twice*, once from each of the last two bullets:

$$\boldsymbol{a}_P = \boldsymbol{a}_C + \dot{\boldsymbol{\omega}}\times\boldsymbol{r} + \boldsymbol{\omega}\times(\boldsymbol{\omega}\times\boldsymbol{r}) + \underbrace{2\,\boldsymbol{\omega}\times\boldsymbol{v}_{\text{rel}}}_{\text{Coriolis}} + \boldsymbol{a}_{\text{rel}}. \tag{3.6}$$

That is where the famous factor of 2 in the Coriolis term comes from: moving within a turning body changes your velocity in two ways at once. Your direction of motion is turned by the rotation, and your distance from the axis changes, so the body has to carry you at a different speed.

Worked example: Propellant in a feed pipe. Propellant flows aft along the vehicle axis at $5\,\mathrm{m/s}$, $\boldsymbol{v}_{\text{rel}} = (-5, 0, 0)$, while the vehicle pitches at $\boldsymbol{\omega} = (0, 0.1, 0)\,\mathrm{rad/s}$. Then

$$2\,\boldsymbol{\omega}\times\boldsymbol{v}_{\text{rel}} = 2\,\big(0.1\cdot 0 - 0\cdot 0,\;\; 0\cdot(-5) - 0\cdot 0,\;\; 0\cdot 0 - 0.1\cdot(-5)\big) = (0,\ 0,\ 1)\ \mathrm{m/s^2}.$$

The pipe walls must push the flowing propellant sideways at $1\,\mathrm{m/s^2}$ to keep it in the pipe, and the propellant pushes back on the vehicle. You feel the same sideways shove if you walk from the centre of a turning merry-go-round toward its edge.

Term	Name	When it is there
$\boldsymbol{a}_C$	centre-of-mass acceleration	always; shared by every point
$\dot{\boldsymbol{\omega}}\times\boldsymbol{r}$	tangential	while the turning rate changes
$\boldsymbol{\omega}\times(\boldsymbol{\omega}\times\boldsymbol{r})$	centripetal	whenever the body turns; points toward the axis
$2\,\boldsymbol{\omega}\times\boldsymbol{v}_{\text{rel}}$	Coriolis	only if P moves relative to the body *and* the body turns
$\boldsymbol{a}_{\text{rel}}$	relative	only if P's motion inside the body speeds up, slows down or bends

For sensors bolted to the structure, the last two terms are zero, and we are back to (3.2). We meet the Coriolis term again in Section 3.7, for the propellant flowing inside the vehicle.

Key idea. The acceleration of a point on a rotating body has three parts:

- the acceleration of the centre of mass, $\boldsymbol{a}_C$, shared by every point;
- a *tangential* part, $\dot{\boldsymbol{\omega}} \times \boldsymbol{r}$, present only while the rotation rate is changing;
- a *centripetal* part, $-\|\boldsymbol{\omega}\|^2 \boldsymbol{r}_\perp$, always pointing toward the rotation axis, present whenever the body turns.

Both extra parts grow with the distance from the centre of mass, which is why the position of an IMU inside a long rocket matters.

Worked example: An IMU that is not at the centre of mass. The inertial measurement unit (IMU) of a rocket usually sits in an avionics bay near the top, not at the centre of mass. Put it 10 m ahead of the centre of mass: $\boldsymbol{r} = (10, 0, 0)\,\mathrm{m}$. Suppose the vehicle pitches at $q = 0.1\,\mathrm{rad/s}$ and the pitch rate is increasing at $\dot{q} = 0.05\,\mathrm{rad/s^2}$, so $\boldsymbol{\omega} = (0, 0.1, 0)$ and $\dot{\boldsymbol{\omega}} = (0, 0.05, 0)$. Recall the component formula for a cross product, $\boldsymbol{a} \times \boldsymbol{b} = (a_2 b_3 - a_3 b_2,\ a_3 b_1 - a_1 b_3,\ a_1 b_2 - a_2 b_1)$.
Tangential part, with $\boldsymbol{a} = \dot{\boldsymbol{\omega}} = (0, 0.05, 0)$ and $\boldsymbol{b} = \boldsymbol{r} = (10, 0, 0)$:

$$\dot{\boldsymbol{\omega}} \times \boldsymbol{r} = \big(0.05 \cdot 0 - 0 \cdot 0,\ \ 0 \cdot 10 - 0 \cdot 0,\ \ 0 \cdot 0 - 0.05 \cdot 10\big) = (0,\ 0,\ -0.5)\ \mathrm{m/s^2}.$$

Centripetal part, in two steps. First the inner product, with $\boldsymbol{a} = \boldsymbol{\omega} = (0, 0.1, 0)$:

$$\boldsymbol{\omega} \times \boldsymbol{r} = \big(0.1 \cdot 0 - 0 \cdot 0,\ \ 0 \cdot 10 - 0 \cdot 0,\ \ 0 \cdot 0 - 0.1 \cdot 10\big) = (0,\ 0,\ -1)\ \mathrm{m/s}.$$

(This is the velocity of the IMU relative to the centre of mass, from (3.1).) Then cross $\boldsymbol{\omega}$ with that result:

$$\boldsymbol{\omega} \times (0, 0, -1) = \big(0.1 \cdot (-1) - 0 \cdot 0,\ \ 0 \cdot 0 - 0 \cdot (-1),\ \ 0 \cdot 0 - 0.1 \cdot 0\big) = (-0.1,\ 0,\ 0)\ \mathrm{m/s^2}.$$

Check with the shortcuts: here $\boldsymbol{r}$ is already perpendicular to the axis, so $\boldsymbol{r}_\perp = \boldsymbol{r}$ and the centripetal part is $-\|\boldsymbol{\omega}\|^2 \boldsymbol{r} = -0.01 \times (10, 0, 0) = (-0.1, 0, 0)$, pointing back toward the centre of mass. The tangential part has size $\|\dot{\boldsymbol{\omega}}\|\ d = 0.05 \times 10 = 0.5$, as computed. The IMU feels an extra $0.5\,\mathrm{m/s^2}$ sideways and $0.1\,\mathrm{m/s^2}$ backwards that the centre of mass does not. A navigation filter that ignores this *lever-arm effect* will think the whole vehicle is accelerating sideways.

3.2.3 What an accelerometer actually measures

This is one of the most important ideas in navigation, and one of the most often misunderstood. An accelerometer is a small mass on a spring inside a box. It measures the force needed to make the small mass follow the box. Gravity pulls equally on the small mass and on the box, so it cannot stretch the spring. **An accelerometer cannot feel gravity.** It measures the *specific*

force

$$\boldsymbol{f} = \boldsymbol{a} - \boldsymbol{g} = \frac{\boldsymbol{F}_{\text{non-gravity}}}{m}, \tag{3.7}$$

the acceleration caused by everything except gravity: thrust, aerodynamic forces, and the push of the ground.

> **Worked example**: Three readings.
>
> - *On the pad.* The rocket is not accelerating, $\boldsymbol{a} = \boldsymbol{0}$, and gravity points down, so $\boldsymbol{f} = -\boldsymbol{g}$: the accelerometer reads $9.81\,\mathrm{m/s^2}$ *upward*. That is the ground pushing up.
> - *At lift-off.* The LV-N1 sea-level thrust is 1.987 MN and its mass 120 t, so the accelerometer reads $1.987 \times 10^6 / 120\,000 \approx 16.56\,\mathrm{m/s^2}$ along the nose. The actual upward acceleration is $16.56 - 9.81 = 6.75\,\mathrm{m/s^2}$.
> - *After engine cut-off, in vacuum.* Only gravity acts, so $\boldsymbol{f} = \boldsymbol{0}$: the accelerometer reads zero although the vehicle is falling. This is "weightlessness".

This is why every navigation system must add a gravity model to the accelerometer output before integrating it: $\boldsymbol{a} = \boldsymbol{f} + \boldsymbol{g}(\boldsymbol{r})$. A mistake in $\boldsymbol{g}$ goes straight into the position estimate.

3.3 Attitude kinematics

3.3.1 Angular velocity, and adding angular velocities

What angular velocity is. Angular velocity is a single arrow that describes how something is turning at this moment:

- its *direction* is the axis of turning, given by the right-hand rule: curl the fingers of your right hand the way the body turns, and your thumb points along the arrow;
- its *length* is how fast the body turns, in radians per second.

Example: the Earth. The axis runs through the poles, with the arrow pointing north, and the Earth makes one full turn (2π radians) every 86 164 s (one sidereal day, measured against the stars). So

$$\omega_E = \frac{2\pi}{86\,164} \approx 7.29 \times 10^{-5}\,\mathrm{rad/s}. \tag{3.8}$$

The notation $\boldsymbol{\omega}^{BA}$. We write $\boldsymbol{\omega}^{BA}$ for "how frame B turns, as seen from frame A". The superscript is read the same way as for the rotation matrices of Chapter 2.

- $\boldsymbol{\omega}^{BE}$: the rocket (B) turning, as seen from the Earth (E);
- $\boldsymbol{\omega}^{EI}$: the Earth turning, as seen from space (I). This is just $\boldsymbol{\omega}_E$;
- $\boldsymbol{\omega}^{BI}$: the rocket turning, as seen from space.

Why angular velocities add. Think of walking along a train. If you walk forward at 1 m/s and the train moves at 20 m/s, your speed relative to the ground is $1 + 20 = 21$ m/s:

$$(\text{you relative to ground}) = (\text{you relative to train}) + (\text{train relative to ground}).$$

Turning rates work the same way:

$$\boldsymbol{\omega}^{CA} = \boldsymbol{\omega}^{CB} + \boldsymbol{\omega}^{BA}. \tag{3.9}$$

Notice how the letters chain: C to B, then B to A, gives C to A. The middle letter "cancels", just as it does when rotation matrices are chained. For a rocket:

$$\underbrace{\boldsymbol{\omega}^{BI}}_{\text{rocket relative to space}} = \underbrace{\boldsymbol{\omega}^{BE}}_{\text{rocket relative to Earth}} + \underbrace{\boldsymbol{\omega}_E}_{\text{Earth relative to space}}. \tag{3.10}$$

> **Reality check**: Rates add, angles do not. This adding rule works for turning *rates*, which are arrows at one instant. It does *not* work for large turning *angles*: Chapter 2 showed that 90° about x followed by 90° about y is not the same as doing them the other way round. Rates add; finite rotations do not.

Why gyroscopes read the Earth's rotation. A gyroscope senses rotation relative to *space* — relative to the stars. It has no idea that the Earth is there. So it always measures $\boldsymbol{\omega}^{BI}$, never $\boldsymbol{\omega}^{BE}$. On the launch pad the rocket is not moving relative to the Earth, so $\boldsymbol{\omega}^{BE} = \mathbf{0}$, and

$$\boldsymbol{\omega}^{BI} = \mathbf{0} + \boldsymbol{\omega}_E.$$

The gyroscopes still read the Earth's spin:

$$7.29 \times 10^{-5} \ \tfrac{\text{rad}}{\text{s}} \times 3600 \ \tfrac{\text{s}}{\text{h}} \times 57.3 \ \tfrac{\text{deg}}{\text{rad}} \approx 15^\circ \text{ per hour}.$$

How this finds north. At the launch site the Earth's spin arrow splits into two local pieces (Chapter 2). At Cape Canaveral, latitude 28.5°:

- a *horizontal piece, pointing exactly north*: $\omega_E \cos 28.5^\circ \approx 13.2^\circ$ per hour;
- a *vertical piece*: $\omega_E \sin 28.5^\circ \approx 7.2^\circ$ per hour.

A sensitive gyroscope on the pad can measure the horizontal piece, and its direction *is* true north. This is called *gyrocompassing*. It is how a rocket's navigation system finds which way it is facing before lift-off, with no magnetic compass and no GPS. It needs good gyroscopes: navigation-grade units drift by roughly 0.01° per hour, far less than the 13° per hour signal, while inexpensive gyroscopes like those in phones drift by many degrees per hour and lose the Earth's signal in their own noise.

Why it matters in flight. The Earth's rate is tiny compared with a rocket pitching at 0.1 rad/s, so the flight-control system hardly notices it. But navigation adds up small errors over time. Ignore ω_E for ten minutes and the attitude estimate is off by

$$7.29 \times 10^{-5} \times 600\,\text{s} \approx 0.044\,\text{rad} \approx 2.5^\circ,$$

enough to put a landing far off target. So navigation software always removes the Earth's rate to get the rocket's rotation relative to the ground:

$$\boldsymbol{\omega}^{BE} = \boldsymbol{\omega}^{BI} - \boldsymbol{\omega}_E. \tag{3.11}$$

3.3.2 How the direction cosine matrix changes

What we are trying to do. The direction cosine matrix $\mathbf{T}^{BI}$ from Chapter 2 is the rocket's "address book" of directions: it tells us where north, east and up are, written in the rocket's own axes. When the rocket turns, every number in that address book changes. The gyroscopes tell us how fast the rocket is turning, $\boldsymbol{\omega}$. We want a simple rule that says: *given* $\boldsymbol{\omega}$*, how fast does each number in* $\mathbf{T}^{BI}$ *change?* Once we have that rule, the computer can keep the address book up to date, step by step, for the whole flight.

The answer turns out to be one short line. Before we get there, let us earn it with an example small enough to check by hand.

A warm-up in two dimensions. Imagine the rocket can only turn about one axis, like a record on a turntable. Its turning angle is θ, and it grows at the rate $\omega = \dot{\theta}$. From Chapter 2, the matrix that rewrites a flat vector in the turned axes is

$$\mathbf{T} = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}.$$

Now differentiate each entry with respect to time. By the chain rule, $\frac{\mathrm{d}}{\mathrm{d}t}\cos\theta = -\sin\theta\,\dot{\theta}$ and $\frac{\mathrm{d}}{\mathrm{d}t}\sin\theta = \cos\theta\,\dot{\theta}$, with $\dot{\theta} = \omega$:

$$\dot{\mathbf{T}} = \omega \begin{bmatrix} -\sin\theta & \cos\theta \\ -\cos\theta & -\sin\theta \end{bmatrix}.$$

Here is the nice surprise. The right-hand side is just a simple matrix times the *original* $\mathbf{T}$. Check it by multiplying out (row times column):

$$\begin{bmatrix} 0 & \omega \\ -\omega & 0 \end{bmatrix} \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix} = \begin{bmatrix} -\omega\sin\theta & \omega\cos\theta \\ -\omega\cos\theta & -\omega\sin\theta \end{bmatrix}. \quad \checkmark$$

So, in two dimensions,

$$\dot{\mathbf{T}} = \begin{bmatrix} 0 & \omega \\ -\omega & 0 \end{bmatrix} \mathbf{T}. \tag{3.12}$$

In words: *the rate of change of the matrix is a small "turning matrix" times the matrix itself.* The turning matrix contains nothing but the turning rate ω, with zeros on the diagonal and the same number with opposite signs on either side. Keep that shape in mind; it is about to come back.

The same idea in three dimensions. In three dimensions the rocket can turn about any axis, so we cannot just write down $\cos\theta$ and $\sin\theta$. Instead we use one property every direction cosine matrix has, and let it do the work. Take it in four small steps.

Step 1: the property. The rows of $\mathbf{T}$ are three perpendicular arrows of length 1 (the rocket's axes), so (Chapter 2)

$$\mathbf{T}\mathbf{T}^{\mathsf{T}} = \mathbf{I}.$$

Think of it as "the axes stay square and the rulers stay one metre long", no matter how the rocket turns.

Step 2: differentiate it. $\mathbf{I}$ never changes, so its rate of change is zero. Using the product rule on the left (exactly like $\frac{\mathrm{d}}{\mathrm{d}t}(fg) = \dot{f}g + f\dot{g}$):

$$\dot{\mathbf{T}}\mathbf{T}^{\mathsf{T}} + \mathbf{T}\dot{\mathbf{T}}^{\mathsf{T}} = \mathbf{0}.$$

Step 3: spot the pattern. Give the first term a name, $\mathbf{S} = \dot{\mathbf{T}}\mathbf{T}^{\mathsf{T}}$. The second term is its transpose, because $(\mathbf{A}\mathbf{B})^{\mathsf{T}} = \mathbf{B}^{\mathsf{T}}\mathbf{A}^{\mathsf{T}}$ gives $\left(\dot{\mathbf{T}}\mathbf{T}^{\mathsf{T}}\right)^{\mathsf{T}} = \mathbf{T}\dot{\mathbf{T}}^{\mathsf{T}}$. So Step 2 says

$$\mathbf{S} + \mathbf{S}^{\mathsf{T}} = \mathbf{0} \qquad \text{or} \qquad \mathbf{S}^{\mathsf{T}} = -\mathbf{S}.$$

A matrix that equals minus its own transpose is called *skew-symmetric*. That is a fancy name for a simple shape: zeros down the diagonal, and every entry has a mirror image across the diagonal with the opposite sign,

$$\mathbf{S} = \begin{bmatrix} 0 & -c & b \\ c & 0 & -a \\ -b & a & 0 \end{bmatrix}.$$

It has only three free numbers, a, b and c, so it holds exactly as much information as one three-dimensional vector. In Chapter 2 we met exactly this shape: it is the cross-product matrix $[\boldsymbol{a}\times]$ of the vector $\boldsymbol{a} = (a, b, c)$. Which vector is it here? The two-dimensional warm-up already told us. Turning about z at rate ω means $\boldsymbol{\omega} = (0, 0, \omega)$, and

$$-[\boldsymbol{\omega}\times] = -\begin{bmatrix} 0 & -\omega & 0 \\ \omega & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & \omega & 0 \\ -\omega & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix},$$

whose top-left corner is exactly the turning matrix in (3.12). So $\mathbf{S} = -[\boldsymbol{\omega}\times]$.

Step 4: tidy up. We have $\dot{\mathbf{T}}\mathbf{T}^{\mathsf{T}} = -[\boldsymbol{\omega}\times]$. Multiply both sides on the right by $\mathbf{T}$, and use $\mathbf{T}^{\mathsf{T}}\mathbf{T} = \mathbf{I}$ to clear the left side:

$$\boxed{\dot{\mathbf{T}}^{BI} = -[\boldsymbol{\omega}\times]\,\mathbf{T}^{BI}.} \tag{3.13}$$

That is the whole rule. Feed in the gyroscope reading $\boldsymbol{\omega}$, and it tells you how fast every entry of the matrix is changing.

Why the minus sign? Sit on a spinning office chair and turn to the left. The room seems to swing to the right. The matrix $\mathbf{T}^{BI}$ describes the *room* (the inertial directions) as seen from the *rocket*, so when the rocket turns one way, the directions it is tracking appear to turn the other way. The minus sign is that feeling, written in mathematics.

Using the rule on a computer. A computer moves forward in small time steps Δt. Over one short step, "rate of change $\times$ time" gives the change, so

$$\mathbf{T}(t + \Delta t) \approx \mathbf{T}(t) + \dot{\mathbf{T}}\,\Delta t = \left(\mathbf{I} - [\boldsymbol{\omega}\times]\Delta t\right)\mathbf{T}(t). \tag{3.14}$$

Worked example: One small step, and a small problem. The rocket turns about z at $\omega = 0.1\,\text{rad/s}$, and the computer uses steps of $\Delta t = 0.01\,\text{s}$. In one step the rocket turns by $\omega\Delta t = 0.001\,\text{rad}$. The update matrix from (3.14) is

$$\mathbf{I} - [\boldsymbol{\omega}\times]\Delta t = \begin{bmatrix} 1 & 0.001 & 0 \\ -0.001 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix},$$

while the *exact* rotation by 0.001 rad would be

$$\begin{bmatrix} \cos 0.001 & \sin 0.001 & 0 \\ -\sin 0.001 & \cos 0.001 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0.9999995 & 0.0010000 & 0 \\ -0.0010000 & 0.9999995 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

They are almost identical. The only difference is on the diagonal: the shortcut has 1 where the truth has 0.9999995. Because of that, each row of the shortcut matrix has length $\sqrt{1^2 + 0.001^2} \approx 1.0000005$ instead of exactly 1. In words: every step, the rocket's "one-metre rulers" get *half a micrometre* longer.

That sounds like nothing. But a 140 s flight at this step size has 14 000 steps, and the stretching compounds:

$$(1.0000005)^{14\,000} \approx 1.007.$$

By the end of the flight the rulers are 0.7% too long and the axes are no longer exactly square. Every direction computed from this matrix is slightly wrong.

The fix is easy. Every so often (often every step), the software "squares up" the matrix: it shrinks each row back to length 1 and nudges the rows back to exactly 90° apart. This is called *re-orthonormalisation*. It costs a handful of multiplications. The lesson is simply that the matrix cannot be left to look after itself.

3.3.3 Quaternion kinematics, again

The quaternion from Chapter 2 does the same job as the direction cosine matrix — it stores the rocket's attitude — but more economically. It changes according to

$$\dot{\boldsymbol{q}} = \tfrac{1}{2}\,\boldsymbol{q} \odot (0,\ \boldsymbol{\omega}), \tag{3.15}$$

which, written out, is four short equations with no division and no tan anywhere, so nothing can ever blow up (Chapter 2 lists them).

It has the same small drift problem as the matrix, but the fix is even easier.

Worked example: The same turn, with a quaternion. Start with no rotation, $\boldsymbol{q} = (1,0,0,0)$, and turn about z at $\boldsymbol{\omega} = (0,0,0.1)$. Then $\dot{\boldsymbol{q}} = \frac{1}{2}(1,0,0,0) \odot (0,0,0,0.1) = (0,0,0,0.05)$. After one step of 0.01 s:

$$\boldsymbol{q} \approx (1,\ 0,\ 0,\ 0.0005).$$

Its length is $\sqrt{1+0.0005^2} \approx 1.000000125$, a touch more than 1. To fix it, divide all four numbers by that length:

$$q \approx (0.999999875,\ 0,\ 0,\ 0.0005),$$

which is exactly the true answer $(\cos 0.0005,\ 0,\ 0,\ \sin 0.0005)$ to the digits shown. One division, and the quaternion is healthy again.

	Numbers stored	Rules to keep	How to fix drift
Direction cosine matrix	9	6 (three lengths, three right angles)	square up the rows
Quaternion	4	1 (length 1)	one division

Key idea. Both descriptions follow the same pattern: *rate of change = (something built from $\boldsymbol{\omega}$) $\times$ (the current attitude)*. Both drift a tiny bit on a computer, and both are easy to repair. The quaternion simply needs fewer numbers and less repair, which is why flight software integrates the quaternion and turns it into a direction cosine matrix only when it needs one. If you followed the two-dimensional warm-up, you have understood the heart of attitude kinematics.

3.4 Newton's law for a body of constant mass

3.4.1 In inertial axes

The linear momentum of a body is its mass times the velocity of its centre of mass, $\boldsymbol{p} = m\boldsymbol{v}_C$. Newton's second law says the external force changes the momentum, *as seen from an inertial frame*:

$$\boldsymbol{F} = \frac{\mathrm{d}^I \boldsymbol{p}}{\mathrm{d}t} = m\,\frac{\mathrm{d}^I \boldsymbol{v}_C}{\mathrm{d}t} \qquad (\text{constant } m). \tag{3.16}$$

In inertial coordinates this is the familiar $m\dot{\boldsymbol{v}} = \boldsymbol{F}$, component by component. It is the simplest form, and it is the one a launch-vehicle simulator integrates.

3.4.2 In body axes

Aircraft simulators, and many control-design models, write the velocity in body axes, $[\boldsymbol{v}_C]^B = (u, v, w)$. Using the transport theorem from I to B:

$$\frac{\mathrm{d}^I \boldsymbol{v}_C}{\mathrm{d}t} = \frac{\mathrm{d}^B \boldsymbol{v}_C}{\mathrm{d}t} + \boldsymbol{\omega} \times \boldsymbol{v}_C,$$

so Newton's law becomes

$$m\big(\dot{\boldsymbol{v}} + \boldsymbol{\omega} \times \boldsymbol{v}\big) = \boldsymbol{F} \qquad (\text{all in body axes}). \tag{3.17}$$

With $\boldsymbol{\omega} = (p, q, r)$, the cross product is $\boldsymbol{\omega} \times \boldsymbol{v} = (qw - rv,\ ru - pw,\ pv - qu)$, so in components:

$$\begin{aligned} \dot{u} &= F_x/m - qw + rv, \\ \dot{v} &= F_y/m - ru + pw, \\ \dot{w} &= F_z/m - pv + qu. \end{aligned} \tag{3.18}$$

> **Worked example**: Where the extra terms come from. A vehicle flies at $u = 500\,\mathrm{m/s}$ along its nose and pitches up at $q = 0.05\,\mathrm{rad/s}$, with no sideways velocity yet ($v = w = 0$). Suppose there is no normal force at all, $F_z = 0$. Then (3.18) gives $\dot{w} = qu = 0.05 \times 500 = 25\,\mathrm{m/s^2}$. No force is pushing the vehicle along z_B, but the *axes* are turning: as the nose rises, the old velocity direction falls below the nose, and in body axes that looks like a growing w — a growing angle of attack. This is exactly how pitching the vehicle creates angle of attack.

The body-axis form makes angles of attack and aerodynamic forces easy to write; the inertial form makes gravity and long trajectories easy. Both are the same physics.

3.5 Translation when the mass changes

3.5.1 Why "$F = \mathrm{d}(mv)/\mathrm{d}t$" is wrong

It is tempting to say that for changing mass, Newton's law becomes $\boldsymbol{F} = \mathrm{d}(m\boldsymbol{v})/\mathrm{d}t = m\dot{\boldsymbol{v}} + \dot{m}\boldsymbol{v}$. This is a classic mistake. Imagine a sandbag falling straight down out of a cart that rolls at constant speed on a flat track. The sand leaves with the cart's own velocity, so nothing pushes the cart; its speed does not change. But the formula above, with $\boldsymbol{F} = \boldsymbol{0}$ and $\dot{m} < 0$, would predict $m\dot{\boldsymbol{v}} = -\dot{m}\boldsymbol{v} \neq \boldsymbol{0}$: the cart would speed up just by losing sand. That is clearly wrong. The formula also depends on which inertial frame you measure $\boldsymbol{v}$ in, which a physical law must not.

The correct approach, already used in Chapter 1 for the rocket equation, is to apply Newton's law to a *fixed set of matter* — the vehicle plus the small mass that leaves it during $\mathrm{d}t$ — and keep careful track of the momentum the leaving mass carries away.

3.5.2 The thrust equation

Let mass leave through the nozzle exit at the rate $\dot{m}_{\text{out}} = -\dot{m} > 0$, with velocity $\boldsymbol{u}_e$ relative to the vehicle, and let the exhaust pressure at the exit plane be p_e while the surrounding air has pressure p_a. Repeating the momentum balance of Chapter 1 in three dimensions, and adding the pressure force on the exit area A_e, gives

$$m\,\dot{\boldsymbol{v}}_C = \boldsymbol{F}_{\text{ext}} + \boldsymbol{T}, \qquad \boldsymbol{T} = -\dot{m}_{\text{out}}\,\boldsymbol{u}_e + (p_e - p_a)\,A_e\,\hat{\boldsymbol{n}}_e, \tag{3.19}$$

where $\hat{\boldsymbol{n}}_e$ points from the exit plane into the vehicle (forward). Since $\boldsymbol{u}_e$ points backwards, the first term points forwards. So: **a rocket obeys an ordinary-looking $m\dot{\boldsymbol{v}} =$ force, provided "force" includes the thrust** — the momentum flux of the exhaust plus the pressure difference across the nozzle exit.

Why thrust grows with altitude. Write the thrust magnitude as

$$T = \underbrace{\dot{m}_{\text{out}} u_e + p_e A_e}_{T_{\text{vac}}} - p_a A_e. \tag{3.20}$$

The first two terms depend only on the engine; together they are the thrust in vacuum. The last term is the back-pressure of the atmosphere on the exit area. As the vehicle climbs, p_a falls and the thrust rises toward its vacuum value.

> **Worked example**: LV-N1 thrust at sea level and in vacuum. LV-N1 has $T_{\text{vac}} = 2.2\,\text{MN}$ and an exit area $A_e = 2.10\,\text{m}^2$. At sea level, $p_a = 101\,325\,\text{Pa}$, so
>
> $$T_{\text{sl}} = 2.2 \times 10^6 - 101\,325 \times 2.10 \approx 1.987\,\text{MN}.$$
>
> The engine gives about 11% more thrust in vacuum than on the pad, from nothing but the missing atmosphere. The same ratio appears in the specific impulses: $310/280 \approx 1.107$.

Where is the centre of mass? Equation (3.19) is written for the *instantaneous* centre of mass C of everything still in the vehicle. As propellant drains, C moves relative to the structure; for LV-N1 it moves aft from station 21.75 m to a maximum of about 28.9 m and back to 26.0 m (Chapter 2). This motion is slow — a few centimetres per second — and the small extra force terms it produces are normally neglected in trajectory simulation. What cannot be neglected is the effect of the moving centre of mass on *moments*: lever arms change, and with them the control authority and the aerodynamic stability. We return to this in Section 3.7.

3.5.3 The velocity budget

The rocket equation, $\Delta v = v_e \ln(m_0/m_f)$, gives the speed an engine could add in empty space. A real ascent gets less. To see where it goes, project (3.19) onto the direction of flight. Let V be the speed, γ the flight-path angle above the horizontal, α the angle between thrust and velocity, and D the drag. For a flat Earth,

$$\dot{V} = \frac{T\cos\alpha - D}{m} - g\sin\gamma. \tag{3.21}$$

Write $T\cos\alpha = T_{\text{vac}} - (T_{\text{vac}} - T) - T(1-\cos\alpha)$ and integrate over the burn:

$$\begin{aligned} V(t_b) = V(0) &+ \underbrace{\int_0^{t_b} \frac{T_{\text{vac}}}{m}\,\mathrm{d}t}_{\text{ideal }\Delta v} - \underbrace{\int_0^{t_b} g\sin\gamma\,\mathrm{d}t}_{\text{gravity loss}} - \underbrace{\int_0^{t_b} \frac{D}{m}\,\mathrm{d}t}_{\text{drag loss}} \\ &- \underbrace{\int_0^{t_b} \frac{T_{\text{vac}} - T}{m}\,\mathrm{d}t}_{\text{back-pressure loss}} - \underbrace{\int_0^{t_b} \frac{T(1-\cos\alpha)}{m}\,\mathrm{d}t}_{\text{steering loss}}. \end{aligned} \tag{3.22}$$

Every term has a clear physical meaning, and together they explain the whole gap between the rocket equation and reality.

Worked example: Where LV-N1's velocity goes. We flew LV-N1 in a six-degree-of-freedom simulation with a flat, non-rotating Earth (to keep the bookkeeping simple) and evaluated each integral in (3.22) along the trajectory:

Term	Velocity [m/s]
ideal Δv, $I_{sp,\mathrm{vac}}\, g_0 \ln(m_0/m_f)$	5767.4
gravity loss	-972.7
drag loss	-86.6
back-pressure loss	-81.3
steering loss	-0.8
burnout speed	4625.9

The terms add up to the simulated burnout speed to within 0.0001 m/s. Gravity is by far the largest loss: about 17% of the ideal Δv is spent just holding the vehicle up. Drag and back-pressure cost less than 3% together, and the gravity turn keeps the steering loss almost negligible. This is why rockets have high thrust-to-weight ratios: the faster the burn, the less time gravity has to take its share.

3.6 Euler's law for a rigid body

3.6.1 Angular momentum of a rigid body

The angular momentum about the centre of mass is the sum of $\boldsymbol{r} \times \boldsymbol{v}$ over every bit of mass, using the velocity relative to the centre of mass, which for a rigid body is $\boldsymbol{\omega} \times \boldsymbol{r}$ by (3.1):

$$\boldsymbol{H} = \int \boldsymbol{r} \times (\boldsymbol{\omega} \times \boldsymbol{r}) \, \mathrm{d}m. \tag{3.23}$$

The vector identity $\boldsymbol{r} \times (\boldsymbol{\omega} \times \boldsymbol{r}) = \|\boldsymbol{r}\|^2 \boldsymbol{\omega} - \boldsymbol{r}(\boldsymbol{r} \cdot \boldsymbol{\omega}) = (\|\boldsymbol{r}\|^2 \mathbf{I} - \boldsymbol{r}\boldsymbol{r}^{\mathsf{T}})\boldsymbol{\omega}$ lets us take $\boldsymbol{\omega}$ outside the integral:

$$\boldsymbol{H} = \underbrace{\int \left(\|\boldsymbol{r}\|^2 \mathbf{I} - \boldsymbol{r}\boldsymbol{r}^{\mathsf{T}}\right) \mathrm{d}m}_{\mathbf{J}} \; \boldsymbol{\omega} = \mathbf{J}\boldsymbol{\omega}. \tag{3.24}$$

This is where the inertia tensor of Chapter 2 comes from: it is exactly the matrix that turns angular velocity into angular momentum.

3.6.2 Euler's law in body axes

Euler's law says the external moment about the centre of mass changes the angular momentum, as seen from an inertial frame: $\boldsymbol{M} = \mathrm{d}^I \boldsymbol{H} / \mathrm{d}t$. The inertia tensor is constant in body axes (for constant mass), so we differentiate in B and use the transport theorem:

$$\boldsymbol{M} = \frac{\mathrm{d}^B (\mathbf{J}\boldsymbol{\omega})}{\mathrm{d}t} + \boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega}).$$

For constant $\mathbf{J}$:

$$\boxed{\mathbf{J}\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega}) = \boldsymbol{M}.} \tag{3.25}$$

The first term is the rotational version of $m\boldsymbol{a}$. The second, the *gyroscopic term*, is there because the body axes in which we write $\boldsymbol{H}$ are themselves turning.

3.6.3 Euler's equations in principal axes

If the body axes are principal axes (Chapter 2), $\mathbf{J} = \operatorname{diag}(J_x, J_y, J_z)$ and (3.25) splits into three scalar equations:

$$\begin{aligned} J_x \dot{p} &= M_x - (J_z - J_y)\, q\, r, \\ J_y \dot{q} &= M_y - (J_x - J_z)\, r\, p, \\ J_z \dot{r} &= M_z - (J_y - J_x)\, p\, q. \end{aligned} \tag{3.26}$$

Without the products of rates, each axis would behave independently. The products couple them: a rotation about one axis, combined with a rotation about another, produces an acceleration about the third.

3.6.4 What spin does to a rocket

A rocket is (nearly) axisymmetric: $J_y = J_z = J_t$ (the transverse inertia) and J_x is the roll inertia, much smaller. Then the roll equation in (3.26) has no coupling term ($J_z - J_y = 0$), so with no roll moment the roll rate p stays constant. The other two become

$$\dot{q} = \lambda\, r + \frac{M_y}{J_t}, \qquad \dot{r} = -\lambda\, q + \frac{M_z}{J_t}, \qquad \lambda = p\,\frac{J_t - J_x}{J_t}. \tag{3.27}$$

With no moments, the solution is (check it by differentiating)

$$q(t) = q_0 \cos \lambda t, \qquad r(t) = -q_0 \sin \lambda t. \tag{3.28}$$

The transverse angular velocity keeps its size but rotates around the body at the rate λ. *A pitch rate turns into a yaw rate and back again.*

> **Worked example**: A rolling LV-N1. At lift-off LV-N1 has $J_x \approx 1.27 \times 10^5\,\mathrm{kg\,m^2}$ and $J_t \approx 8.59 \times 10^6\,\mathrm{kg\,m^2}$. If it rolls at $p = 0.5\,\mathrm{rad/s}$ (about $29\,^\circ/\mathrm{s}$),
>
> $$\lambda = 0.5 \times \frac{8.59 \times 10^6 - 1.27 \times 10^5}{8.59 \times 10^6} \approx 0.493\,\mathrm{rad/s}.$$
>
> A pitch rate becomes a pure yaw rate after a quarter of a cycle, $\pi/(2\lambda) \approx 3.2\,\mathrm{s}$, and the full cycle takes $2\pi/\lambda \approx 12.8\,\mathrm{s}$. An autopilot that commands pitch and expects pitch will be badly confused. This is why launch vehicles keep their roll rate small, and why a roll-control system is needed even on a vehicle that is meant not to roll.

3.7 Euler's law when the mass changes

3.7.1 The question

As propellant burns, $\mathbf{J}$ changes: for LV-N1, J_t falls from 8.59×10^6 to $2.41 \times 10^6\,\mathrm{kg\,m^2}$ over 141 s. How must (3.25) change?

As with translation, the obvious guess is wrong. If we write "$\mathrm{d}(\mathbf{J}\boldsymbol{\omega})/\mathrm{d}t = \boldsymbol{M}$" and set $\boldsymbol{M} = \mathbf{0}$, angular momentum is conserved and the rate must *grow* as $\mathbf{J}$ shrinks, like a skater pulling in her arms. That would mean a rocket's slightest wobble grows on its own as it burns propellant. Real rockets do not do that. The guess forgets the angular momentum that the exhaust carries away.

3.7.2 A control-volume derivation

Draw a surface around the vehicle, fixed to the body, through which mass leaves only at the nozzle exit. This is a *control volume*. For the matter inside it, we assume:

- the contents move as a rigid body (no slosh, no flow inside other than at a negligible speed);
- the exhaust leaves through a point $\boldsymbol{r}_e$ on the body axis, behind the centre of mass;
- the exhaust velocity relative to the nozzle, $\boldsymbol{u}_e$, is along the axis.

The rate of change of angular momentum of the contents equals the external moment minus the angular momentum flowing out each second:

$$\frac{\mathrm{d}}{\mathrm{d}t}\left(\mathbf{J}\boldsymbol{\omega}\right) = \boldsymbol{M} - \dot{m}_{\mathrm{out}}\,\boldsymbol{r}_e \times \boldsymbol{v}_e, \tag{3.29}$$

where $\boldsymbol{v}_e$ is the exhaust velocity relative to the centre of mass. That velocity has two parts: $\boldsymbol{u}_e$, which is parallel to $\boldsymbol{r}_e$ and so contributes nothing to $\boldsymbol{r}_e \times \boldsymbol{v}_e$; and $\boldsymbol{\omega} \times \boldsymbol{r}_e$, because the nozzle exit is swinging round as the vehicle turns. So the flux is

$$\dot{m}_{\mathrm{out}}\,\boldsymbol{r}_e \times (\boldsymbol{\omega} \times \boldsymbol{r}_e) = \dot{m}_{\mathrm{out}}\left(\|\boldsymbol{r}_e\|^2\,\mathbf{I} - \boldsymbol{r}_e\boldsymbol{r}_e^{\mathsf{T}}\right)\boldsymbol{\omega}.$$

For a nozzle on the axis at distance ℓ_e behind the centre of mass, this is $\dot{m}_{\mathrm{out}}\ell_e^2\,\boldsymbol{\omega}_\perp$, where $\boldsymbol{\omega}_\perp = (0, q, r)$ is the transverse (pitch and yaw) part of the angular velocity.

Now expand the left side in body axes. Unlike before, $\mathbf{J}$ is not constant, so the product rule gives an extra term:

$$\frac{\mathrm{d}^I(\mathbf{J}\boldsymbol{\omega})}{\mathrm{d}t} = \dot{\mathbf{J}}\boldsymbol{\omega} + \mathbf{J}\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega}).$$

Putting the pieces together:

$$\boxed{\mathbf{J}\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega}) = \boldsymbol{M} - \dot{\mathbf{J}}\,\boldsymbol{\omega} - \dot{m}_{\mathrm{out}}\,\ell_e^2\,\boldsymbol{\omega}_\perp.} \tag{3.30}$$

This is the equation we wrote in Chapter 1 as $\mathbf{J}\dot{\boldsymbol{\omega}} + \dot{\mathbf{J}}\boldsymbol{\omega} + \boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega}) = \boldsymbol{M}$, now with the jet-damping term that was promised there.

3.7.3 Reading the two extra terms

Jet damping, $-\dot{m}_{\text{out}}\ell_e^2\boldsymbol{\omega}_\perp$. When the vehicle pitches, the nozzle swings sideways. The exhaust leaving it has to be given that sideways speed, and the reaction pushes the nozzle back. The effect always opposes the transverse rotation, like a shock absorber, and it is strongest when the nozzle is far from the centre of mass.

The inertia-rate term, $-\dot{\mathbf{J}}\boldsymbol{\omega}$. Because $\dot{\mathbf{J}}$ is negative (the vehicle is getting lighter), this term *adds* to the rotation. It is the skater effect, and on its own it would make wobbles grow.

The net effect on the transverse motion is a damping coefficient

$$c = \dot{m}_{\text{out}}\ell_e^2 + \dot{J}_t, \qquad J_t\,\dot{\boldsymbol{\omega}}_\perp = \boldsymbol{M}_\perp - c\,\boldsymbol{\omega}_\perp + (\text{gyroscopic terms}), \tag{3.31}$$

which is positive (damping) when the nozzle is far enough behind the centre of mass that the jet damping beats the falling inertia.

> **Worked example**: Jet damping on LV-N1. At lift-off the centre of mass is at station 21.75 m and the nozzle exit at 40 m, so $\ell_e = 18.25\,\text{m}$, and
>
> $$\dot{m}_{\text{out}}\ell_e^2 = 723.67 \times 18.25^2 \approx 2.41 \times 10^5\,\text{kg m}^2/\text{s}.$$
>
> From the mass model, $\dot{J}_t \approx -1.37 \times 10^5\,\text{kg m}^2/\text{s}$ at lift-off. The net coefficient is $c \approx 1.04 \times 10^5\,\text{kg m}^2/\text{s}$, and the time it takes a free wobble to shrink by a factor e is
>
> $$\tau = \frac{J_t}{c} \approx \frac{8.59 \times 10^6}{1.04 \times 10^5} \approx 83\,\text{s}.$$
>
> At maximum dynamic pressure ($t \approx 52\,\text{s}$), the numbers are $\dot{m}_{\text{out}}\ell_e^2 \approx 1.45 \times 10^5$, $\dot{J}_t \approx -4.9 \times 10^4$ and $\tau \approx 41\,\text{s}$.

So jet damping is real but weak: it takes tens of seconds to act, while the flight-control system works in a second or two. It is not a substitute for control, but it does decide whether small disturbances slowly die out or slowly grow. The Python example in Section 3.10 shows how large the difference is over a whole burn.

> **Reality check**: A modelling choice, stated openly. Equation (3.30) rests on the assumptions listed above. A more complete treatment also includes the Coriolis forces on propellant flowing inside the vehicle (from the tanks to the engine), and the slow motion of the centre of mass relative to the structure. For a launch vehicle with modest internal flow speeds these are normally small, and (3.30) is a good engineering model. Different books arrange these terms in different ways; when you compare equations from two sources, check which effects each one keeps.

3.8 The complete six-degree-of-freedom equations

We can now write the whole simulator. The state is

$$\boldsymbol{x} = (\boldsymbol{r},\ \boldsymbol{v},\ \boldsymbol{q},\ \boldsymbol{\omega}) \in \mathbb{R}^{13},$$

plus the mass, which for a constant flow rate is simply $m(t) = m_0 - \dot{m}_{\text{out}} t$. Then:

Key idea: The six-degree-of-freedom equations of motion.

$$\dot{\boldsymbol{r}} = \boldsymbol{v} \qquad \text{position, inertial axes}$$

$$\dot{\boldsymbol{v}} = \frac{1}{m(t)}\,\mathbf{R}(\boldsymbol{q})\big(\boldsymbol{T} + \boldsymbol{F}_A\big) + \boldsymbol{g}(\boldsymbol{r}) \qquad \text{Newton with thrust, (3.19)}$$

$$\dot{\boldsymbol{q}} = \tfrac{1}{2}\,\boldsymbol{q} \odot (0,\ \boldsymbol{\omega}) \qquad \text{attitude, (3.15)}$$

$$\mathbf{J}\dot{\boldsymbol{\omega}} = \boldsymbol{M}_T + \boldsymbol{M}_A - \boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega}) - \dot{\mathbf{J}}\boldsymbol{\omega} - \dot{m}_{\text{out}} \ell_e^2 \boldsymbol{\omega}_\perp \qquad \text{Euler with variable mass, (3.30)}$$

Here $\mathbf{R}(\boldsymbol{q})$ turns body components into inertial ones (Chapter 2), $\boldsymbol{T}$ and $\boldsymbol{M}_T$ are the thrust and its moment about the centre of mass, and $\boldsymbol{F}_A$, $\boldsymbol{M}_A$ the aerodynamic force and moment. The equations themselves are now complete. What is still missing are the *models* that supply the right-hand sides:

- gravity $\boldsymbol{g}(\boldsymbol{r})$ — point mass and J_2, from Chapter 2;
- the air: density, pressure and speed of sound at each altitude — Chapter 4;
- aerodynamic forces and moments, with the centre of pressure — Chapter 4;
- thrust, its variation with altitude, and the gimbal actuator — Chapter 5;
- the mass properties $m(t)$, $s_{cg}(t)$, $\mathbf{J}(t)$ and $\dot{\mathbf{J}}(t)$ — built from the tank geometry in Chapter 2 and used here.

Chapter 6 assembles all of them into a working simulator.

3.9 Linearising: from six degrees of freedom to a simple pitch model

In Chapter 1 we designed a pitch controller for the model $\ddot{\theta} = \mu_\alpha \theta + \mu_c \delta$ with made-up values of μ_α and μ_c. We can now derive that model from the full equations, and compute real values.

Take the pitch row of Euler's law, ignore the slow mass terms over a few seconds, and keep only the two largest moments:

- the *aerodynamic* moment: normal force $\bar{q} S C_{N_\alpha} \alpha$ acting at the centre of pressure, a distance $s_{cg} - s_{cp}$ ahead of the centre of mass;
- the *thrust* moment: thrust T tilted by the small gimbal angle δ, acting a distance $\ell_g = s_g - s_{cg}$ behind the centre of mass.

Near a steady flight condition, a small pitch disturbance θ changes the angle of attack by the same amount, so

$$J_t \ddot{\theta} = \bar{q} S C_{N_\alpha} (s_{cg} - s_{cp})\,\theta + T \ell_g\,\delta \quad \Longrightarrow \quad \mu_\alpha = \frac{\bar{q} S C_{N_\alpha} (s_{cg} - s_{cp})}{J_t}, \qquad \mu_c = \frac{T \ell_g}{J_t}. \tag{3.32}$$

This is the simplest useful linearisation. It ignores how the trajectory itself bends, which is slow compared with the attitude motion.

Worked example: LV-N1 at maximum dynamic pressure. At $t \approx 52\,\mathrm{s}$ the simulation gives $\bar{q} = 56.9\,\mathrm{kPa}$ at Mach 1.89, where $C_{N_\alpha} = 2.42/\mathrm{rad}$ and the centre of pressure is at station 17.68 m. The centre of mass is at 25.86 m, so the static margin is 8.18 m, and $J_t = 3.88 \times 10^6\,\mathrm{kg\,m^2}$, $T = 2.15\,\mathrm{MN}$, $\ell_g = 13.14\,\mathrm{m}$. With $S = 7.07\,\mathrm{m^2}$:

$$\mu_\alpha = \frac{56\,870 \times 7.07 \times 2.42 \times 8.18}{3.88 \times 10^6} \approx 2.05/\mathrm{s^2}, \qquad \mu_c = \frac{2.15 \times 10^6 \times 13.14}{3.88 \times 10^6} \approx 7.29/\mathrm{s^2}.$$

The uncontrolled poles are $\pm\sqrt{2.05} \approx \pm 1.43$, so a pitch disturbance would double every $\ln 2 / 1.43 \approx 0.48\,\mathrm{s}$. The values used for the example in Chapter 1 ($\mu_\alpha = 4$, $\mu_c = 8$) were of the right size. By the stability condition of that chapter, any proportional gain above $\mu_\alpha / \mu_c \approx 0.28$ stabilises this flight condition.

Doing this at many points along the trajectory gives a family of linear models, one for each flight condition. That family is the starting point for gain scheduling in Chapter 11.

3.10 A Python example: what happens to a wobble during the burn

The code below follows a small pitch rate of 0.01 rad/s through the whole LV-N1 burn, with no external moment, under three models of the rotational dynamics:

- `rigid`: $J_t \dot{q} = 0$, mass changes ignored entirely;
- `naive`: $\mathrm{d}(J_t q)/\,\mathrm{d}t = 0$, the "skater" guess;
- `full`: equation (3.30), $J_t \dot{q} = -(\dot{J}_t + \dot{m}_{\mathrm{out}} \ell_e^2)\, q$.

It uses the same tank model as Chapter 2 and the RK4 integrator from Chapter 1.

```
import math

# LV-N1 notional vehicle (stations in metres from the nose)
m_dry, s_dry = 18000.0, 26.0 # dry mass [kg] and its CG station
m_p0, s_top, s_bot, r_p = 102000.0, 8.0, 34.0, 1.45
L, R, s_exit = 40.0, 1.5, 40.0
mdot = 723.67 # propellant flow [kg/s]
t_b = m_p0 / mdot # burn time [s]

def props(t):
    """mass, CG station and transverse inertia J_t about the CG at time t
       """
    mp = max(m_p0 - mdot * t, 0.0)
    Lp = mp / m_p0 * (s_bot - s_top)
    sp = s_bot - Lp / 2
    m = m_dry + mp
    scg = (m_dry * s_dry + mp * sp) / m
    Jt = (m_dry * (3*R*R + L*L) / 12 + m_dry * (s_dry - scg)**2
          + mp * (3*r_p*r_p + Lp*Lp) / 12 + mp * (sp - scg)**2)
    return m, scg, Jt
```

```
def wdot(t, w, model):
    m, scg, Jt = props(t)
    h = 1e-4
    Jt_dot = (props(t + h)[2] - props(t - h)[2]) / (2 * h)
    le = s_exit - scg
    if model == 'full': # J w' = -(J_dot + mdot*le^2) w
        return -(Jt_dot + mdot * le * le) * w / Jt
    if model == 'naive': # d(J w)/dt = 0 -> J w' = -J_dot w
        return -Jt_dot * w / Jt
    return 0.0 # 'rigid': J w' = 0

for model in ('rigid', 'naive', 'full'):
    w, t, dt = 0.01, 0.0, 0.01 # initial pitch rate 0.01 rad/s, no
        external moment
    while t < t_b - 1e-9:
        h = min(dt, t_b - t)
        k1 = wdot(t, w, model)
        k2 = wdot(t + h/2, w + h/2*k1, model)
        k3 = wdot(t + h/2, w + h/2*k2, model)
        k4 = wdot(t + h, w + h*k3, model)
        w += h/6 * (k1 + 2*k2 + 2*k3 + k4)
        t += h
    print(f"{model:6s}: pitch rate at burnout = {w:.5f} rad/s")
```

Running it prints:

```
rigid : pitch rate at burnout = 0.01000 rad/s
naive : pitch rate at burnout = 0.03565 rad/s
full : pitch rate at burnout = 0.00024 rad/s
```

The three answers are completely different. The rigid model keeps the wobble unchanged. The naive model makes it grow by a factor of 3.6 — exactly the ratio of lift-off to burnout inertia, 8.59/2.41, as conservation of $J_t q$ requires. The full model, with jet damping, shrinks it to about 2% of its starting value. The equation you choose decides whether your simulated rocket is self-correcting or self-destructing.

Summary

- A rigid body has six degrees of freedom. Slosh, bending and nozzle inertia are left out on purpose; mass loss cannot be.
- Points on a turning body move with $\boldsymbol{v}_P = \boldsymbol{v}_C + \boldsymbol{\omega} \times \boldsymbol{r}$ and accelerate with extra tangential and centripetal terms. An IMU away from the centre of mass feels these lever-arm accelerations.
- Accelerometers measure specific force $\boldsymbol{f} = \boldsymbol{a} - \boldsymbol{g}$, never gravity. Navigation must add a gravity model back.
- Direction cosine matrices change as $\dot{\mathbf{T}}^{BI} = -[\boldsymbol{\omega}\times]\mathbf{T}^{BI}$ and drift from orthogonal when integrated; quaternions are easier to keep healthy.

- Newton's law in body axes gains the term $\boldsymbol{\omega} \times \boldsymbol{v}$; pitching the vehicle is what creates angle of attack.
- For a rocket, $m\dot{\boldsymbol{v}} = \boldsymbol{F}_{\text{ext}} + \boldsymbol{T}$, where thrust includes the pressure term $(p_e - p_a)A_e$. "$F = \mathrm{d}(mv)/\mathrm{d}t$" is wrong.
- The velocity budget splits ideal Δv into gravity, drag, back-pressure and steering losses; for LV-N1 gravity loss is about 17%.
- Euler's law $\mathbf{J}\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times \mathbf{J}\boldsymbol{\omega} = \boldsymbol{M}$ couples the axes; a rolling rocket turns pitch into yaw at the rate $\lambda = p(J_t - J_x)/J_t$.
- With variable mass, Euler's law gains $-\dot{\mathbf{J}}\boldsymbol{\omega}$ (which makes wobbles grow) and jet damping $-\dot{m}_{\text{out}}\ell_e^2\boldsymbol{\omega}_\perp$ (which makes them shrink). For LV-N1 the net effect is weak damping, with a time constant of roughly 25 to 85 seconds depending on the moment in the burn.
- Linearising the pitch equation gives μ_α and μ_c; at maximum dynamic pressure LV-N1 has $\mu_\alpha \approx 2.05$ and $\mu_c \approx 7.29\ 1/\mathrm{s}^2$.

Exercises

3.1 Lever arm. An IMU sits 6 m ahead of the centre of mass. The vehicle yaws at $r = 0.08\,\mathrm{rad/s}$, and r is constant. Find the acceleration of the IMU relative to the centre of mass. Which way does it point?

3.2 Specific force. During a steady vertical climb, an accelerometer along the nose reads $20\,\mathrm{m/s^2}$. What is the vehicle's actual acceleration? What would the accelerometer read during free fall after engine cut-off?

3.3 Gyroscopes on the pad. At latitude 45°, what does a gyroscope pointing north and horizontal read? One pointing straight up? (Hint: split $\boldsymbol{\omega}_E$ into local components, as in Chapter 2.)

3.4 DCM drift. Show that the rows of $\mathbf{I} - [\boldsymbol{\omega}\times]\Delta t$ have length $\sqrt{1 + (\omega_j^2 + \omega_k^2)\Delta t^2}$ for the appropriate j, k. Estimate the growth of a row's length after 14 000 steps with $\|\boldsymbol{\omega}\|\,\Delta t = 10^{-3}$.

3.5 Body-axis equations. Derive (3.18) by writing out $\boldsymbol{\omega} \times \boldsymbol{v}$ in components. Then explain in words why $\dot{v} = -ru$ for a vehicle yawing with no side force.

3.6 The sandbag. A 100 kg cart moves at 5 m/s and drops sand straight down (relative to the cart) at 2 kg/s. Use a momentum balance on cart plus sand to show that the cart's speed does not change. Then show what "$F = \mathrm{d}(mv)/\mathrm{d}t$" would wrongly predict.

3.7 Thrust and altitude. Using LV-N1's numbers, find the thrust at 11 km, where $p_a = 22\,632\,\mathrm{Pa}$. By what percentage is it higher than at sea level?

3.8 Gravity loss. A vehicle flies straight up for 20 s and then follows a path whose flight-path angle falls linearly from 90° to 30° over the next 120 s. With $g = 9.81\,\mathrm{m/s^2}$, estimate the gravity loss.

3.9 Torque-free motion. Check by differentiation that $q = q_0 \cos \lambda t$, $r = -q_0 \sin \lambda t$ solves the moment-free equations of Section 3.6. Show that $q^2 + r^2$ is constant.

3.10 Spin. Repeat the rolling-LV-N1 example at burnout, where $J_x \approx 2.03 \times 10^4$ and $J_t \approx 2.41 \times 10^6\,\mathrm{kg\,m^2}$. Does the coupling get better or worse?

3.11 Jet damping. At $t = 100\,\mathrm{s}$, LV-N1 has its centre of mass at station 28.63 m, $J_t = 2.77 \times 10^6\,\mathrm{kg\,m^2}$ and $\dot{J}_t \approx -3.8 \times 10^3\,\mathrm{kg\,m^2/s}$. Compute ℓ_e, the jet-damping coefficient, the net coefficient c and the time constant. Why is $\dot{J}_t$ so small at this moment? (Hint: look at how the centre of mass moves near this time.)

3.12 Linearisation. Using (3.32), how does μ_α change if the centre of pressure moves 1 m further forward? What minimum proportional gain does the pitch controller of Chapter 1 now need?

3.13 Python. Modify the example to start with $q = 0.01\,\mathrm{rad/s}$ at $t = 60\,\mathrm{s}$ instead of at lift-off. How much damping is left by burnout? Then move the nozzle exit 2 m further aft and repeat.

Reading for this chapter

- P. H. Zipfel, *Modeling and Simulation of Aerospace Vehicle Dynamics*, 2nd ed.: Chapter 4 (kinematics of translation and rotation, including attitude determination with quaternions), Chapter 5 (translational dynamics) and Chapter 6 (attitude dynamics: inertia tensor, angular momentum, Euler's law and gyrodynamics).
- K. J. Åström and R. M. Murray, *Feedback Systems*: Chapter 2 (system modelling, including the balance-system and vectored-thrust examples) and Section 4.1 (solving differential equations).
- R. R. Labbe, *Kalman and Bayesian Filters in Python*: Section 7.3 (modelling dynamic systems) and Section 7.6 (numerical integration of differential equations).

Chapter 4

Atmosphere and Aerodynamics

> *A rocket spends only two minutes in the air, but those two minutes decide whether it survives. This chapter is about the invisible thing it is pushing through.*

Chapter 3 gave us the equations of motion. They have holes in them: the aerodynamic force $\boldsymbol{F}_A$ and moment $\boldsymbol{M}_A$, and the air pressure p_a inside the thrust equation. This chapter fills those holes. By the end you will be able to answer, with numbers, four questions about any point on the trajectory:

1. How thick is the air here? (*density, pressure, temperature* — Sections 4.1–4.3)
2. How fast is the vehicle moving *through the air*, not over the ground? (*wind and the air-relative velocity* — Section 4.4)
3. How hard is the air pushing? (*dynamic pressure and Mach number* — Section 4.5)
4. Which way does that push try to turn the vehicle? (*aerodynamic coefficients, centre of pressure, stability* — Sections 4.6–4.7)

We then write all of it as code (Section 4.9), and finish with two numerical tools from computational statistics that every simulator writer needs: *finding roots* and *fitting a model to data by maximum likelihood* (Section 4.10), in both R and Python.

You need only school algebra, the idea of a derivative and an integral, and the vectors of Chapter 2. Every formula is derived step by step, and every step is followed by numbers.

4.1 Air is a gas: pressure, density and temperature

Three numbers describe the air at any place:

- the **pressure** p, in pascals ($\mathrm{Pa} = \mathrm{N/m^2}$): how hard the air pushes on each square metre of any surface it touches;
- the **density** ρ (the Greek letter "rho"), in $\mathrm{kg/m^3}$: how many kilograms of air are packed into one cubic metre;

- the **temperature** T, in kelvin (K): how fast the air molecules are jiggling. Kelvin is Celsius plus 273.15, so $15\,^\circ\mathrm{C} = 288.15\,\mathrm{K}$.

They are not independent. For air, the three are tied together by the *ideal gas law*:

> **Key idea**: The ideal gas law.
>
> $$p = \rho\, R\, T, \qquad R = 287.053\,\mathrm{J/(kg\,K)} \text{ (for air)}. \tag{4.1}$$
>
> Know any two of p, ρ, T, and you know the third.

Why should this be true? Pressure is the drumming of molecules on a wall. More molecules in the box (higher ρ) means more hits per second. Faster molecules (higher T) means each hit is harder *and* hits come more often. So pressure grows with both, and the simplest possible rule — proportional to each — turns out to be almost exactly right for air at the conditions a rocket meets. The constant R is just the number that makes the units work for air.

> **Worked example**: Air at sea level. On a standard day at sea level, $p_0 = 101\,325\,\mathrm{Pa}$ and $T_0 = 288.15\,\mathrm{K}$. Rearranging (4.1) for density:
>
> $$\rho_0 = \frac{p_0}{R\,T_0} = \frac{101\,325}{287.053 \times 288.15} = \frac{101\,325}{82\,714.3} = 1.225\,\mathrm{kg/m^3}.$$
>
> A cubic metre of air — a box you could just about sit in — weighs a little over a kilogram. The air inside an ordinary room weighs more than the person standing in it.

4.2 Why the air gets thinner as you climb

4.2.1 The weight of the air above you

Picture a thin flat slab of air, one square metre across and $\mathrm{d}h$ metres thick, sitting at altitude h. Three vertical forces act on it:

- the air below pushes it *up* with the pressure $p(h)$, times its area 1;
- the air above pushes it *down* with the pressure $p(h + \mathrm{d}h)$;
- its weight pulls it *down*: mass $\rho \times (1 \times \mathrm{d}h)$, times g.

The air is not flying away and not collapsing, so the forces balance:

$$p(h) - p(h + \mathrm{d}h) - \rho\, g\, \mathrm{d}h = 0.$$

The difference $p(h + \mathrm{d}h) - p(h)$ is the small change in pressure, $\mathrm{d}p$. So $-\mathrm{d}p = \rho g\, \mathrm{d}h$, or

> **Key idea**: Hydrostatic balance.
>
> $$\frac{\mathrm{d}p}{\mathrm{d}h} = -\rho\, g. \tag{4.2}$$
>
> Climb one metre and the pressure drops by the weight of one cubic metre of air.

In plain words: **the pressure at any height is the weight of all the air above it.** Near the ground, $\rho g = 1.225 \times 9.81 \approx 12\,\mathrm{Pa/m}$, so climbing one storey (about 3 m) lowers the pressure by about 36 Pa — one part in three thousand. That is why your ears pop in a fast lift.

4.2.2 Combining the two laws

Equation (4.2) has two unknowns, p and ρ. The gas law removes one: from (4.1), $\rho = p/(RT)$. Substituting,

$$\frac{\mathrm{d}p}{\mathrm{d}h} = -\frac{g}{R\,T}\,p \qquad \Longleftrightarrow \qquad \frac{\mathrm{d}p}{p} = -\frac{g}{R\,T}\,\mathrm{d}h. \tag{4.3}$$

The second form is the useful one: the *fractional* drop in pressure, $\mathrm{d}p/p$, is proportional to the climb $\mathrm{d}h$. Everything now depends on how T changes with height. Nature uses two simple patterns, and we solve each.

4.2.3 Pattern 1: constant temperature (an isothermal layer)

If T is constant, the right-hand side of (4.3) is a constant times $\mathrm{d}h$. It helps to give the constant a name:

$$H = \frac{R\,T}{g} \qquad \text{(the } \textit{scale height}\text{, in metres).} \tag{4.4}$$

Now integrate both sides of $\mathrm{d}p/p = -\,\mathrm{d}h/H$ from a base altitude h_b (where $p = p_b$) up to h:

$$\int_{p_b}^{p} \frac{\mathrm{d}p'}{p'} = -\frac{1}{H}\int_{h_b}^{h} \mathrm{d}h' \quad\Longrightarrow\quad \ln p - \ln p_b = -\frac{h-h_b}{H} \quad\Longrightarrow\quad \ln\frac{p}{p_b} = -\frac{h-h_b}{H}.$$

Undo the logarithm by taking $e^{(\cdot)}$ of both sides:

> **Key idea**: Isothermal layer.
>
> $$p(h) = p_b\,\exp\!\left(-\frac{h-h_b}{H}\right), \qquad \rho(h) = \frac{p(h)}{R\,T}. \tag{4.5}$$
>
> Every time you climb one scale height H, the pressure is divided by $e \approx 2.718$.

> **Worked example**: Scale heights. At sea-level temperature, $H = 287.053 \times 288.15/9.80665 = 8435\,\mathrm{m}$. In the cold stratosphere, $T = 216.65\,\mathrm{K}$ and $H = 287.053 \times 216.65/9.80665 = 6342\,\mathrm{m}$. Colder air is denser for its pressure, so it "piles up" less high and the pressure falls faster.
>
> *Check against the standard atmosphere.* Between 11 and 20 km the temperature is constant at 216.65 K and the pressure at 11 km is 22 632 Pa. Climbing 9000 m:
>
> $$p(20\,\mathrm{km}) = 22\,632 \times e^{-9000/6342} = 22\,632 \times e^{-1.4192} = 22\,632 \times 0.2419 = 5475\,\mathrm{Pa},$$
>
> which is the tabulated value, 5474.9 Pa.

4.2.4 Pattern 2: temperature changing in a straight line (a gradient layer)

In the lowest layer, the troposphere, the air cools steadily as you climb: about 6.5 K for every kilometre. Write this as

$$T(h) = T_b + L\,(h - h_b),$$

where L is the *lapse rate* ($L = -0.0065\,\mathrm{K/m}$ in the troposphere; negative means "cooling with height"). Put this T into (4.3):

$$\frac{\mathrm{d}p}{p} = -\frac{g}{R}\,\frac{\mathrm{d}h}{T_b + L(h - h_b)}.$$

Step 1: change variable. The denominator is T itself, and $\mathrm{d}T = L\,\mathrm{d}h$, so $\mathrm{d}h = \mathrm{d}T/L$:

$$\frac{\mathrm{d}p}{p} = -\frac{g}{R\,L}\,\frac{\mathrm{d}T}{T}.$$

Step 2: integrate both sides from the base of the layer (p_b, T_b):

$$\ln\frac{p}{p_b} = -\frac{g}{R\,L}\,\ln\frac{T}{T_b}.$$

Step 3: use $a\ln x = \ln x^a$ and undo the logarithm:

Key idea: Gradient layer.

$$p(h) = p_b\left(\frac{T(h)}{T_b}\right)^{-g/(RL)}, \qquad T(h) = T_b + L(h - h_b). \tag{4.6}$$

Worked example: Pressure at the top of the troposphere. At 11 km, $T = 288.15 - 0.0065 \times 11\,000 = 216.65\,\mathrm{K}$. The exponent is

$$-\frac{g}{RL} = -\frac{9.80665}{287.053 \times (-0.0065)} = +5.2559.$$

So

$$p = 101\,325 \times \left(\frac{216.65}{288.15}\right)^{5.2559} = 101\,325 \times (0.75187)^{5.2559}.$$

To raise 0.75187 to the power 5.2559, take the logarithm: $\ln 0.75187 = -0.28520$; times 5.2559 gives -1.4990; and $e^{-1.4990} = 0.22336$. Hence $p = 101\,325 \times 0.22336 = 22\,632\,\mathrm{Pa}$: at the cruising height of an airliner, the air pressure is less than a quarter of what it is on the ground.

4.3 The U.S. Standard Atmosphere 1976

The real atmosphere changes with the weather, the season and the latitude. Engineers agree to use one *standard* average instead, so that everyone's numbers can be compared. The most widely used is the U.S. Standard Atmosphere 1976 ("US76"). Below 86 km it is built entirely from the two patterns above: seven layers, each with a straight-line temperature.

To find the air at any altitude:

Layer	Base height h_b [km]	Base temperature T_b [K]	Lapse rate L [K/km]	Base pressure p_b [Pa]	Name
1	0	288.15	-6.5	101 325	troposphere
2	11	216.65	0.0	22 632.1	tropopause
3	20	216.65	$+1.0$	5474.89	stratosphere
4	32	228.65	$+2.8$	868.019	stratosphere
5	47	270.65	0.0	110.906	stratopause
6	51	270.65	-2.8	66.9389	mesosphere
7	71	214.65	-2.0	3.956 42	mesosphere

Table 4.1: The seven layers of the U.S. Standard Atmosphere 1976 below 86 km. Each base pressure is computed from the layer below it with (4.5) or (4.6); only the sea-level value is given.

1. find the layer whose base is the highest one below you;
2. compute T from the straight line of that layer;
3. compute p with (4.5) if $L = 0$, otherwise with (4.6);
4. compute $\rho = p/(RT)$ from the gas law.

Only the sea-level pressure is ever "given"; every other base pressure in Table 4.1 is produced by marching up from the ground, one layer at a time. That is exactly what our code will do.

4.3.1 A small correction: geopotential altitude

Gravity weakens as you climb: $g(h) = g_0 \left(r_0/(r_0+h)\right)^2$, with $r_0 = 6356.766$ km. The formulas above assumed a constant g_0. Rather than redo them, US76 uses a trick: it measures height with a slightly "squeezed" ruler, the *geopotential altitude*

$$z = \frac{r_0\, h}{r_0 + h}, \tag{4.7}$$

chosen so that $g_0\ \mathrm{d}z = g(h)\ \mathrm{d}h$ exactly.[1] The layer heights in Table 4.1 are geopotential heights. At $h = 11$ km, $z = 10.981$ km — a 19 m difference, small but enough to change the temperature in the fourth digit. At 80 km the difference is a full kilometre.

4.3.2 The speed of sound

Sound is a pressure wave carried by molecules bumping into each other, so it travels faster when they move faster — when the air is warmer:

$$a = \sqrt{\gamma\, R\, T}, \qquad \gamma = 1.4 \text{ for air.} \tag{4.8}$$

At sea level, $a = \sqrt{1.4 \times 287.053 \times 288.15} = \sqrt{115\,800} = 340.3$ m/s. In the stratosphere, $a = \sqrt{1.4 \times 287.053 \times 216.65} = 295.1$ m/s. Notice that a depends on temperature only, not on pressure or density.

[1] Check: $\mathrm{d}z/\mathrm{d}h = r_0^2/(r_0+h)^2$, so $g_0\,\mathrm{d}z = g_0\,(r_0/(r_0+h))^2\,\mathrm{d}h = g(h)\,\mathrm{d}h$.

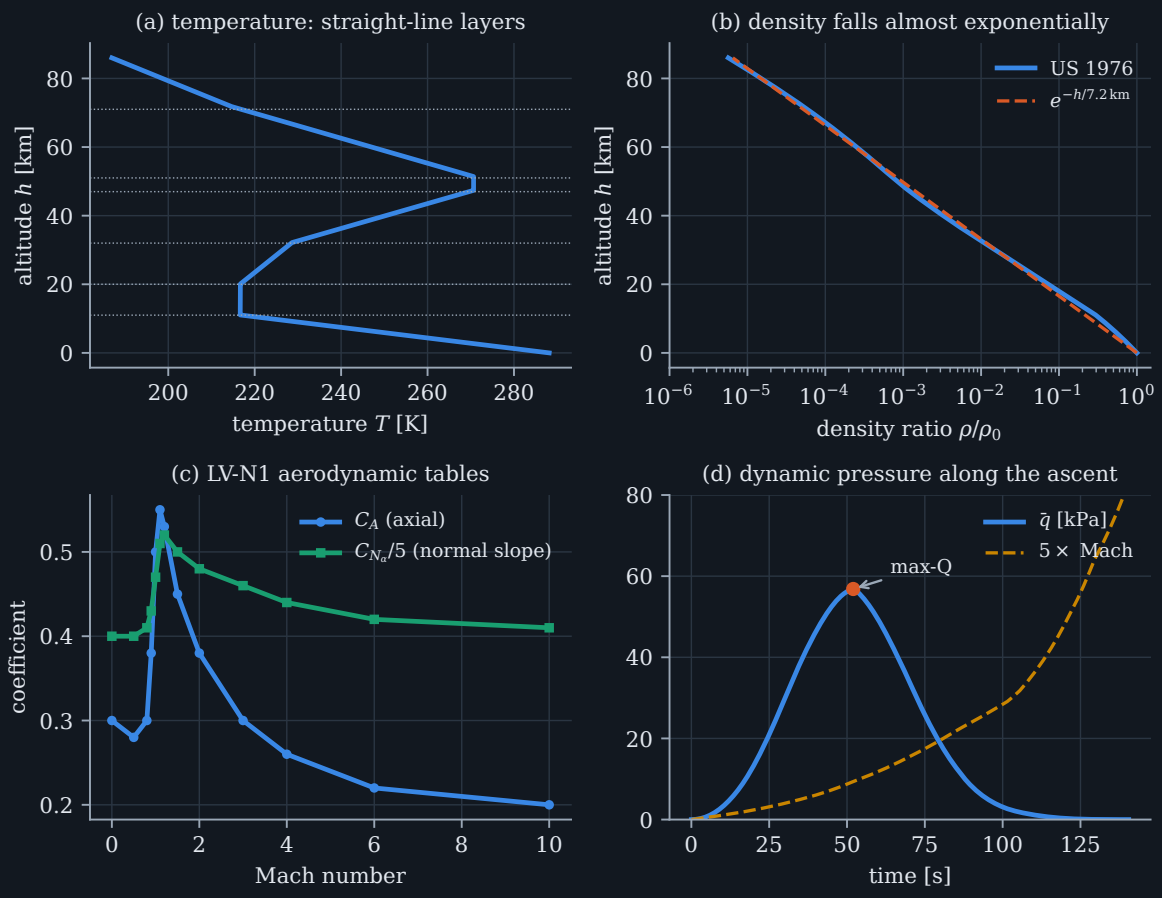


Figure 4.1: (a) Temperature in US76 is made of straight pieces; dotted lines mark the layer bases. (b) Density on a logarithmic axis: the seven layers together are close to one exponential. (c) LV-N1's axial-force coefficient C_A and normal-force slope C_{N_α} against Mach number; both peak just above Mach 1. (d) Dynamic pressure along the LV-N1 ascent, from the simulator of Chapter 6: it rises, peaks at about 57 kPa near 52 s ("max-Q"), and fades as the air runs out. The dashed curve is five times the Mach number.

4.3.3 The simplest model: one exponential

Figure 4.1(b) shows something remarkable: on a logarithmic scale, the density of all seven layers lies very close to a single straight line. That line is the *exponential atmosphere*,

$$\rho(h) \approx \rho_0\, e^{-h/H}, \qquad H \approx 7.2\,\text{km}, \tag{4.9}$$

which is equation (4.5) pretending that the whole sky has one average temperature. It is wrong by up to about 30% at some heights, but it has one parameter and can be done in your head. It is the right tool for a first estimate, and in Section 4.10 we will *measure* the best H from data.

4.4 Wind, and moving through the air

The air does not care how fast the rocket moves over the ground. It cares how fast the rocket moves *through it*. These are different for two reasons: the air turns with the Earth, and the air blows.

Key idea: Air-relative velocity.

$$v_{\text{air}} = v - \omega_E \times r - w, \tag{4.10}$$

where v is the inertial velocity, $\omega_E \times r$ is the velocity of the air that is simply turning with the Earth (the transport theorem of Chapter 2), and w is the wind relative to the ground. All aerodynamics uses v_{air}; gravity and the equations of motion use v.

Worked example: Standing still is not standing still. On the pad at latitude 28.5°, the rocket is not moving relative to the ground, so $v_{\text{air}} = \mathbf{0}$ on a calm day. But in inertial space it is moving east with the ground at

$$\omega_E \, R_E \cos(\text{latitude}) = 7.292 \times 10^{-5} \times 6\,378\,137 \times \cos 28.5^\circ = 408.7\,\text{m/s}.$$

If a simulator forgot the $\omega_E \times r$ term, the vehicle would feel a 409 m/s — Mach 1.2 — gale on the launch pad.

Modelling wind. For simulation, wind is usually given as a *profile*: speed and direction as functions of altitude. The strongest winds sit in the jet stream, around 10 km to 14 km, which is unfortunately also where the dynamic pressure is highest. Our simulator uses a smooth bump that is zero on the ground, peaks at 12 km and fades to zero by 25 km, scaled by a peak speed. In Chapter 6 we will choose the peak speed at random, with the methods of Section 2.11, to see how the vehicle copes with many different days.

4.5 Mach number and dynamic pressure

Two numbers made from the air-relative speed $V = \|v_{\text{air}}\|$ summarise "how violent" the air is.

Mach number.

$$\text{Ma} = \frac{V}{a}. \tag{4.11}$$

It compares the vehicle's speed with the speed at which the air can "get out of the way" (the speed of pressure waves). Below Mach 1 (*subsonic*) the air ahead feels the vehicle coming and starts to move aside early. Above Mach 1 (*supersonic*) it gets no warning; it is pushed aside suddenly, in a shock wave. Near Mach 1 (*transonic*, roughly 0.8 to 1.2) both happen at once, the flow is unsteady and the forces jump. That is why the curves in Fig. 4.1(c) peak there.

Dynamic pressure.

$$\bar{q} = \tfrac{1}{2}\,\rho\,V^2. \tag{4.12}$$

Where does the $\frac{1}{2}\rho V^2$ come from? It is the kinetic energy of one cubic metre of the oncoming air (mass ρ, speed V). When that air is brought to rest against the nose, its kinetic energy turns into extra pressure, so $\bar{q}$ is literally "the pressure the motion adds". Every aerodynamic force is $\bar{q}$ times an area times a pure number, as we will see next.

> **Worked example**: Dynamic pressure, from a car to a rocket. A car at 100 km/h = 27.8 m/s at sea level: $\bar{q} = 0.5 \times 1.225 \times 27.8^2 = 473\,\text{Pa}$.
>
> LV-N1 at $t = 52$ s: altitude 10.9 km, $\rho = 0.3695\,\text{kg/m}^3$, $V = 554.6$ m/s:
>
> $$\bar{q} = 0.5 \times 0.3695 \times 554.6^2 = 0.5 \times 0.3695 \times 307\,581 = 56.8\,\text{kPa},$$
>
> about 120 times the car. The speed of sound there is 295.6 m/s, so Ma $= 554.6/295.6 = 1.88$.

Why there is a "max-Q". Along the ascent, V grows while ρ shrinks. At first the speed wins: V^2 grows from zero. Later the thinning air wins: ρ falls by a factor of e every 7 km. In between, $\bar{q}$ must reach a peak, Fig. 4.1(d). For LV-N1 the peak is 56.8 kPa at 52 s. Max-Q is the moment of greatest aerodynamic load, and the vehicle's structure, its control system, and the choice of trajectory are all designed around it.

4.6 Aerodynamic forces on a slender body

4.6.1 Angle of attack

The *angle of attack* α is the angle between the vehicle's nose direction (body x axis) and the oncoming air. Write the air-relative velocity in body axes, $\boldsymbol{v}^B_{\text{air}} = (u, v, w)$. The part along the nose is u; the part across the body is $\sqrt{v^2 + w^2}$. Then

$$\alpha = \operatorname{atan2}\!\left(\sqrt{v^2 + w^2},\; u\right). \tag{4.13}$$

($\operatorname{atan2}(y, x)$ is the angle of the point (x, y), which works in every quadrant.) A rocket is round, so it does not matter *which way* across the body the air comes from; only the total angle matters. This is the *total angle of attack*.

4.6.2 Axial and normal force

For a long thin body we split the aerodynamic force into two parts, measured in body axes:

- the **axial force** A, along the body, pushing backwards (mostly friction and the pressure on the nose and base);
- the **normal force** N, across the body, in the plane that contains the body axis and the wind (the air pushing sideways on a body that is not pointing straight into it).

Each is written as dynamic pressure, times a reference area S, times a dimensionless *coefficient*:

> **Key idea**: Aerodynamic force.
>
> $$A = \bar{q}\, S\, C_A(\mathrm{Ma}), \qquad N = \bar{q}\, S\, C_{N_\alpha}(\mathrm{Ma})\ \alpha. \tag{4.14}$$
>
> The reference area is the cross-section, $S = \pi(1.5\,\mathrm{m})^2 = 7.07\,\mathrm{m}^2$ for LV-N1. All the difficult physics is hidden inside the coefficients, which come from wind tunnels and computer fluid dynamics and are stored as tables against Mach number.

The normal force is proportional to α because, for small angles, the sideways part of the wind is proportional to α; C_{N_α} is the slope ("per radian"). For a slender body this straight line is good up to roughly 5°–8°, which is more than a launch vehicle should ever see at high $\bar{q}$.

Directions. The axial force points along $-x_B$. The normal force points against the cross-flow: if the air arrives with cross components (v, w), the normal force is $-N\,(0, v, w)/\sqrt{v^2 + w^2}$. In body axes,

$$\boldsymbol{F}_A^B = \left(-A,\ \ -N\frac{v}{\sqrt{v^2+w^2}},\ \ -N\frac{w}{\sqrt{v^2+w^2}}\right). \tag{4.15}$$

Lift and drag. Aircraft people prefer *drag* D (along the airflow) and *lift* L (across the airflow). They are the same force, measured in axes rotated by α:

$$D = A\cos\alpha + N\sin\alpha, \qquad L = N\cos\alpha - A\sin\alpha.$$

At $\alpha = 0$ they agree: $D = A$ and $L = N = 0$.

> **Worked example**: LV-N1 at max-Q with a 2° angle of attack. From the previous example, $\bar{q} = 56.82\,\mathrm{kPa}$ and $\mathrm{Ma} = 1.876$, so $\bar{q}S = 56\,820 \times 7.0686 = 401.6\,\mathrm{kN}$. Reading the tables of Fig. 4.1(c) (Section 4.8 shows how), $C_A = 0.3974$ and $C_{N_\alpha} = 2.425/\mathrm{rad}$. With $\alpha = 2^\circ = 0.034\,91\,\mathrm{rad}$:
>
> $$A = 401\,640 \times 0.3974 = 159.6\,\mathrm{kN}, \qquad N = 401\,640 \times 2.425 \times 0.03491 = 34.0\,\mathrm{kN}.$$
>
> The drag is about 7% of the thrust, 2.15 MN. The normal force is smaller, but as we see next, its *position* makes it dangerous.

4.7 Centre of pressure and static stability

4.7.1 Where does the force act?

The air presses all over the body, but for moments we can replace all those small pushes by one force acting at one point, the *centre of pressure* (CP). Its station s_{cp} (metres back from the nose, as in Chapter 3) is also a table against Mach number. The moment of the aerodynamic force about the centre of mass (station s_{cg}) is then

$$\boldsymbol{M}_A = \boldsymbol{r}_{cp} \times \boldsymbol{F}_A^B, \qquad \boldsymbol{r}_{cp} = (s_{cg} - s_{cp},\ 0,\ 0). \tag{4.16}$$

(Stations grow backwards while x_B points forwards; so if the CP is *ahead* of the CG, $s_{cp} < s_{cg}$ and $\boldsymbol{r}_{cp}$ points forwards.) Working out the cross product with the rule of Chapter 2,

$$\boldsymbol{M}_A = \big(0,\ -(s_{cg} - s_{cp})F_z,\ (s_{cg} - s_{cp})F_y\big).$$

4.7.2 Stable or unstable?

Think of a dart. Its feathers put the CP *behind* the CG. If the dart tips its nose up, the air pushes on the feathers and swings the nose back down: **CP behind CG is statically stable.** Now throw the dart backwards, feathers first. The CP is now ahead of the CG, and any small tilt is *amplified*: **CP ahead of CG is statically unstable.**

The distance between them is the *static margin*, often quoted in body diameters ("calibres"):

$$\text{static margin} = s_{cp} - s_{cg} \qquad (>0 \text{ stable},\ <0 \text{ unstable}).$$

Worked example: LV-N1 is unstable. At max-Q the tables give $s_{cp} = 17.68\,\text{m}$ and the CG is at $s_{cg} = 25.84\,\text{m}$. The CP is 8.17 m — 2.7 calibres — ahead of the CG. With the normal force of the last example,

$$|M_A| = 8.17 \times 34\,000 = 278\,\text{kN m},$$

and it acts to *increase* α. Without control, the vehicle would turn sideways and break up within a second or two. This is the $\mu_\alpha > 0$ of Eq. (3.32): we have now computed it from the aerodynamics instead of assuming it.

Can the engine win? The gimbal of Chapter 5 can tilt the 2.15 MN thrust by up to 6° at a lever arm of about 13.2 m, giving $2.15 \times 10^6 \times 13.16 \times \sin 6^\circ = 2.96\,\text{MN m}$ — about ten times the aerodynamic moment. The vehicle can be held, but only by active control, every few hundredths of a second, for the whole flight through the air.

Reality check. Almost all large launch vehicles are aerodynamically unstable for at least part of their flight, because fins heavy enough to fix it would cost more performance than the control system does. Keeping α small near max-Q is therefore a design rule: engineers limit the product $\bar{q}\,\alpha$ (the "q-alpha" load), typically to a few thousand pascal-radians.

4.8 Reading a table: linear interpolation

The coefficients are stored only at a few Mach numbers (13 for LV-N1). Between two stored points (x_i, y_i) and (x_{i+1}, y_{i+1}) we draw a straight line. If x lies between them, first find how far along the gap it is,

$$w = \frac{x - x_i}{x_{i+1} - x_i} \qquad (0 \text{ at the left end},\ 1 \text{ at the right}),$$

and then go that fraction of the way up (or down) from y_i to y_{i+1}:

$$y = y_i + w\,(y_{i+1} - y_i). \tag{4.17}$$

Worked example: The axial coefficient at Mach 1.876. The table has $C_A = 0.45$ at Mach 1.5 and 0.38 at Mach 2.0. So $w = (1.876 - 1.5)/(2.0 - 1.5) = 0.752$ and

$$C_A = 0.45 + 0.752 \times (0.38 - 0.45) = 0.45 - 0.0526 = 0.3974.$$

The same recipe gives $C_{N_\alpha} = 2.5 + 0.752 \times (2.4 - 2.5) = 2.425$ and $s_{cp} = 17.6 + 0.752 \times 0.1 = 17.675\,\text{m}$.

Outside the table we hold the end value ("flat extrapolation"): it is safer to repeat the last measured number than to extend a straight line into the unknown.

4.9 Putting it in code

Here is the whole chapter as two small functions. The first is US76: it builds the base pressures once by marching up the layers, then answers any altitude with the recipe of Section 4.3. We give it in both languages, because the statistics section below needs it in R too.

R

```r
# U.S. Standard Atmosphere 1976, up to 86 km
g0 <- 9.80665; Rair <- 287.053; r0 <- 6356766
hb <- c(0, 11000, 20000, 32000, 47000, 51000, 71000)
Tb <- c(288.15, 216.65, 216.65, 228.65, 270.65, 270.65, 214.65)
Lb <- c(-0.0065, 0, 0.001, 0.0028, 0, -0.0028, -0.002)
pb <- numeric(7); pb[1] <- 101325 # base pressures
for (i in 2:7) {
  dh <- hb[i] - hb[i-1]
  pb[i] <- if (Lb[i-1] == 0)
    pb[i-1] * exp(-g0 * dh / (Rair * Tb[i-1])) else
    pb[i-1] * (1 + Lb[i-1] * dh / Tb[i-1])^(-g0 / (Rair * Lb[i-1]))
}
atmos <- function(h) { # h: geometric altitude [m]
  z <- r0 * h / (r0 + h) # geopotential altitude
  i <- max(which(hb <= z)) # which layer?
  if (Lb[i] == 0) {
    T <- Tb[i]; p <- pb[i] * exp(-g0 * (z - hb[i]) / (Rair * T))
  } else {
    T <- Tb[i] + Lb[i] * (z - hb[i])
    p <- pb[i] * (T / Tb[i])^(-g0 / (Rair * Lb[i]))
  }
  c(T = T, p = p, rho = p / (Rair * T), a = sqrt(1.4 * Rair * T))
}
round(sapply(c(0, 11000, 30000), atmos), 4)
```

```
        [,1] [,2] [,3]
T 288.1500 216.7735 226.5091
p 101325.0000 22699.9522 1197.0303
rho 1.2250 0.3648 0.0184
a 340.2941 295.1537 301.7087
```

Python

```python
# U.S. Standard Atmosphere 1976, up to 86 km
import math
g0, Rair, r0 = 9.80665, 287.053, 6356766.0
hb = [0, 11000, 20000, 32000, 47000, 51000, 71000]
Tb = [288.15, 216.65, 216.65, 228.65, 270.65, 270.65, 214.65]
```

```
Lb = [-0.0065, 0, 0.001, 0.0028, 0, -0.0028, -0.002]
pb = [101325.0] # base pressures
for i in range(1, 7):
    dh = hb[i] - hb[i-1]
    if Lb[i-1] == 0:
        pb.append(pb[-1] * math.exp(-g0 * dh / (Rair * Tb[i-1])))
    else:
        pb.append(pb[-1] * (1 + Lb[i-1] * dh / Tb[i-1])
                  ** (-g0 / (Rair * Lb[i-1])))

def atmos(h): # h: geometric altitude [m]
    z = r0 * h / (r0 + h) # geopotential altitude
    i = max(k for k in range(7) if hb[k] <= z) # which layer?
    if Lb[i] == 0:
        T = Tb[i]; p = pb[i] * math.exp(-g0 * (z - hb[i]) / (Rair * T))
    else:
        T = Tb[i] + Lb[i] * (z - hb[i])
        p = pb[i] * (T / Tb[i]) ** (-g0 / (Rair * Lb[i]))
    return T, p, p / (Rair * T), math.sqrt(1.4 * Rair * T)

if __name__ == '__main__':
    for h in (0, 11000, 30000):
        print(h, [round(x, 4) for x in atmos(h)])
```

```
0 [288.15, 101325.0, 1.225, 340.2941]
11000 [216.7735, 22699.9522, 0.3648, 295.1537]
30000 [226.5091, 1197.0303, 0.0184, 301.7087]
```

Read the R output column by column: sea level gives the familiar 288.15 K, 101 325 Pa, 1.225 kg/m^3, 340.29 m/s. At 11 km the temperature is 216.77 K, not 216.65: that is the geopotential correction at work (we are at $z = 10.981$ km, still just inside layer 1). At 30 km only 1.2% of the sea-level pressure is left.

The second function does the aerodynamics of Sections 4.6 to 4.8. It takes the altitude, the air-relative velocity in body axes and the CG station, and returns forces and moments. This is the function the simulator of Chapter 6 calls thousands of times per second of flight.

```
import math
from c4_atmos import atmos

# LV-N1 aerodynamic tables (slender body, no fins)
MACH = [0.0, 0.5, 0.8, 0.9, 1.0, 1.1, 1.2, 1.5, 2.0, 3.0, 4.0, 6.0,
    10.0]
CA = [0.30, 0.28, 0.30, 0.38, 0.50, 0.55, 0.53, 0.45, 0.38, 0.30, 0.26,
    0.22, 0.20]
CNA = [2.0, 2.0, 2.05, 2.15, 2.35, 2.55, 2.6, 2.5, 2.4, 2.3, 2.2, 2.1,
    2.05]
SCP = [16.0, 16.0, 16.1, 16.3, 16.8, 17.2, 17.4, 17.6, 17.7, 17.8, 17.8,
     17.8, 17.8]
S_REF = math.pi * 1.5**2 # reference area [m^2]

def interp(x, xs, ys):
    """straight-line interpolation in a table (held flat at the ends)"""
    if x <= xs[0]: return ys[0]
    if x >= xs[-1]: return ys[-1]
    i = 0
    while xs[i + 1] < x: # find the interval
```

```
        i += 1
    w = (x - xs[i]) / (xs[i + 1] - xs[i]) # 0 at left end, 1 at right
    return ys[i] + w * (ys[i + 1] - ys[i])

def aero(h, v_air_body, s_cg):
    """forces [N] and pitch/yaw moments [N m] in body axes"""
    T, p, rho, a = atmos(h)
    u, v, w = v_air_body
    V = math.sqrt(u*u + v*v + w*w)
    mach, qbar = V / a, 0.5 * rho * V * V
    vt = math.hypot(v, w) # cross-flow speed
    alpha = math.atan2(vt, u) # total angle of attack
    A = qbar * S_REF * interp(mach, MACH, CA) # axial force
    N = qbar * S_REF * interp(mach, MACH, CNA) * alpha # normal force
    Fy, Fz = (-N * v / vt, -N * w / vt) if vt > 0 else (0.0, 0.0)
    arm = s_cg - interp(mach, MACH, SCP) # CP ahead of CG by 'arm'
    My, Mz = -arm * Fz, arm * Fy # r x F with r = (arm, 0, 0)
    return dict(mach=mach, qbar=qbar, alpha_deg=math.degrees(alpha),
                A=A, N=N, Fz=Fz, My=My, arm=arm)

if __name__ == "__main__":
    # LV-N1 near max-Q, airflow 2 degrees off the nose
    V, al = 554.6, math.radians(2.0)
    out = aero(10900.0, (V*math.cos(al), 0.0, V*math.sin(al)), 25.84)
    for k, x in out.items():
        print(f"{k:>9s} = {x:12.4g}")
```

```
    mach =        1.876
    qbar =     5.682e+04
alpha_deg =            2
       A =     1.596e+05
       N =     3.399e+04
      Fz =    -3.399e+04
      My =     2.776e+05
     arm =        8.165
```

Every number agrees with the hand calculations of this chapter. The positive pitch moment M_y is the destabilising moment: it pushes α further in the direction it already has.

4.10 Computational statistics: finding roots and fitting models

This section connects the chapter to the STAT 3585 course and to Robert and Casella, *Monte Carlo Statistical Methods*, Section 1.2 (likelihood methods, built on the statistical models of Section 1.1) and Appendix A (probability distributions). Two jobs come up constantly when you build a simulator:

1. **Root finding**: "at what altitude does the pressure fall to 1% of sea level?" means solving an equation $f(h) = 0$ that we cannot rearrange by hand.
2. **Fitting**: "what scale height H best describes these balloon measurements?" means choosing a parameter so that a model agrees best with noisy data.

Both are done by *iterating*: guess, check, improve, repeat.

4.10.1 Root finding: bisection and Newton's method

We want the altitude $h^\star$ where $p(h^\star) = 0.01\,p_0 = 1013.25\,\text{Pa}$. Define $f(h) = p(h) - 1013.25$. We need $f(h^\star) = 0$.

Bisection: the safe method. $f(0) > 0$ (too much pressure on the ground) and $f(50\,\text{km}) < 0$ (too little up high). Because f is continuous, the root lies in between. Test the midpoint. If $f(\text{mid}) > 0$ the root is above it, otherwise below. Throw away the wrong half, and repeat. Every step halves the interval, so after n steps it is $50\,000/2^n$ metres wide. To reach 1 m, we need $2^n > 50\,000$, i.e. $n = 16$ steps. Bisection is slow but it *cannot fail*, as long as the starting interval brackets the root.

Newton's method: the fast method. Near a guess h_k, replace the curve by its tangent line, $f(h) \approx f(h_k) + f'(h_k)(h - h_k)$, and jump to where the tangent crosses zero:

$$h_{k+1} = h_k - \frac{f(h_k)}{f'(h_k)}. \tag{4.18}$$

Newton needs the slope f'. Here physics gives it for free. Pressure falls exponentially, so it is much easier to work with $\ln p$, which is nearly a straight line in h. Take $f(h) = \ln p(h) - \ln p^\star$. By (4.3), $f'(h) = \frac{1}{p}\frac{\mathrm{d}p}{\mathrm{d}h} = -\frac{g}{RT} = -\frac{1}{H}$. Newton's step becomes

$$h_{k+1} = h_k + H(h_k)\left(\ln p(h_k) - \ln p^\star\right) :$$

"climb one local scale height for every factor of e of pressure still to lose." Near the answer, Newton *doubles the number of correct digits* at every step.

The built-in tools. R's `uniroot` and SciPy's `brentq` combine the two: bisection's guarantee with near-Newton speed (Brent's method). In practice you call these, but it is worth knowing what they do inside.

R

```r
source("c4_atmos.R") # gives atmos(h)
target <- 0.01 * 101325 # 1% of sea-level
    pressure
f <- function(h) atmos(h)["p"] - target

# 1) bisection by hand
lo <- 0; hi <- 50000; n <- 0
while (hi - lo > 1) { # stop at 1 m
  mid <- (lo + hi) / 2; n <- n + 1
  if (f(mid) > 0) lo <- mid else hi <- mid
}
cat("bisection:", round((lo + hi) / 2, 1),
     "m after", n, "steps\n")

# 2) Newton on log(p); hydrostatics gives
    the slope -g/(R T)
h <- 20000
for (k in 1:4) {
  s <- atmos(h); g <- 9.80665 * (6356766 /
      (6356766 + h))^2
  H <- 287.053 * s[["T"]] / g # local scale
       height
  h <- h + H * (log(s[["p"]]) - log(target)
      )
  cat("Newton step", k, ":", round(h, 3), "
      m\n")
}

# 3) R's built-in root finder
uniroot(f, c(0, 50000), tol = 1e-6)$root
```

```
bisection: 31118.4 m after 16 steps
Newton step 1 : 30828.91 m
Newton step 2 : 31118.31 m
Newton step 3 : 31118.51 m
Newton step 4 : 31118.51 m
[1] 31118.51
```

Python

```python
from c4_atmos import atmos # gives atmos(h)

from scipy.optimize import brentq
import math
target = 0.01 * 101325 # 1% of sea-level
    pressure
f = lambda h: atmos(h)[1] - target

# 1) bisection by hand
lo, hi, n = 0.0, 50000.0, 0
while hi - lo > 1: # stop at 1 m
   mid = (lo + hi) / 2; n += 1
   if f(mid) > 0: lo = mid
   else: hi = mid
print("bisection:", round((lo + hi) / 2, 1)
    , "m after", n, "steps")

# 2) Newton on log(p); hydrostatics gives
    the slope -g/(R T)
h = 20000.0
for k in range(1, 5):
   T, p, rho, a = atmos(h)
   g = 9.80665 * (6356766 / (6356766 + h))
        ** 2
   H = 287.053 * T / g # local scale
       height
   h = h + H * (math.log(p) - math.log(
       target))
   print("Newton step", k, ":", round(h, 3)
       , "m")

# 3) SciPy's built-in root finder
print(brentq(f, 0, 50000, xtol=1e-6))
```

```
bisection: 31118.4 m after 16 steps
Newton step 1 : 30828.911 m
Newton step 2 : 31118.31 m
Newton step 3 : 31118.505 m
Newton step 4 : 31118.505 m
31118.50517860638
```

All three agree: the pressure is 1% of sea level at 31.12 km. Look at Newton's progress: errors of 290 m, 0.2 m, then less than a millimetre. Bisection needed 16 steps for 1 m; Newton needed 3 for a millimetre — because we gave it the slope, and the slope came from the hydrostatic law of Section 4.2.

4.10.2 Fitting a model by maximum likelihood

The data. Suppose a weather balloon measures the air density every 2 km from the ground to 30 km, with about 5% random error. We want the exponential model (4.9) that best describes these 16 measurements. (We create the measurements from US76 plus random noise, using the random-number tools of Section 2.11, so we know the truth.)

The statistical model. Taking logarithms of (4.9) turns the curve into a straight line, $\ln\rho = \ln\rho_0 - h/H$. Write $b_0 = \ln\rho_0$ and let each measurement have a random error:

$$y_i = \ln\rho_i = b_0 - \frac{h_i}{H} + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma^2) \text{ independently.} \tag{4.19}$$

This is a *statistical model* in the sense of Robert and Casella, Section 1.1: a family of probability distributions for the data, indexed by unknown parameters $\boldsymbol{\theta} = (b_0, H, \sigma)$.

The likelihood. The normal density of one error is $\frac{1}{\sigma\sqrt{2\pi}}\exp\!\left(-\varepsilon^2/(2\sigma^2)\right)$. Because the errors are independent, the probability density of all 16 measurements is the product of 16 such terms. Read as a function of the parameters, this product is the *likelihood*:

$$L(\boldsymbol{\theta}) = \prod_{i=1}^{n} \frac{1}{\sigma\sqrt{2\pi}} \exp\!\left(-\frac{\left(y_i - b_0 + h_i/H\right)^2}{2\sigma^2}\right).$$

Products of small numbers are awkward, so take the logarithm; the logarithm of a product is a sum:

$$\ell(\boldsymbol{\theta}) = \ln L(\boldsymbol{\theta}) = -n\ln\sigma - \frac{n}{2}\ln(2\pi) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}\left(y_i - b_0 + h_i/H\right)^2. \tag{4.20}$$

The **maximum likelihood estimate** (MLE) is the $\boldsymbol{\theta}$ that makes ℓ as large as possible: the parameter values under which the data we actually saw were most probable.

> **Key idea**: Maximum likelihood = least squares, for normal errors. Look at (4.20). For any fixed σ, the only part that depends on b_0 and H is $-\frac{1}{2\sigma^2}\times$(sum of squared misfits). Making ℓ large is the same as making the *sum of squares* small. So with normal errors, the MLE of the line is exactly the least-squares line — the one R's `lm` and NumPy's `polyfit` compute. Setting $\partial\ell/\partial\sigma = 0$ then gives $\hat{\sigma}^2 = \frac{1}{n}\sum(\text{misfit})^2$.

Two ways to compute it. (1) Hand the function $-\ell(\boldsymbol{\theta})$ to a general optimiser (R's `optim`, SciPy's `minimize`), which walks downhill until the slope is zero. We optimise over $\ln\sigma$ rather than σ so that the optimiser can never try a negative σ. (2) Fit the straight line with least squares. Both must agree.

How sure are we? The delta method. Least squares gives the slope $b_1 = -1/H$ and its standard error $\mathrm{se}(b_1)$. We want the error in $H = -1/b_1$. A small change Δb_1 changes H by about $\frac{\mathrm{d}H}{\mathrm{d}b_1}\Delta b_1 = \frac{1}{b_1^2}\Delta b_1$, so

$$\mathrm{se}(H) \approx \frac{\mathrm{se}(b_1)}{b_1^2}.$$

R

```r
source("c4_atmos.R")
set.seed(4)
h <- seq(0, 30000, by = 2000) # balloon
    altitudes [m]
rho <- sapply(h, function(x) atmos(x)[["
    rho"]]) *
     exp(rnorm(length(h), 0, 0.05)) # 5%
         measurement noise
y <- log(rho)

# minus the log-likelihood of y = b0 - h/H
     + eps, eps ~ N(0, s^2)
nll <- function(th) {
  b0 <- th[1]; H <- th[2] * 1000; s <- exp(
      th[3])
  -sum(dnorm(y, b0 - h / H, s, log = TRUE))
}
fit <- optim(c(0, 8, log(0.1)), nll,
    method = "BFGS")
round(c(rho0 = exp(fit$par[1]), H_km = fit
    $par[2],
  sigma = exp(fit$par[3])), 4)

# cross-check: ordinary least squares on
    the same line
ls <- lm(y ~ h); b1 <- unname(coef(ls)[2])
se1 <- summary(ls)$coef[2, 2]
round(c(H_km = -1 / b1 / 1000,
  se_km = se1 / b1^2 / 1000), 4) # delta
     method
```

```
  rho0 H_km sigma
1.5575 6.9567 0.1151
  H_km se_km
6.9566 0.1615
```

Python

```python
import numpy as np
from scipy.optimize import minimize
from scipy.stats import norm
from c4_atmos import atmos
rng = np.random.default_rng(4)
h = np.arange(0, 30001, 2000.0) # balloon
    altitudes [m]
rho = np.array([atmos(x)[2] for x in h]) \
    * np.exp(rng.normal(0, 0.05, h.size))
        # 5% measurement noise
y = np.log(rho)

# minus the log-likelihood of y = b0 - h/H
     + eps, eps ~ N(0, s^2)
def nll(th):
   b0, H, s = th[0], th[1] * 1000, np.exp(
       th[2])
   return -norm.logpdf(y, b0 - h / H, s).
       sum()

fit = minimize(nll, [0, 8, np.log(0.1)],
    method="BFGS")
print("rho0", round(np.exp(fit.x[0]), 4),
    "H_km",
    round(fit.x[1], 4), "sigma", round(np.
        exp(fit.x[2]), 4))

# cross-check: ordinary least squares on
    the same line
(b1, b0), C = np.polyfit(h, y, 1, cov="
    unscaled")
res = y - (b0 + b1 * h); s2 = res @ res /
    (h.size - 2)
se1 = np.sqrt(s2 * C[0, 0])
print("H_km", round(-1 / b1 / 1000, 4),
    "se_km", round(se1 / b1**2 / 1000, 4))
        # delta method
```

```
rho0 1.497 H_km 7.0309 sigma 0.1018
H_km 7.0309 se_km 0.1458
```

Reading the results. R and Python use different random numbers (Section 2.11.1), so their balloons "measured" slightly different values, but both find $H \approx 7.0$ km with a standard error of about 0.15 km; the general optimiser and least squares agree to the fourth digit, as the key idea predicts. Two numbers deserve a second look:

- $\hat{\rho}_0 \approx 1.5$, not 1.225. The exponential line is a compromise over 0 km to 30 km: to follow the fast fall in the cold stratosphere, it has to start too high at the ground.
- $\hat{\sigma} \approx 0.10$–$0.12$, about *twice* the 5% noise we put in. The leftover misfit is not all noise; part of it is the model being wrong.

> **Reality check.** This is the most useful lesson of fitting: **the likelihood tells you not only the best parameters, but also when the model itself is too simple.** If the residual scatter is larger than the known measurement error, the model is missing something —

here, the seven different temperature layers. That is why the simulator uses US76 and keeps the exponential model only for quick estimates.

Summary

- Air obeys the ideal gas law $p = \rho R T$; at sea level $\rho_0 = 1.225\,\mathrm{kg/m^3}$.
- Hydrostatic balance, $\mathrm{d}p/\,\mathrm{d}h = -\rho g$: pressure is the weight of the air above. With the gas law it gives exponential decay in isothermal layers (4.5) and a power law in gradient layers (4.6).
- US76 stacks seven such layers; heights are geopotential; the speed of sound is $a = \sqrt{\gamma R T}$. One exponential with $H \approx 7\,\mathrm{km}$ is a good rough model.
- Aerodynamics uses the air-relative velocity $\boldsymbol{v} - \boldsymbol{\omega}_E \times \boldsymbol{r} - \boldsymbol{w}$.
- Mach number V/a sets the type of flow; dynamic pressure $\frac{1}{2}\rho V^2$ sets the size of every force. LV-N1 reaches max-Q, 56.8 kPa, at 52 s, Mach 1.88.
- Axial force $\bar{q} S C_A$ and normal force $\bar{q} S C_{N_\alpha} \alpha$ act at the centre of pressure. CP ahead of CG means static instability; LV-N1 is unstable by 2.7 calibres at max-Q and must be flown by its engine gimbal.
- Coefficient tables are read by linear interpolation.
- Roots are found by bisection (safe) or Newton (fast); models are fitted by maximum likelihood, which equals least squares for normal errors. A residual scatter larger than the known noise means the model is too simple.

Exercises

4.1 Gas law. On a hot day at sea level, $T = 35\,^\circ\mathrm{C}$ and $p = 101\,325\,\mathrm{Pa}$. Find ρ, and the percentage by which the drag on a vehicle at a given speed is lower than on a standard day.

4.2 Scale height. Show that the scale height $H = RT/g$ has units of metres. Find the altitude at which the pressure in an isothermal 250 K atmosphere is half its sea-level value. (Answer: $H \ln 2$.)

4.3 Layer 3. Using Table 4.1, compute T and p at a geopotential height of 25 km by hand, and check with the code.

4.4 Geopotential. Find the geometric altitude h at which $z = 86\,\mathrm{km}$. (Invert (4.7): $h = r_0 z/(r_0 - z)$.)

4.5 Wind. A rocket at 12 km moves at 300 m/s straight up (relative to the ground) through a 60 m/s horizontal wind. Find its air-relative speed and its angle of attack if its nose points straight up.

4.6 Max-Q in the exponential atmosphere. Suppose a vehicle climbs vertically with constant acceleration k, so $V = kt$ and $h = \tfrac{1}{2}kt^2$. Show that $\bar{q} = \tfrac{1}{2}\rho_0 \, 2kh\, e^{-h/H}$, and by differentiating, that max-Q occurs at $h = H$, whatever k is.

4.7 Stability. Where must the centre of pressure be for LV-N1 to be neutrally stable at max-Q? How much fin area would roughly be needed to move it there? (Discuss qualitatively.)

4.8 R/Python: root finding. Find the altitude where the density is 1% of its sea-level value with `uniroot` / `brentq`, and with a Newton iteration whose slope you derive yourself.

4.9 R/Python: maximum likelihood. Refit the scale height using only the balloon data below 11 km. Does $\hat{\sigma}$ come closer to 0.05? Explain why.

4.10 R/Python: Monte Carlo of the estimator. Repeat the balloon experiment 1000 times with different seeds, store $\hat{H}$ each time, and compare the standard deviation of the 1000 values with the delta-method standard error.

Reading for this chapter

- P. H. Zipfel, *Modeling and Simulation of Aerospace Vehicle Dynamics*, 2nd ed.: Sections 8.2.1–8.2.3 (atmosphere, gravitational attraction, drag polar), Section 7.3 (aerodynamic forces and moments), Section 10.2.1 (aerodynamics in a 6-DOF simulation, including axisymmetric rocket shapes) and Sections 10.3.2–10.3.3 (winds and turbulence).
- S. Boyd and L. Vandenberghe, *Convex Optimization*: Section 6.1 (norm approximation) and Section 6.5 (function fitting and interpolation) — the least-squares fitting of Section 4.10, in general form.
- K. J. Åström and R. M. Murray, *Feedback Systems*: Sections 4.3–4.4 (stability), as background for static and dynamic stability.
- C. P. Robert and G. Casella, *Monte Carlo Statistical Methods*, 2nd ed.: Section 1.2 (likelihood methods), Section 1.3 (Bayesian methods, review) and Appendix A (probability distributions) — **STAT 3585 background**.

Chapter 5

Propulsion and Gimbal Actuator Modeling

> *The engine is the only thing on the rocket that pushes it upward, and the only thing that can steer it while it is still in the thick air. This chapter is about modelling both jobs.*

In Chapter 3 we met the thrust equation, and in Chapter 4 we found that the air tries to tip LV-N1 over. This chapter builds the two models that answer back:

1. a **propulsion model**: how much thrust the engine gives at every altitude, how fast it uses propellant, and how the vehicle's mass changes (Sections 5.1–5.2);
2. a **thrust-vector-control model**: how tilting the engine on its gimbal produces a turning moment, and how the hydraulic or electric *actuator* that tilts it actually behaves — with a delay, a speed limit and an angle limit (Sections 5.3–5.5).

We turn both into code, and then use them to meet a new kind of optimisation from STAT 3585: *Monte Carlo optimisation*, where randomness itself does the searching (Section 5.6).

5.1 How an engine makes thrust

5.1.1 Throwing mass backwards

A rocket engine is a machine for throwing mass backwards very fast. Every second it takes $\dot{m}$ kilograms of propellant, burns it, and blows it out of the nozzle at an exhaust speed u_e relative to the vehicle. By Newton's third law, pushing that mass backwards pushes the vehicle forwards. In Chapter 3 we derived the full result, (3.19). Along the vehicle axis its size is

$$T = \dot{m}\, u_e + (p_e - p_a)\, A_e, \tag{5.1}$$

where p_e is the gas pressure at the nozzle exit, A_e the exit area and p_a the outside air pressure from Chapter 4. The two terms have simple meanings:

- $\dot{m}\,u_e$, the **momentum thrust**: (mass per second) $\times$ (speed) is momentum per second, which is a force;
- $(p_e - p_a)A_e$, the **pressure thrust**: the gas at the exit pushes out on the exit area with p_e, the air pushes back with p_a, and the difference is a force.

5.1.2 The trick that makes it simple: vacuum thrust

The engine does not know where it is. The flow inside the nozzle — $\dot{m}$, u_e and p_e — is the same on the pad and in space. Only p_a changes. So collect everything that belongs to the engine into one number, the *vacuum thrust* (the thrust when $p_a = 0$):

$$T_{\text{vac}} = \dot{m}\,u_e + p_e A_e.$$

Then (5.1) becomes

> **Key idea**: Thrust at any altitude.
>
> $$T(h) = T_{\text{vac}} - p_a(h)\,A_e. \tag{5.2}$$
>
> Thrust is the vacuum thrust minus the air pressure pushing on the nozzle exit. Two engine numbers (T_{vac}, A_e) and the atmosphere of Chapter 4 give the thrust everywhere.

5.1.3 Specific impulse

How good is an engine? Not "how strong" — a bigger engine is stronger — but "how much push per kilogram of propellant". That is the *specific impulse*:

$$I_{sp} = \frac{T}{\dot{m}\,g_0}, \qquad g_0 = 9.806\,65\,\text{m/s}^2. \tag{5.3}$$

Its unit is seconds. One way to picture it: an engine with $I_{sp} = 310\,\text{s}$ could hold up its own propellant's weight (on Earth) for 310 seconds while burning it. The product

$$c = I_{sp}\,g_0$$

is the *effective exhaust velocity*. With it, the thrust is simply $T = \dot{m}\,c$, and the rocket equation of Chapter 1 reads $\Delta v = c\ln(m_0/m_f)$.

5.1.4 Finding the engine numbers for LV-N1

A data sheet usually gives T_{vac}, $I_{sp,\text{vac}}$ and $I_{sp,\text{sl}}$. Everything else follows in three steps.

> **Worked example**: From the data sheet to the model. LV-N1: $T_{\text{vac}} = 2.2\,\text{MN}$, $I_{sp,\text{vac}} = 310\,\text{s}$, $I_{sp,\text{sl}} = 280\,\text{s}$.

Step 1: mass flow. From (5.3) in vacuum,

$$\dot{m} = \frac{T_{\text{vac}}}{I_{sp,\text{vac}}\, g_0} = \frac{2\,200\,000}{310 \times 9.80665} = \frac{2\,200\,000}{3040.06} = 723.67\,\text{kg/s}.$$

Step 2: sea-level thrust. The same flow at sea level gives $T_{\text{sl}} = \dot{m}\, I_{sp,\text{sl}}\, g_0 = 723.67 \times 280 \times 9.80665 = 1.9871\,\text{MN}$.

Step 3: exit area. Put $h = 0$ into (5.2) and solve for A_e:

$$A_e = \frac{T_{\text{vac}} - T_{\text{sl}}}{p_0} = \frac{2\,200\,000 - 1\,987\,097}{101\,325} = 2.101\,\text{m}^2,$$

a nozzle about 1.64 m across. And the burn time is $t_b = m_{\text{prop}}/\dot{m} = 102\,000/723.67 = 140.95\,\text{s}$.

Reality check. Real engines are rated at a fixed *nozzle expansion*: the nozzle is shaped so that p_e matches the air pressure at one chosen altitude. A first-stage nozzle is kept short so that p_e is not far below sea-level pressure (otherwise the flow separates from the nozzle wall); an upper-stage nozzle is very long, because in vacuum a bigger A_e always helps. Our model, (5.2), holds for any fixed nozzle; only T_{vac} and A_e change from engine to engine.

5.2 Mass, centre of mass and inertia during the burn

The engine sets how fast the vehicle gets lighter:

$$m(t) = m_0 - \dot{m}\, t \qquad (0 \le t \le t_b), \qquad m(t) = m_{\text{dry}} \quad (t > t_b). \tag{5.4}$$

For LV-N1, m falls from 120 t to 18 t: by burnout the vehicle weighs less than a sixth of its lift-off mass, while the thrust has *grown*. The *thrust-to-weight ratio* $T/(mg_0)$ tells the story:

$$\text{lift-off: } \frac{1.9871 \times 10^6}{120\,000 \times 9.80665} = 1.69, \qquad \text{burnout: } \frac{2.2 \times 10^6}{18\,000 \times 9.80665} = 12.5.$$

At lift-off the vehicle accelerates at only $0.69\,g$ net; at burnout it would pull $12.5\,g$. Real vehicles throttle down or shut off engines near the end of a burn to protect the payload and the structure. LV-N1, for simplicity, does not.

The centre of mass and the inertia come from the tank model of Chapters 2 and 3: the propellant is a cylinder of shrinking length that drains from the top, so the CG moves aft and then forward again, and the transverse inertia J_t falls by a factor of 3.6:

	mass [t]	CG station [m]	J_t [kg m^2]	gimbal arm $\ell_g = 39 - s_{cg}$ [m]
lift-off ($t = 0$)	120.0	21.75	8.59×10^6	17.25
max-Q ($t = 52\,\text{s}$)	82.4	25.84	3.89×10^6	13.16
burnout ($t = t_b$)	18.0	26.00	2.41×10^6	13.00

We will need ℓ_g in a moment: it is the lever arm through which the engine turns the vehicle.

5.3 Thrust vector control: steering with the engine

5.3.1 Tilting the engine

The engine is mounted on a *gimbal* — a joint like a ball-and-socket — at station 39 m. Two actuators tilt it: one by the *pitch angle* δ_p (about the body y axis), the other by the *yaw angle* δ_y (about the body z axis). The thrust, which normally points straight along x_B, now points slightly sideways. Tilting first in yaw and then in pitch gives the thrust vector in body axes:

$$\boldsymbol{F}_T^B = T\,\big(\cos\delta_p\cos\delta_y,\ \ -\cos\delta_p\sin\delta_y,\ \ \sin\delta_p\big). \tag{5.5}$$

Check the length: $\cos^2\delta_p\cos^2\delta_y+\cos^2\delta_p\sin^2\delta_y+\sin^2\delta_p=\cos^2\delta_p+\sin^2\delta_p=1$. Tilting never changes the size of the thrust, only its direction.

5.3.2 The turning moment

The force acts at the gimbal, which is ℓ_g metres *behind* the CG: $\boldsymbol{r}_g=(-\ell_g,0,0)$ in body axes. The moment about the CG is a cross product (Chapter 2):

$$\boldsymbol{M}_T=\boldsymbol{r}_g\times\boldsymbol{F}_T^B=\begin{vmatrix}\boldsymbol{i}&\boldsymbol{j}&\boldsymbol{k}\\ -\ell_g&0&0\\ F_x&F_y&F_z\end{vmatrix}=\big(0\cdot F_z-0\cdot F_y,\ \ 0\cdot F_x-(-\ell_g)F_z,\ \ (-\ell_g)F_y-0\cdot F_x\big),$$

so

Key idea: Gimbal moment.

$$\boldsymbol{M}_T=\big(0,\ \ \ell_g T\sin\delta_p,\ \ \ell_g T\cos\delta_p\sin\delta_y\big)\ \approx\ \big(0,\ \ T\ell_g\,\delta_p,\ \ T\ell_g\,\delta_y\big)\ \text{for small angles.} \tag{5.6}$$

The roll component is zero: a single gimballed engine on the centre line cannot roll the vehicle. (Real vehicles roll with several gimballed engines, or small roll thrusters.)

Worked example: One degree of gimbal at max-Q. $T=2.1515\,\text{MN}$, $\ell_g=13.16\,\text{m}$, $\delta_p=1^\circ=0.017\,453\,\text{rad}$:

$$M_y=2.1515\times10^6\times13.16\times0.017453=494\,\text{kN m}.$$

Divide by $J_t=3.889\times10^6\,\text{kg m}^2$: the pitch acceleration is $0.127\,\text{rad/s}^2$, i.e. $7.3^\circ/\text{s}^2$ per degree of gimbal. This is the control effectiveness $\mu_c=T\ell_g/J_t$ of (3.32).

How much gimbal holds the aerodynamic moment? In Chapter 4 we found $M_A=277.6\,\text{kN m}$ at max-Q with $\alpha=2^\circ$. Setting $T\ell_g\sin\delta_p=M_A$:

$$\delta_p=\arcsin\frac{277\,600}{2.1515\times10^6\times13.16}=\arcsin 0.00980=0.56^\circ.$$

Just over half a degree of the available six.

Control authority changes during flight. Using the table of Section 5.2, $\mu_c = T\ell_g/J_t$ is about $4.0/\text{s}^2$ at lift-off, 7.3 at max-Q and 11.9 at burnout. The vehicle becomes three times as responsive as it burns, mainly because it gets lighter (smaller J_t). A controller tuned at lift-off would be far too aggressive at burnout; this is why gains must be scheduled (Chapter 11).

The price of steering. The forward part of the thrust is $T\cos\delta_p\cos\delta_y$. At the full 6°, $1-\cos 6^\circ = 0.0055$: steering costs half a percent of thrust. Small, but it is a loss that grows with every unnecessary gimbal movement, which is one reason to keep the vehicle aligned with the airflow.

5.3.3 From a wanted moment to a gimbal command

The control law (Chapter 11) decides what moment it wants, $M_{y,c}$. Inverting (5.6) gives the gimbal command, which is then clamped to what the hardware allows:

$$\delta_{p,c} = \text{clamp}\left(\arcsin\frac{M_{y,c}}{T\,\ell_g},\ -\delta_{\max},\ \delta_{\max}\right), \qquad \delta_{\max} = 6^\circ. \tag{5.7}$$

($\text{clamp}(x,a,b)$ means: x, but no smaller than a and no larger than b.) The same is done for yaw.

5.4 The gimbal actuator

5.4.1 Why an actuator is not instant

The engine weighs tonnes. To tilt it, a hydraulic ram (or electric motor) must push on it, and pushing takes time. If we pretended that $\delta = \delta_c$ instantly, our simulated vehicle would be easier to control than the real one, and a controller that works in simulation might fail in flight. Three effects matter, and we model them one at a time.

5.4.2 Effect 1: the lag

The actuator moves towards the commanded angle at a speed proportional to how far it still has to go. This is (1.16) of Chapter 1:

$$\tau\,\dot\delta = \delta_c - \delta \qquad\Longleftrightarrow\qquad \dot\delta = \frac{\delta_c - \delta}{\tau}. \tag{5.8}$$

Far from the target it moves fast; close to the target it slows down.

Solving it step by step. Let the command jump from 0 to a constant δ_c at $t = 0$, with $\delta(0) = 0$. Write the error $e = \delta_c - \delta$. Because δ_c is constant, $\dot e = -\dot\delta = -e/\tau$. "The rate of change is a constant times the value" is the equation of exponential decay, so $e(t) = e(0)\,e^{-t/\tau} = \delta_c\,e^{-t/\tau}$. Therefore

$$\delta(t) = \delta_c\big(1 - e^{-t/\tau}\big). \tag{5.9}$$

Worked example: What the time constant means. LV-N1's actuator has $\tau = 0.05\,\text{s}$. After one time constant the error has shrunk by $e^{-1} = 0.368$, so the actuator has done 63.2% of the move. After $3\tau = 0.15\,\text{s}$ it has done $1 - e^{-3} = 95.0\%$; after $5\tau = 0.25\,\text{s}$, 99.3%. A rule of thumb: **"three time constants to get there"**. In the language of Chapter 1 the actuator is the transfer function $1/(\tau s + 1)$, which follows commands well up to about $1/\tau = 20\,\text{rad/s}$ ($\approx 3\,\text{Hz}$) and smooths out anything faster.

5.4.3 Effect 2: the rate limit

The ram can only pump fluid so fast, so the angle can only change so fast: for LV-N1, at most $\dot{\delta}_{\max} = 20\,^\circ/\text{s}$. Combine this with the lag by clamping the speed that (5.8) asks for:

$$\dot{\delta} = \text{clamp}\left(\frac{\delta_c - \delta}{\tau},\ -\dot{\delta}_{\max},\ \dot{\delta}_{\max}\right). \tag{5.10}$$

Worked example: A 4° step with the rate limit. At $t = 0$ the lag asks for $\dot{\delta} = 4/0.05 = 80\,^\circ/\text{s}$ — four times more than allowed. So the gimbal moves at the limit, $20\,^\circ/\text{s}$, in a straight line. The lag asks for less than the limit once $(4 - \delta)/0.05 < 20$, i.e. once $\delta > 3^\circ$. That happens at $t = 3/20 = 0.15\,\text{s}$. After that the lag takes over and closes the last degree exponentially:

$$\delta(t) = 4 - 1 \cdot e^{-(t - 0.15)/0.05} \quad (t \geq 0.15\,\text{s}).$$

So $\delta(0.20) = 4 - e^{-1} = 3.63^\circ$ and $\delta(0.25) = 4 - e^{-2} = 3.86^\circ$. Without the rate limit it would already be at 3.80° by 0.15 s. Figure 5.1(b) shows both.

Reality check. Rate limits are dangerous in a feedback loop. When the actuator is rate-limited it lags the command more and more, which looks to the controller like extra delay, which can turn a stable loop into an oscillating one ("rate-limit-induced oscillation"). Several aircraft accidents have been traced to it. Always simulate with the limits switched on.

5.4.4 Effect 3: the position limit

The nozzle can only tilt so far before it hits the structure: $|\delta| \leq \delta_{\max} = 6^\circ$. We apply this to the command (equation (5.7)). A real actuator also has a hard stop, but if the command never asks for more than 6° the angle cannot exceed it either.

5.4.5 A second-order actuator (for later)

A more detailed model treats the actuator like a spring–mass–damper: $\ddot{\delta} + 2\zeta\omega_n\dot{\delta} + \omega_n^2\delta = \omega_n^2\delta_c$, with a natural frequency ω_n (typically 30 rad/s to 60 rad/s) and damping ζ (about 0.7). It captures overshoot and the effect of the nozzle's own mass. It needs two states (δ and $\dot{\delta}$) instead of one. The first-order model is enough for the rest of Part I.

5.5 Putting the actuator on a computer

A computer moves in steps of h seconds. The simplest update is forward Euler (Chapter 1):

$$\delta_{k+1} = \delta_k + h\,\frac{\delta_c - \delta_k}{\tau} = \Big(1 - \frac{h}{\tau}\Big)\delta_k + \frac{h}{\tau}\,\delta_c. \tag{5.11}$$

Look at the number multiplying δ_k: $1 - h/\tau$. Each step the old error is multiplied by it.

- If $h < \tau$, it is between 0 and 1: the error shrinks smoothly. Good.
- If $\tau < h < 2\tau$, it is between -1 and 0: the error flips sign every step but still shrinks. The simulated actuator overshoots and rings, which the real one never does.
- If $h > 2\tau$, it is below -1: the error *grows* every step. The simulation explodes.

With $\tau = 0.05\,\text{s}$, any step above 0.1 s is unstable. This is a property of the *numerical method*, not of the actuator. Figure 5.1(c) shows it.

The exact update. For a constant command over a step, (5.9) can be applied exactly: whatever the error is now, after h seconds it is multiplied by $e^{-h/\tau}$. So

$$\delta_{k+1} = e^{-h/\tau}\,\delta_k + \big(1 - e^{-h/\tau}\big)\,\delta_c. \tag{5.12}$$

Since $0 < e^{-h/\tau} < 1$ for every h, this can never overshoot or explode. In the full simulator we will instead carry δ as part of the state and integrate it with RK4 at a step $h = 0.01\,\text{s} = \tau/5$, which is safe and accurate.

> **Worked example**: Euler versus exact with $h = 0.12\,\text{s}$. $h/\tau = 2.4$, so Euler multiplies the error by $1 - 2.4 = -1.4$ each step. Starting from $\delta = 0$ with $\delta_c = 4°$: $\delta_1 = 0 + 2.4 \times 4 = 9.6°$ (error -5.6), then the error becomes $+7.84$, so $\delta_2 = -3.84°$, then $14.98°$, $-11.37°$, … The exact update multiplies by $e^{-2.4} = 0.0907$: $3.637°$, $3.967°$, $3.997°$, $4.000°$. The code below reproduces both columns.

R

```r
# gimbal actuator: first-order lag + rate limit + position limit
tau <- 0.05; rmax <- 20; dmax <- 6 # s, deg/s, deg
clamp <- function(x, lo, hi) min(max(x, lo), hi)
step_act <- function(d, dc, h) {
  dc <- clamp(dc, -dmax, dmax) # position limit
  rate <- clamp((dc - d) / tau, -rmax, rmax) # rate limit
  d + h * rate
}
h <- 0.001; d <- 0; out <- c()
for (k in 1:400) { # 0.4 s, 4 deg command
  d <- step_act(d, 4, h)
  if (k %in% c(50, 100, 150, 200, 250, 400)) out <- c(out, d)
}
round(out, 3) # at 0.05 ... 0.40 s

# Euler versus exact stepping of the pure lag, h = 0.12 s
a <- exp(-0.12 / tau); de <- 0; dx <- 0
for (k in 1:4) {
  de <- de + 0.12 / tau * (4 - de) # forward Euler
  dx <- a * dx + (1 - a) * 4 # exact update
  cat(k, round(de, 3), round(dx, 3), "\n")
}
```

```
[1] 1.000 2.000 3.000 3.636 3.867 3.994
1 9.6 3.637
2 -3.84 3.967
3 14.976 3.997
4 -11.366 4
```

Python

```python
# gimbal actuator: first-order lag + rate limit + position limit
import math
tau, rmax, dmax = 0.05, 20.0, 6.0 # s, deg/s, deg
clamp = lambda x, lo, hi: min(max(x, lo), hi)
def step_act(d, dc, h):
    dc = clamp(dc, -dmax, dmax) # position limit
    rate = clamp((dc - d) / tau, -rmax, rmax) # rate limit
    return d + h * rate

h, d, out = 0.001, 0.0, []
for k in range(1, 401): # 0.4 s, 4 deg command
    d = step_act(d, 4.0, h)
    if k in (50, 100, 150, 200, 250, 400):
        out.append(round(d, 3))
print(out) # at 0.05 ... 0.40 s

# Euler versus exact stepping of the pure lag, h = 0.12 s
a = math.exp(-0.12 / tau); de = dx = 0.0
for k in range(1, 5):
    de = de + 0.12 / tau * (4 - de) # forward Euler
    dx = a * dx + (1 - a) * 4 # exact update
    print(k, round(de, 3), round(dx, 3))
```

```
[1.0, 2.0, 3.0, 3.636, 3.867, 3.994]
1 9.6 3.637
2 -3.84 3.967
3 14.976 3.997
4 -11.366 4.0
```

The first line of output is the rate-limited step at 0.05, 0.10, 0.15, 0.20, 0.25 and 0.40 s: 1°, 2°, 3° in a straight line, then 3.636° and 3.867° (the exponential tail; the small difference from 3.632 and 3.865 is the Euler step of 1 ms), and finally 3.994°.

5.5.1 The engine in code

The propulsion model is a few lines on top of the atmosphere of Chapter 4:

```python
import math
from c4_atmos import atmos # Chapter 4
g0 = 9.80665
T_vac, Isp_vac, Isp_sl = 2.2e6, 310.0, 280.0
m0, m_prop = 120000.0, 102000.0

mdot = T_vac / (Isp_vac * g0) # mass flow [kg/s]
A_e = (T_vac - mdot * Isp_sl * g0) / 101325.0 # exit area [m^2]
t_b = m_prop / mdot # burn time [s]
print(f"mdot = {mdot:.2f} kg/s, A_e = {A_e:.4f} m^2, t_b = {t_b:.2f} s")
```

```
def thrust(h):
    p_a = atmos(h)[1] if h < 86000 else 0.0
    return T_vac - p_a * A_e # T = T_vac - p_a A_e

print(" h[km] T[MN] Isp[s]")
for h in (0, 5, 10, 20, 40, 80):
    T = thrust(1000.0 * h)
    print(f"{h:5d} {T/1e6:8.4f} {T/(mdot*g0):8.1f}")

# thrust-to-weight on the pad and at burnout
print("T/W lift-off:", round(thrust(0) / (m0 * g0), 3),
      " burnout:", round(T_vac / ((m0 - m_prop) * g0), 2))
```

```
mdot = 723.67 kg/s, A_e = 2.1012 m^2, t_b = 140.95 s
 h[km] T[MN] Isp[s]
    0 1.9871 280.0
    5 2.0864 294.0
   10 2.1443 302.2
   20 2.1884 308.4
   40 2.1994 309.9
   80 2.2000 310.0
T/W lift-off: 1.689 burnout: 12.46
```

Most of the gain happens low down: by 10 km the thrust has already gained 7.9%; above 40 km it is indistinguishable from the vacuum value (Fig. 5.1(a)).

5.6 Computational statistics: Monte Carlo optimisation

This section follows Robert and Casella, Sections 5.1–5.2 (STAT 3585). In Chapter 4 we minimised a function with a deterministic method (`optim`, Newton). Here the *search itself* is random.

5.6.1 The problem: identifying the actuator from a test

The actuator's data sheet says $\tau = 50$ ms, but every real unit is a little different, and there is often a small pure *delay* d before it starts to move (the time for electronics and valves to react). To find the real values, engineers bolt the engine to a test stand, command a 4° step, and record the angle every 2 ms for 0.3 s: 151 noisy readings y_i. The model is

$$\delta(t;\tau,d) = 4\big(1 - e^{-(t-d)/\tau}\big) \text{ for } t > d, \qquad 0 \text{ otherwise},$$

and we want the (τ, d) that minimises the misfit

$$S(\tau,d) = \sum_{i=1}^{151} \big(y_i - \delta(t_i;\tau,d)\big)^2. \tag{5.13}$$

As in Chapter 4, with normal noise this least-squares fit is also the maximum-likelihood fit. Notice that S has a kink wherever a data time t_i equals d (the model switches formula there), so its derivative is awkward. Methods that do not need an exact derivative are attractive.

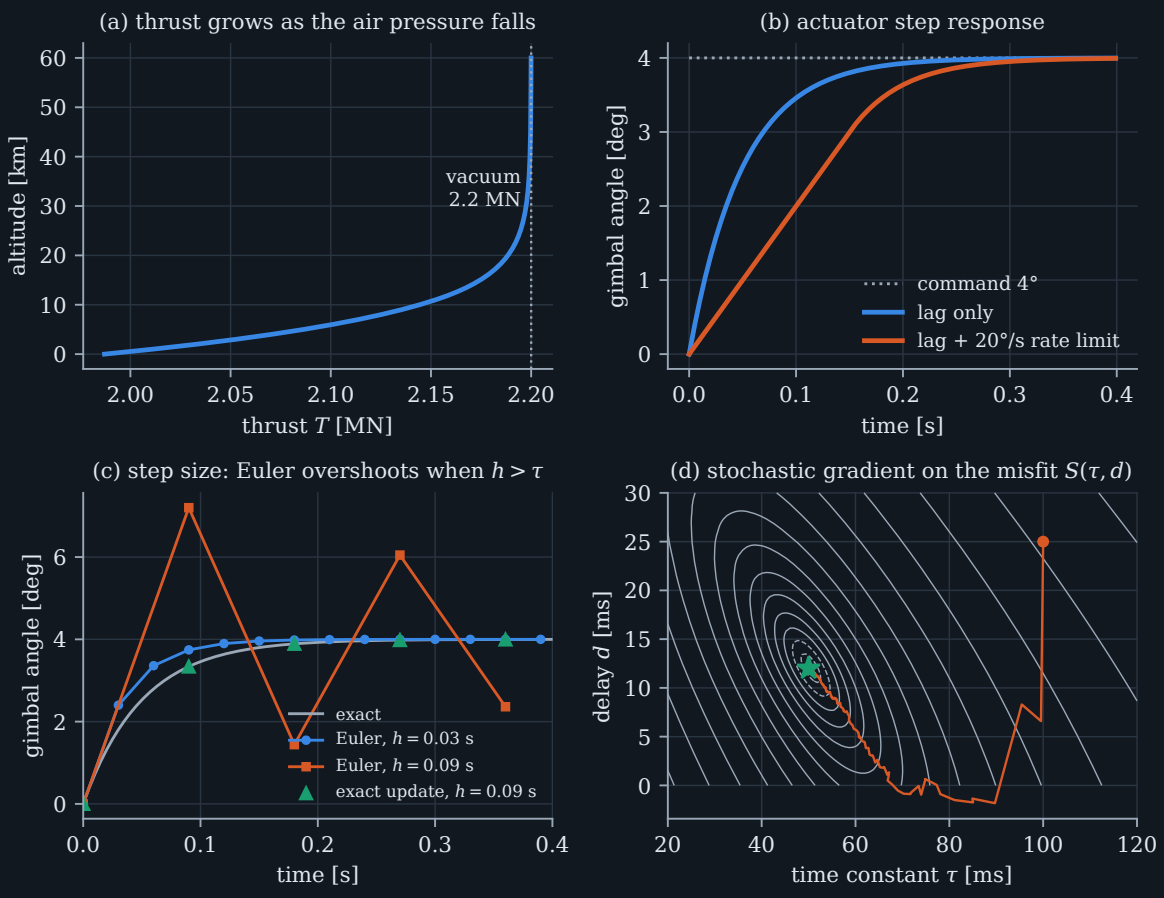


Figure 5.1: (a) LV-N1 thrust against altitude, (5.2). (b) Response to a 4° step: first-order lag alone, and with the 20 °/s rate limit (the straight start). (c) The same lag simulated by forward Euler: smooth with $h = 0.03\,\text{s} < \tau$, overshooting and ringing with $h = 0.09\,\text{s}$, while the exact update (5.12) stays on the true curve with the same large step. (d) Contours of the misfit $S(\tau, d)$ of Section 5.6, with the path of the stochastic-gradient search from its start (dot) towards the true values (star).

5.6.2 Method 1: stochastic exploration (random search)

The bluntest idea in Robert and Casella's Section 5.2.1: throw many random guesses into the box of plausible values, evaluate S at each, keep the best. With $\tau \sim \mathcal{U}(10, 200)$ ms and $d \sim \mathcal{U}(0, 50)$ ms, 5000 guesses cover the box at a spacing of roughly $190 \times 50/5000 \approx 2$ square milliseconds each, so the best one is typically within a millisecond or two of the true minimum.

- *Strength:* it cannot get stuck, needs no derivative and no starting guess.
- *Weakness:* the cost explodes with the number of parameters. To get the same spacing in 10 dimensions you would need 5000^5 guesses. It is a tool for 1–3 parameters, or for a rough first look.

5.6.3 Method 2: stochastic gradient

Gradient descent walks downhill: $\boldsymbol{\theta}_{j+1} = \boldsymbol{\theta}_j - a_j \nabla S(\boldsymbol{\theta}_j)$. When the gradient is unavailable, Robert and Casella's Section 5.2.2 estimates it *with a random direction*. Pick a random unit vector $\boldsymbol{\zeta}_j$ (Chapter 2's normalised Gaussian trick), and measure the slope of S along it with a central difference of width β_j:

$$\hat{\nabla} S(\boldsymbol{\theta}_j) = \frac{S(\boldsymbol{\theta}_j + \beta_j \boldsymbol{\zeta}_j) - S(\boldsymbol{\theta}_j - \beta_j \boldsymbol{\zeta}_j)}{2\beta_j}\, \boldsymbol{\zeta}_j, \qquad \boldsymbol{\theta}_{j+1} = \boldsymbol{\theta}_j - a_j\, \hat{\nabla} S(\boldsymbol{\theta}_j). \tag{5.14}$$

Why does this work? The fraction is (approximately) the slope along $\boldsymbol{\zeta}_j$, which equals $\nabla S \cdot \boldsymbol{\zeta}_j$. So the step is along $\boldsymbol{\zeta}_j$, scaled by how much S decreases that way. Averaged over all random directions, $\mathbb{E}[(\nabla S \cdot \boldsymbol{\zeta})\boldsymbol{\zeta}] = \nabla S/2$ in two dimensions: *on average, the step points straight downhill*. Each single step is noisy, but the noise averages out.

The step sizes. The step a_j must shrink, so the noise dies away, but not too fast, or the search stops before it arrives. The standard conditions are $\sum a_j = \infty$ (you can travel any distance) and $a_j \to 0$. We use $a_j = 20/(j+10)^{0.6}$ and $\beta_j = 1/j^{0.1}$, with τ and d measured in milliseconds so that both parameters have a similar scale.

R

```r
# test-stand data: gimbal step of 4 deg, sampled every 2 ms
set.seed(5)
t <- seq(0, 0.3, by = 0.002); dc <- 4
model <- function(th) { # th = (tau, delay), in ms
  x <- pmax(t - th[2] / 1000, 0)
  dc * (1 - exp(-x / (th[1] / 1000)))
}
y <- model(c(50, 12)) + rnorm(length(t), 0, 0.05)
S <- function(th) sum((y - model(th))^2) # misfit

# 1) random search: 5000 uniform guesses
U <- cbind(runif(5000, 10, 200), runif(5000, 0, 50))
v <- apply(U, 1, S)
cat("random search:", round(U[which.min(v), ], 2),
    " S =", round(min(v), 4), "\n")

# 2) stochastic gradient (Robert & Casella, eq. 5.4)
th <- c(100, 25)
for (j in 1:1000) {
  a <- 20 / (j + 10)^0.6; b <- 1 / j^0.1
  z <- rnorm(2); z <- z / sqrt(sum(z^2)) # random direction
  g <- (S(th + b * z) - S(th - b * z)) / (2 * b) * z
  th <- th - a * g
  if (j %in% c(1, 10, 100, 1000))
    cat("SG step", j, ":", round(th, 2), " S =", round(S(th), 4), "\n")
}

# 3) deterministic optimiser for comparison
round(optim(c(100, 25), S)$par, 2)
```

```
random search: 48.68 12.89 S = 0.3964
SG step 1 : 83.58 19.26 S = 51.7567
SG step 10 : 68.89 0.53 S = 6.3207
SG step 100 : 52.09 10.57 S = 0.5009
SG step 1000 : 49.67 12.16 S = 0.3722
[1] 49.68 12.15
```

Python

```python
# test-stand data: gimbal step of 4 deg, sampled every 2 ms
import numpy as np
from scipy.optimize import minimize
rng = np.random.default_rng(5)
t = np.arange(0, 0.3001, 0.002); dc = 4.0
def model(th): # th = (tau, delay), in ms
    x = np.clip(t - th[1] / 1000, 0, None)
    return dc * (1 - np.exp(-x / (th[0] / 1000)))
y = model([50, 12]) + rng.normal(0, 0.05, t.size)
S = lambda th: np.sum((y - model(th))**2) # misfit

# 1) random search: 5000 uniform guesses
U = np.column_stack([rng.uniform(10, 200, 5000),
                     rng.uniform(0, 50, 5000)])
v = np.array([S(u) for u in U])
print("random search:", U[v.argmin()].round(2),
      " S =", round(v.min(), 4))

# 2) stochastic gradient (Robert & Casella, eq. 5.4)
th = np.array([100.0, 25.0])
for j in range(1, 1001):
    a = 20 / (j + 10)**0.6; b = 1 / j**0.1
    z = rng.normal(size=2); z /= np.linalg.norm(z)
    g = (S(th + b * z) - S(th - b * z)) / (2 * b) * z
    th = th - a * g
    if j in (1, 10, 100, 1000):
        print("SG step", j, ":", th.round(2), " S =", round(S(th), 4))

# 3) deterministic optimiser for comparison
print(minimize(S, [100, 25], method="Nelder-Mead").x.round(2))
```

```
random search: [50.15 12.19] S = 0.3085
SG step 1 : [99.13 25.52] S = 106.2832
SG step 10 : [69.44 -0.54] S = 6.311
SG step 100 : [52.43 10.83] S = 0.3875
SG step 1000 : [50.48 12.04] S = 0.306
[50.52 12. ]
```

Reading the results. All three methods land near the true $\tau = 50\,\text{ms}$, $d = 12\,\text{ms}$, and the stochastic gradient ends where Nelder–Mead ends to within 0.01 ms in R and 0.04 ms in Python. The path is not straight (Fig. 5.1(d)): after 10 steps it has even overshot to a delay near zero, a typical random wobble early on, but the shrinking steps bring it in. The final misfit is about 0.31–0.37. That is what pure noise predicts: 151 readings with a noise standard deviation of 0.05° give $\mathbb{E}[S] \approx 151 \times 0.05^2 = 0.38$. When the misfit has fallen to the noise level, there is nothing left to fit.

Key idea: When to use which. Use a deterministic optimiser (`optim`, `minimize`) when the function is smooth and cheap. Use random search for a first look at a few parameters. Use stochastic-gradient methods when each evaluation of the function is itself noisy — for example, when S is the average of a Monte Carlo simulation — because then exact derivatives do not exist at all. That is exactly the situation of Chapter 6.

Summary

- Thrust is momentum thrust plus pressure thrust; with the vacuum thrust it is simply $T(h) = T_{\text{vac}} - p_a(h)A_e$. LV-N1: 1.987 MN on the pad, 2.2 MN in vacuum.
- Specific impulse $I_{sp} = T/(\dot{m}g_0)$ measures efficiency; $c = I_{sp}g_0$ is the effective exhaust velocity. From T_{vac}, $I_{sp,\text{vac}}$ and $I_{sp,\text{sl}}$ we get $\dot{m} = 723.67\,\text{kg/s}$, $A_e = 2.10\,\text{m}^2$ and $t_b = 140.95\,\text{s}$.
- Mass falls linearly; thrust-to-weight rises from 1.69 to 12.5; the gimbal lever arm and J_t change, so control authority $T\ell_g/J_t$ triples during the burn.
- A gimbal tilt gives the moment $T\ell_g \sin\delta$; one engine cannot roll the vehicle; steering costs $1 - \cos\delta$ of the thrust.
- The actuator is modelled as a lag ($\tau = 50\,\text{ms}$, "three time constants to get there"), a rate limit ($20\,^\circ/\text{s}$) and a position limit (6°).
- Forward Euler for the lag is stable only for $h < 2\tau$ and smooth only for $h < \tau$; the exact update $e^{-h/\tau}$ is always safe.
- Monte Carlo optimisation: random search for a first look; stochastic gradient (5.14) when derivatives are unavailable or the function is noisy. A fit is finished when the misfit reaches the noise level.

Exercises

5.1 Engine numbers. An upper-stage engine has $T_{\text{vac}} = 100\,\text{kN}$ and $I_{sp,\text{vac}} = 345\,\text{s}$. Find $\dot{m}$ and c. How long does 20 t of propellant last?

5.2 Specific impulse with altitude. Show that $I_{sp}(h) = I_{sp,\text{vac}} - p_a(h)A_e/(\dot{m}g_0)$. Evaluate it for LV-N1 at 5 km and compare with the code output.

5.3 Gimbal geometry. Verify (5.6) for $\delta_p = 0$, $\delta_y = 3^\circ$ by computing the cross product with numbers at lift-off.

5.4 Authority. Using the table of Section 5.2, compute μ_c at lift-off, max-Q and burnout, and the gimbal angle needed for a pitch acceleration of $0.05\,\text{rad/s}^2$ at each.

5.5 Lag. For $\tau = 0.05\,\text{s}$ find the time to reach 90% and 99% of a step. Show that the time to reach a fraction f is $-\tau\ln(1-f)$.

5.6 Rate limit. Repeat the rate-limited step for a 6° command. When does the lag take over, and when is the gimbal within 0.1° of the command?

5.7 Euler stability. Run the actuator code with Euler steps $h = 0.04$, 0.08 and 0.11 s, and classify each as smooth, ringing or unstable. Explain with $1 - h/\tau$.

5.8 R/Python: random search. How does the error of the random-search estimate shrink as you use 500, 5000 and 50 000 guesses? Plot the error against the number of guesses on log axes and estimate the slope.

5.9 R/Python: stochastic gradient. Rerun the stochastic gradient with $a_j = 20/(j + 10)$ (an exponent of 1 instead of 0.6). Does it still converge in 1000 steps? Why or why not?

5.10 R/Python: three parameters. Add an unknown gain k (the actuator reaches $k \times 4^\circ$ instead of 4°; true $k = 0.98$) and fit (τ, d, k) with both methods.

Reading for this chapter

- P. H. Zipfel, *Modeling and Simulation of Aerospace Vehicle Dynamics*, 2nd ed.: Section 8.2.4 (rocket propulsion) and Section 8.3.2 (the three-stage rocket simulation), Section 9.2.2 (propulsion in a 5-DOF setting) and Section 10.2.3 (actuators, including thrust vector control).
- K. J. Åström and R. M. Murray, *Feedback Systems*: Sections 5.1–5.3 (input/output response: step response, lag and bandwidth) and Sections 8.1–8.3 (transfer functions and block diagrams), for the actuator as $1/(\tau s + 1)$.
- S. Boyd and L. Vandenberghe, *Convex Optimization*: Chapter 2, Example 2.3 (the second-order cone), because the gimbal limit $|\delta| \le \delta_{\max}$ and a bound on thrust are cone constraints when they enter guidance in Chapter 9.
- R. R. Labbe, *Kalman and Bayesian Filters in Python*: Chapter 4 (Gaussian probabilities), needed for the Monte Carlo work of Chapter 6.
- C. P. Robert and G. Casella, *Monte Carlo Statistical Methods*, 2nd ed.: Sections 5.1–5.2 (Monte Carlo optimisation: stochastic exploration and stochastic gradient methods) — **STAT 3585**.

Chapter 6

Building the 6-DOF Simulator and Monte Carlo Analysis

> *Everything so far has been a part. This chapter puts the parts together into one program that flies LV-N1 from the pad to burnout — and then flies it four hundred more times, each on a slightly different day, to see what could go wrong.*

This is the chapter where you start programming in earnest. By the end you will have:

1. a complete six-degree-of-freedom simulator of about 200 lines of Python, built from the models of Chapters 2–5, every line of which you can trace back to an equation in this book (Sections 6.1–6.8);
2. evidence that it is *right*, not just that it runs (Section 6.9);
3. the mathematics of *Monte Carlo analysis* — how to turn hundreds of random runs into trustworthy numbers with error bars, how to estimate one-in-a-million failures, and how to get the same accuracy with far fewer runs (Section 6.10). This is the core of STAT 3585: Robert and Casella's Chapter 3 and Sections 4.1 and 4.4, in both R and Python.

6.1 What a simulator is

Strip away the details and a flight simulator does only three things, over and over:

> **Key idea**: A simulator in one sentence. Keep a list of numbers, the *state* $\boldsymbol{x}$, that describes the vehicle completely at one instant. Write one function $\boldsymbol{f}(t, \boldsymbol{x})$ that says how fast every number in that list is changing. Then step forward in time: $\boldsymbol{x}(t+h) \approx \boldsymbol{x}(t) + (\text{a clever average of } \boldsymbol{f}) \times h$.

The equation behind it is the one from Chapter 1:

$$\dot{\boldsymbol{x}} = \boldsymbol{f}(t, \boldsymbol{x}).$$

Everything we learned in Chapters 2–5 lives *inside* $\boldsymbol{f}$. The integrator (RK4, from Chapter 1) lives *outside* it and never needs to know anything about rockets.

6.1.1 The state of LV-N1

Our state has 15 numbers:

Symbol	Meaning	Size	Its rate of change comes from
$\boldsymbol{r}$	position of the CG, inertial axes [m]	3	the velocity $\boldsymbol{v}$
$\boldsymbol{v}$	velocity of the CG, inertial axes [m/s]	3	Newton's law (Ch. 3)
$\boldsymbol{q}$	attitude quaternion, body $\rightarrow$ inertial	4	quaternion kinematics (Ch. 2)
$\boldsymbol{\omega}$	angular velocity, body axes [rad/s]	3	Euler's law with jet damping (Ch. 3)
$\delta_p,\ \delta_y$	gimbal angles [rad]	2	the actuator model (Ch. 5)

What is *not* in the state? The mass, the CG and the inertia. Because the engine burns at a constant rate, they are known functions of time, (5.4), and are recomputed inside $\boldsymbol{f}$. Anything that can be computed from t and $\boldsymbol{x}$ should be computed, not stored: fewer states means fewer things that can drift out of agreement.

6.2 The architecture

Figure 6.1 shows how the models connect. Read it from left to right: the state goes in, the environment and vehicle models turn it into forces and moments, the equations of motion turn those into rates of change, and the integrator turns the rates into the next state. The guidance and control box closes the loop: it looks at the state and commands the gimbal.

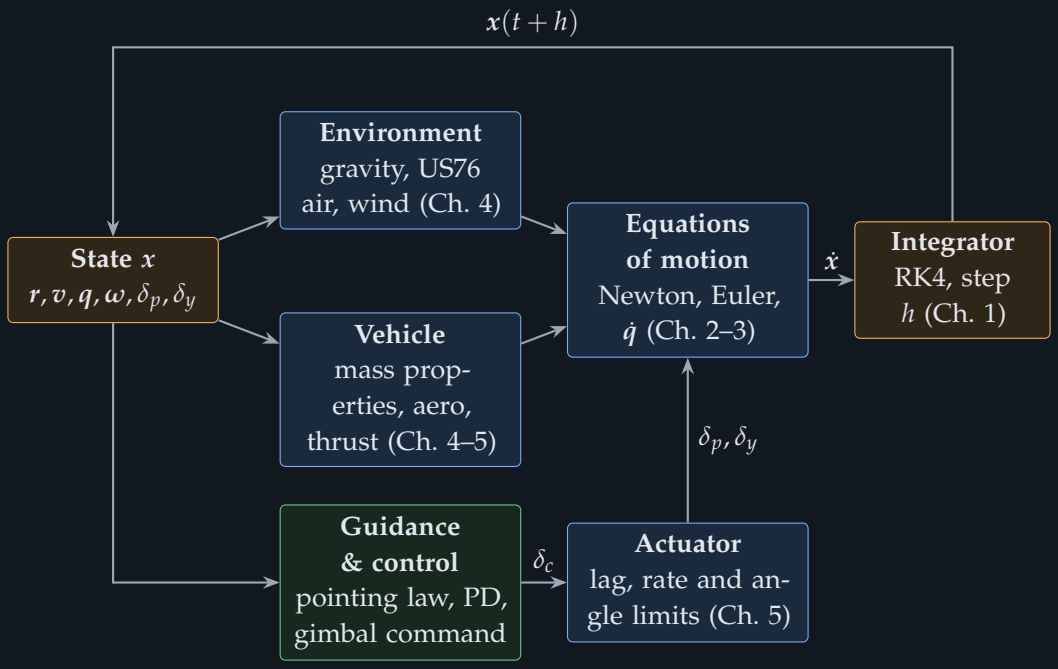


Figure 6.1: The structure of the simulator. Everything except the state and the integrator — environment, vehicle, guidance, control, actuator and equations of motion — is one Python function, $\boldsymbol{f}(t, \boldsymbol{x})$. The integrator calls it four times per step.

Reality check. Professional simulators (Zipfel's CADAC, NASA's POST and Trick) have exactly this shape, only with more boxes: separate modules that each own one model, talk through clearly named variables, and can be swapped for better versions one at a time. Our 200 lines are a small version of the same idea. Write each box as its own function and you can later replace the point-mass gravity by J_2, or the first-order actuator by a second-order one, without touching anything else.

6.3 Setting up the frames and the starting state

The inertial frame. We use an Earth-centred inertial frame whose axes coincide with the Earth-fixed axes at lift-off ($t = 0$), and whose z axis is the Earth's spin axis (Chapter 2). We place the launch site at latitude 28.5° on the x–z plane. The site's "up", "east" and "north" directions are then

$$\hat{\boldsymbol{u}} = (\cos 28.5^\circ,\ 0,\ \sin 28.5^\circ) = (0.8788,\ 0,\ 0.4772),$$
$$\hat{\boldsymbol{e}} = (0,\ 1,\ 0), \qquad \hat{\boldsymbol{n}} = \hat{\boldsymbol{u}} \times \hat{\boldsymbol{e}} = (-0.4772,\ 0,\ 0.8788).$$

The Earth is modelled as a sphere whose radius equals the ellipsoid radius at the site, $R_E = 6373.27\,\text{km}$, so the vehicle starts at $\boldsymbol{r}_0 = R_E\hat{\boldsymbol{u}}$ and its altitude is $h = \|\boldsymbol{r}\| - R_E$.

The starting velocity. On the pad the vehicle is at rest *relative to the ground*, but the ground is turning: $\boldsymbol{v}_0 = \boldsymbol{\omega}_E \times \boldsymbol{r}_0$, which is 408.7 m/s east (Chapter 4). This free speed is why rockets launch eastwards.

The starting attitude. At lift-off the body axes are: x_B up, z_B pointing west (opposite to the planned downrange direction, so that a positive pitch rotates the nose east), and $y_B = z_B \times x_B$ completing a right-handed set. Putting these three directions in the columns of a matrix gives the rotation $\mathbf{R}$ from body to inertial axes, and the quaternion follows from Chapter 2's conversion:

$$q_0 = \tfrac{1}{2}\sqrt{1 + R_{11} + R_{22} + R_{33}}, \qquad q_1 = \frac{R_{32} - R_{23}}{4q_0},\ \ q_2 = \frac{R_{13} - R_{31}}{4q_0},\ \ q_3 = \frac{R_{21} - R_{12}}{4q_0}.$$

Here $R_{11} + R_{22} + R_{33} = 0.8788 + 0 + 0$, so $q_0 = \tfrac{1}{2}\sqrt{1.8788} = 0.6853$. The angular velocity and the gimbal angles start at zero.

6.4 Gravity

Newton's law of gravitation for a spherical Earth gives the acceleration

$$\boldsymbol{g}(\boldsymbol{r}) = -\frac{\mu}{\|\boldsymbol{r}\|^3}\,\boldsymbol{r}, \qquad \mu = 3.986\,004\,418 \times 10^{14}\,\text{m}^3/\text{s}^2. \tag{6.1}$$

Read it as: size $\mu/\|\boldsymbol{r}\|^2$, direction $-\boldsymbol{r}/\|\boldsymbol{r}\|$ (towards the centre).

> **Worked example**: Gravity on the pad and at burnout. On the pad, $\|\boldsymbol{r}\| = 6\,373\,268$ m: $g = 3.986 \times 10^{14}/6\,373\,268^2 = 9.813\,\mathrm{m/s^2}$. At burnout, 109.7 km higher: $g = 3.986 \times 10^{14}/6\,483\,000^2 = 9.484\,\mathrm{m/s^2}$, 3.4% less. A constant g would make every second of the climb slightly too expensive.

The real Earth is flattened, which adds the J_2 term of Chapter 2. Its relative size is about $\frac{3}{2}J_2(a/r)^2 \approx 0.0016$, i.e. $0.016\,\mathrm{m/s^2}$. Over a 141 s burn that is about 2 m/s and 150 m — important for reaching an exact orbit, unimportant for learning. Our teaching simulator uses (6.1); adding J_2 is Exercise 6.3.

6.5 The derivative function, step by step

Here is the heart of the program: given t and $\boldsymbol{x}$, compute $\dot{\boldsymbol{x}}$. Each step uses only results of the steps before it, and each is an equation you have already met.

Step 1. **Unpack** $\boldsymbol{r}, \boldsymbol{v}, \boldsymbol{q}, \boldsymbol{\omega}, \delta_p, \delta_y$ from $\boldsymbol{x}$.

Step 2. **Mass properties.** Propellant left $m_p = m_{p,0} - \dot{m}\,t$; then m, s_{cg}, J_x, J_t from the tank model (Chapters 2–3, Section 5.2).

Step 3. **Air.** Altitude $h = \|\boldsymbol{r}\| - R_E$; then T, p_a, ρ, a from US76 (Section 4.3).

Step 4. **Air-relative velocity** $\boldsymbol{v}_{\text{air}} = \boldsymbol{v} - \boldsymbol{\omega}_E \times \boldsymbol{r} - \boldsymbol{w}$ (4.10), then into body axes with $\mathbf{R}(\boldsymbol{q})^{\mathsf{T}}$: (u, v, w).

Step 5. **Thrust** $T = T_{\text{vac}} - p_a A_e$ while burning (5.2); gimbal arm $\ell_g = s_{\text{gimbal}} - s_{cg}$.

Step 6. **Aerodynamics.** $\bar{q} = \frac{1}{2}\rho V^2$, Mach, α, then A and N from the tables, and the moment about the CG (Sections 4.6 and 4.7).

Step 7. **Guidance and control.** Wanted nose direction, pointing error, PD moment, gimbal command, actuator rates (Section 6.6 below and Section 5.4).

Step 8. **Thrust vector** $\boldsymbol{F}_T^B$ (5.5) and its moment $\boldsymbol{r}_g \times \boldsymbol{F}_T^B$.

Step 9. **Newton:** $\dot{\boldsymbol{v}} = \mathbf{R}(\boldsymbol{q})(\boldsymbol{F}_T^B + \boldsymbol{F}_A^B)/m + \boldsymbol{g}(\boldsymbol{r})$.

Step 10. **Euler with jet damping:** $J\dot{\boldsymbol{\omega}} = \boldsymbol{M}_T + \boldsymbol{M}_A - \boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega}) - \dot{m}\,\ell_e^2\,\boldsymbol{\omega}_\perp$ (Chapter 3).

Step 11. **Attitude:** $\dot{\boldsymbol{q}} = \frac{1}{2}\,\boldsymbol{q} \odot (0, \boldsymbol{\omega})$ (Chapter 2).

Step 12. **Pack** $\dot{\boldsymbol{x}} = (\boldsymbol{v},\ \dot{\boldsymbol{v}},\ \dot{\boldsymbol{q}},\ \dot{\boldsymbol{\omega}},\ \dot{\delta}_p,\ \dot{\delta}_y)$ and return it.

Two small but important points:

- In Step 10 we keep the jet-damping term but drop $\dot{\mathbf{J}}\boldsymbol{\omega}$. Chapter 3 showed the two partly cancel and that the remaining effect is small for LV-N1; the research version of this simulator includes both. Keeping your model choices *written down* like this is part of good simulation practice.

- The thrust switches off when $t > t_b$. That is a sudden jump in $\boldsymbol{f}$. RK4 assumes $\boldsymbol{f}$ is smooth, so we make sure a time step ends exactly at t_b (the simulation simply stops there).

6.6 Guidance and control, just enough to fly

Guidance (Chapter 10) and control (Chapter 11) get their own chapters. To test the simulator we need only a simple, classical recipe for getting through the atmosphere:

1. **Vertical rise** for 6 s: clear the tower, point straight up.
2. **Pitch kick** over the next 10 s: tilt the wanted nose direction smoothly from vertical to 16° towards the east, following $\theta(t) = 16^\circ \times \frac{1}{2}\big(1 - \cos(\pi t'/10)\big)$, where t' is the time since the kick began. (The cosine shape starts and ends with zero rate, so nothing is jerked.)
3. **Gravity turn**: over 6 s, blend the wanted direction into the direction of the air-relative velocity, and then simply keep pointing along it.

> **Key idea**: Why a gravity turn?. If the nose points along the airflow, the angle of attack is zero, so there is no normal force and no aerodynamic moment — which matters because, as Chapter 4 showed, LV-N1 is unstable. Meanwhile gravity pulls the velocity downwards, bending the path gently over towards the horizontal. The vehicle turns *without being turned*: gravity does the steering, the engine only follows. The small pitch kick at the start chooses which way it will fall.

Control. In body axes, the wanted direction $\hat{\boldsymbol{x}}_d$ has small components $(\cdot, d_y, d_z)$ when the nose is nearly on target. The pointing errors are $e_y = -d_z$ (about y_B) and $e_z = d_y$ (about z_B), and a PD law asks for an angular acceleration, hence a moment:

$$M_{y,c} = J_t\big(k_p e_y - k_d \omega_y\big) - M_{A,y}, \qquad k_p = \omega_n^2,\ k_d = 2\zeta\omega_n,$$

with $\omega_n = 1.5\,\mathrm{rad/s}$ and $\zeta = 0.7$ (and the same for z). The last term *cancels the predicted aerodynamic moment*: the controller uses the aerodynamic model to work out the destabilising push and cancels it in advance. Without it, Chapter 4's $\mu_\alpha \approx 2/\mathrm{s}^2$ at max-Q would almost cancel $k_p = 2.25$. The gimbal command then follows from (5.7).

6.7 Stepping forward in time

The integrator is RK4, (1.13): four evaluations of $\boldsymbol{f}$ per step, combined with weights $\frac{1}{6}, \frac{2}{6}, \frac{2}{6}, \frac{1}{6}$. Its *order* is 4: halve the step and the error falls by $2^4 = 16$. Forward Euler has order 1: halve the step, halve the error. You can see both on the simplest equation there is, $\dot{y} = -y$ with $y(0) = 1$, whose exact solution at $t = 1$ is e^{-1}:

R

```r
# Euler and RK4 on y' = -y, y(0) = 1, up
    to t = 1
f <- function(y) -y
for (h in c(0.1, 0.05, 0.025)) {
  ye <- 1; yr <- 1
  for (k in 1:round(1 / h)) {
    ye <- ye + h * f(ye) # Euler
    k1 <- f(yr); k2 <- f(yr + h/2 * k1) #
        RK4
    k3 <- f(yr + h/2 * k2); k4 <- f(yr + h *
         k3)
    yr <- yr + h/6 * (k1 + 2*k2 + 2*k3 + k4)

  }
  cat(sprintf("h=%.3f Euler err %.2e RK4
      err %.2e\n",
          h, abs(ye - exp(-1)), abs(yr -
              exp(-1))))
}
```

```
h=0.100 Euler err 1.92e-02 RK4 err 3.33e-07
h=0.050 Euler err 9.39e-03 RK4 err 2.00e-08
h=0.025 Euler err 4.65e-03 RK4 err 1.22e-09
```

Python

```python
# Euler and RK4 on y' = -y, y(0) = 1, up
    to t = 1
import math
f = lambda y: -y
for h in (0.1, 0.05, 0.025):
    ye = yr = 1.0
    for k in range(round(1 / h)):
        ye = ye + h * f(ye) # Euler
        k1 = f(yr); k2 = f(yr + h/2 * k1) #
            RK4
        k3 = f(yr + h/2 * k2); k4 = f(yr + h
             * k3)
        yr = yr + h/6 * (k1 + 2*k2 + 2*k3 +
            k4)
    print(f"h={h:.3f} Euler err {abs(ye -
        math.exp(-1)):.2e}"
          f" RK4 err {abs(yr - math.exp(-1))
              :.2e}")
```

```
h=0.100 Euler err 1.92e-02 RK4 err 3.33e-07
h=0.050 Euler err 9.39e-03 RK4 err 2.00e-08
h=0.025 Euler err 4.65e-03 RK4 err 1.22e-09
```

Euler's error halves each time ($1.92 \to 0.94 \to 0.47$ in units of 10^{-2}); RK4's falls by $3.33/0.200 = 16.7$ and then $20.0/1.22 = 16.4$. And notice the sizes: at the *same* step, RK4 is about 60 000 times more accurate, for four times the work.

Choosing the step. The fastest thing in our model is the actuator, $\tau = 0.05\,\mathrm{s}$. Chapter 5 showed that a step must be well below τ. We use $h = 0.01\,\mathrm{s} = \tau/5$: 14 095 steps and 56 380 evaluations of f for the whole burn, about two seconds on a laptop.

Keeping the quaternion honest. After each step we divide q by its length. The exact equations keep $\|q\| = 1$; RK4 keeps it only approximately, and without the correction the error would slowly accumulate (Chapter 2).

6.8 The complete program

Here is the whole simulator. The numbered comments match the sections of this chapter; the functions `interp`, `mass_props`, `wind_shape` and the atmosphere are the ones from Chapters 2–5. Type it in (or use the file supplied with the book), run it, and then change things.

```python
"""lvsim.py -- a teaching six-degree-of-freedom ascent simulator for LV-N1.

Chapter 6 of "Hybrid Quantum-Classical Autonomy for Launch Vehicles".
Every block below matches one section of Chapters 2-6. Standard library only.
"""
import math
from c4_atmos import atmos # Chapter 4: US Standard Atmosphere

# ---------------------------------------------------------------- 1. constants
```

```
G0, MU, WE = 9.80665, 3.986004418e14, 7.2921159e-5 # g0, Earth GM, Earth spin
RE = 6378137.0 * (1 - math.sin(math.radians(28.5))**2 / 298.257223563)

# ------------------------------------------------------------------- 2. vehicle LV-N1
V = dict(L=40.0, R=1.5, m_dry=18000.0, s_dry=26.0, m_prop=102000.0,
       s_top=8.0, s_bot=34.0, r_tank=1.45, T_vac=2.2e6, Isp=310.0,
       Isp_sl=280.0, s_gimbal=39.0, s_exit=40.0,
       d_max=math.radians(6.0), rate_max=math.radians(20.0), tau=0.05)
V['mdot'] = V['T_vac'] / (V['Isp'] * G0) # Chapter 5
V['A_e'] = (V['T_vac'] - V['mdot'] * V['Isp_sl'] * G0) / 101325.0
V['S'] = math.pi * V['R']**2
MACH = [0.0, 0.5, 0.8, 0.9, 1.0, 1.1, 1.2, 1.5, 2.0, 3.0, 4.0, 6.0, 10.0]
CA = [0.30, 0.28, 0.30, 0.38, 0.50, 0.55, 0.53, 0.45, 0.38, 0.30, 0.26, 0.22, 0.20]
CNA = [2.0, 2.0, 2.05, 2.15, 2.35, 2.55, 2.6, 2.5, 2.4, 2.3, 2.2, 2.1, 2.05]
SCP = [16.0, 16.0, 16.1, 16.3, 16.8, 17.2, 17.4, 17.6, 17.7, 17.8, 17.8, 17.8, 17.8]

# ------------------------------------------------------------------- 3. small helpers (
    Chapter 2)
def add(a, b, k=1.0): return [a[i] + k * b[i] for i in range(len(a))]
def dot(a, b): return a[0]*b[0] + a[1]*b[1] + a[2]*b[2]
def norm(a): return math.sqrt(dot(a, a))
def cross(a, b):
   return [a[1]*b[2] - a[2]*b[1], a[2]*b[0] - a[0]*b[2], a[0]*b[1] - a[1]*b[0]]
def clamp(x, lo, hi): return min(max(x, lo), hi)
def interp(x, xs, ys): # Chapter 4
   if x <= xs[0]: return ys[0]
   if x >= xs[-1]: return ys[-1]
   i = 0
   while xs[i + 1] < x: i += 1
   return ys[i] + (x - xs[i]) / (xs[i+1] - xs[i]) * (ys[i+1] - ys[i])
def qmul(p, q): # quaternion product
   return [p[0]*q[0] - p[1]*q[1] - p[2]*q[2] - p[3]*q[3],
         p[0]*q[1] + q[0]*p[1] + p[2]*q[3] - p[3]*q[2],
         p[0]*q[2] + q[0]*p[2] + p[3]*q[1] - p[1]*q[3],
         p[0]*q[3] + q[0]*p[3] + p[1]*q[2] - p[2]*q[1]]
def body_to_inertial(q, v): # R(q) v
   w = qmul(qmul(q, [0.0] + list(v)), [q[0], -q[1], -q[2], -q[3]])
   return w[1:]
def inertial_to_body(q, v): # R(q)^T v
   return body_to_inertial([q[0], -q[1], -q[2], -q[3]], v)

# ------------------------------------------------------------------- 4. models
def mass_props(mp, v):
   """mass, CG station, roll and transverse inertia (Chapters 2-3)."""
   Lp = max(mp, 0.0) / v['m_prop'] * (v['s_bot'] - v['s_top'])
   sp = v['s_bot'] - Lp / 2
   m = v['m_dry'] + mp
   scg = (v['m_dry'] * v['s_dry'] + mp * sp) / m
   R, r = v['R'], v['r_tank']
   Jx = 0.5 * v['m_dry'] * R*R + 0.5 * mp * r*r
   Jt = (v['m_dry'] * (3*R*R + v['L']**2) / 12 + v['m_dry'] * (v['s_dry'] - scg)**2
        + mp * (3*r*r + Lp*Lp) / 12 + mp * (sp - scg)**2)
   return m, scg, Jx, Jt

def wind_shape(h): # 0 on the ground, 1 at 12 km, 0 above 25 km
   if h <= 0 or h >= 25000: return 0.0
   if h < 12000: return math.sin(0.5 * math.pi * h / 12000)**2
   return math.cos(0.5 * math.pi * (h - 12000) / 13000)**2

# ------------------------------------------------------------------- 5. the simulator
class LVSim:
   def __init__(self, **disp):
      """disp: dispersions, e.g. k_isp=1.01, wind_e=20.0 (all neutral by default)"""
      self.d = dict(k_mdot=1.0, k_isp=1.0, k_CA=1.0, k_CNa=1.0, d_scp=0.0,
                k_mdry=1.0, wind_n=0.0, wind_e=0.0, gimbal_bias=0.0)
```

```
        self.d.update(disp)
        d, v = self.d, dict(V)
        v['m_dry'] *= d['k_mdry']
        self.v = v
        self.mdot = V['mdot'] * d['k_mdot']
        self.T_vac = self.mdot * V['Isp'] * d['k_isp'] * G0
        self.t_b = V['m_prop'] / self.mdot
        lat = math.radians(28.5)
        # launch site and local directions (inertial frame = Earth frame at t = 0)
        self.up = [math.cos(lat), 0.0, math.sin(lat)]
        self.east = [0.0, 1.0, 0.0]
        self.north = cross(self.up, self.east)
        self.r0 = [RE * u for u in self.up]
        self.kp, self.kd = 1.5**2, 2 * 0.7 * 1.5 # PD gains (wn, zeta)

    # ---- environment
    def gravity(self, r):
        rn = norm(r)
        return [-MU / rn**3 * x for x in r] # point-mass gravity

    # ---- guidance: vertical rise, pitch kick, then follow the airflow
    def wanted_x(self, t, v_air):
        if t < 6.0: return self.up
        th = math.radians(16.0) * (0.5 * (1 - math.cos(math.pi * min(t - 6, 10) / 10)))
        x_kick = add([math.cos(th) * u for u in self.up], self.east, math.sin(th))
        if t < 16.0 or norm(v_air) < 1: return x_kick
        w = 0.5 * (1 - math.cos(math.pi * min(t - 16, 6) / 6)) # blend over 6 s
        x = add([(1 - w) * a for a in x_kick], v_air, w / norm(v_air))
        return [a / norm(x) for a in x]

    # ---- the right-hand side f(t, x): Chapter 3's equations of motion
    def f(self, t, s, log=None):
        v, d = self.v, self.d
        r, vel, q, w, dp, dy = s[0:3], s[3:6], s[6:10], s[10:13], s[13], s[14]
        burning = t <= self.t_b + 1e-9
        m, scg, Jx, Jt = mass_props(v['m_prop'] - self.mdot * min(t, self.t_b), v)
        h = norm(r) - RE
        Tk, p, rho, a = atmos(max(h, 0.0)) if h < 86000 else (186.9, 0.0, 0.0, 274.1)
        # air-relative velocity (Chapter 4)
        wind = [wind_shape(h) * (d['wind_n'] * self.north[i] + d['wind_e'] * self.east[i])
                for i in range(3)]
        v_air = [vel[0] + WE * r[1] - wind[0], vel[1] - WE * r[0] - wind[1], vel[2] - wind
            [2]]
        u = inertial_to_body(q, v_air)
        Va = norm(u)
        # propulsion and thrust-vector control (Chapter 5)
        T = self.T_vac - p * v['A_e'] if burning else 0.0
        lg = v['s_gimbal'] - scg
        # aerodynamics (Chapter 4)
        qbar = 0.5 * rho * Va * Va
        F_A, M_A, alpha = [0.0, 0.0, 0.0], [0.0, 0.0, 0.0], 0.0
        if rho > 0 and Va > 1e-3:
            mach = Va / a
            vt = math.hypot(u[1], u[2])
            alpha = math.atan2(vt, u[0])
            N = qbar * v['S'] * interp(mach, MACH, CNA) * d['k_CNa'] * alpha
            F_A = [-qbar * v['S'] * interp(mach, MACH, CA) * d['k_CA'],
                   -N * u[1] / vt if vt > 0 else 0.0, -N * u[2] / vt if vt > 0 else 0.0]
            M_A = cross([scg - interp(mach, MACH, SCP) - d['d_scp'], 0, 0], F_A)
        # control: PD on the pointing error, then moment -> gimbal command (Chapter 5)
        xd = inertial_to_body(q, self.wanted_x(t, v_air))
        e_y, e_z = -xd[2], xd[1]
        My = Jt * (self.kp * e_y - self.kd * w[1]) - M_A[1] # cancel predicted aero moment
        Mz = Jt * (self.kp * e_z - self.kd * w[2]) - M_A[2]
        dpc = dyc = 0.0
```

```
        if T > 1:
            dpc = math.asin(clamp(My / (T * lg), -1, 1))
            dyc = math.asin(clamp(Mz / (T * lg), -1, 1))
        dpc, dyc = clamp(dpc, -v['d_max'], v['d_max']), clamp(dyc, -v['d_max'], v['d_max'])
        ddp = clamp((dpc - dp) / v['tau'], -v['rate_max'], v['rate_max']) # actuator
        ddy = clamp((dyc - dy) / v['tau'], -v['rate_max'], v['rate_max'])
        de = dp + d['gimbal_bias']
        F_T = [T * math.cos(de) * math.cos(dy), -T * math.cos(de) * math.sin(dy), T * math.
            sin(de)]
        M_T = cross([-lg, 0, 0], F_T)
        # translation: Newton in inertial axes
        F_I = body_to_inertial(q, add(F_T, F_A))
        acc = add([x / m for x in F_I], self.gravity(r))
        # rotation: Euler's law with jet damping (Chapter 3)
        M = add(M_T, M_A)
        if burning:
            c = self.mdot * (v['s_exit'] - scg)**2
            M[1] -= c * w[1]; M[2] -= c * w[2]
        wJw = cross(w, [Jx * w[0], Jt * w[1], Jt * w[2]])
        wdot = [(M[0] - wJw[0]) / Jx, (M[1] - wJw[1]) / Jt, (M[2] - wJw[2]) / Jt]
        # attitude: quaternion kinematics (Chapter 2)
        qd = [0.5 * x for x in qmul(q, [0.0] + list(w))]
        if log is not None:
            log.update(t=t, h=h, V=Va, qbar=qbar, alpha=alpha, mach=Va / a, T=T, m=m,
                       delta=math.hypot(dp, dy))
        return list(vel) + acc + qd + wdot + [ddp, ddy]

    # ---- initial state: on the pad, body x up, turning with the Earth
    def x0(self):
        zb = [-x for x in self.east] # body z points opposite to downrange
        yb = cross(zb, self.up)
        R = [[self.up[i], yb[i], zb[i]] for i in range(3)] # body -> inertial
        q0 = 0.5 * math.sqrt(1 + R[0][0] + R[1][1] + R[2][2])
        q = [q0, (R[2][1] - R[1][2]) / (4*q0), (R[0][2] - R[2][0]) / (4*q0),
             (R[1][0] - R[0][1]) / (4*q0)]
        vel = [-WE * self.r0[1], WE * self.r0[0], 0.0]
        return list(self.r0) + vel + q + [0.0] * 5

    # ---- RK4 (Chapter 1), with quaternion re-normalisation
    def run(self, dt=0.01, t_end=None, every=None):
        t_end = self.t_b if t_end is None else t_end
        s, t, hist, peak = self.x0(), 0.0, [], dict(qbar=0, qalpha=0, delta=0)
        info = {}
        while t < t_end - 1e-12:
            h = min(dt, t_end - t)
            k1 = self.f(t, s)
            k2 = self.f(t + h/2, add(s, k1, h/2))
            k3 = self.f(t + h/2, add(s, k2, h/2))
            k4 = self.f(t + h, add(s, k3, h))
            s = [s[i] + h/6 * (k1[i] + 2*k2[i] + 2*k3[i] + k4[i]) for i in range(15)]
            t += h
            n = math.sqrt(sum(x*x for x in s[6:10]))
            s[6:10] = [x / n for x in s[6:10]]
            self.f(t, s, info)
            peak['qbar'] = max(peak['qbar'], info['qbar'])
            peak['qalpha'] = max(peak['qalpha'], info['qbar'] * info['alpha'])
            peak['delta'] = max(peak['delta'], info['delta'])
            if every and abs(t / every - round(t / every)) < 1e-6:
                hist.append(dict(info))
        r, vel = s[0:3], s[3:6]
        v_rel = [vel[0] + WE * r[1], vel[1] - WE * r[0], vel[2]] # Earth-relative
        up = [x / norm(r) for x in r]
        gamma = math.degrees(math.asin(dot(v_rel, up) / norm(v_rel)))
        return dict(t=t, h=norm(r) - RE, V=norm(v_rel), V_inertial=norm(vel),
                    gamma=gamma, **peak, hist=hist, state=s)
```

```
if __name__ == '__main__':
    import time
    t0 = time.time()
    res = LVSim().run()
    print(f"burnout after {res['t']:.2f} s (run took {time.time() - t0:.1f} s)")
    print(f"altitude {res['h'] / 1000:9.2f} km")
    print(f"speed (Earth) {res['V']:9.1f} m/s")
    print(f"speed (inertial) {res['V_inertial']:9.1f} m/s")
    print(f"flight-path angle {res['gamma']:9.2f} deg")
    print(f"max-Q {res['qbar'] / 1000:9.2f} kPa")
    print(f"max q*alpha {res['qalpha']:9.0f} Pa rad")
    print(f"max gimbal {math.degrees(res['delta']):9.3f} deg")
```

```
burnout after 140.95 s (run took 1.9 s)
altitude 109.73 km
speed (Earth) 4608.5 m/s
speed (inertial) 4973.6 m/s
flight-path angle 29.69 deg
max-Q 56.71 kPa
max q*alpha 857 Pa rad
max gimbal 0.306 deg
```

Reading the flight. Figure 6.2 tells the story of the ascent:

- $t = 6\,\mathrm{s}$: 126 m up, 42 m/s. The kick begins.
- $t = 16\,\mathrm{s}$: 0.94 km, 122 m/s. The kick has put the nose 16° from vertical, while the velocity is still nearly vertical: α peaks at 5.9°. This is harmless only because $\bar{q}$ is still tiny (8 kPa).
- $t = 22\,\mathrm{s}$: the gravity turn is established; α falls below 1°.
- $t = 52\,\mathrm{s}$: max-Q, 56.7 kPa at 10.9 km, Mach 1.88, $\alpha = 0.54°$. The largest $\bar{q}\,\alpha$ load of the flight is 857 Pa rad.
- $t = t_b = 140.95\,\mathrm{s}$: burnout at 109.7 km, flying at 4608 m/s relative to the Earth (4974 m/s inertial), climbing at 29.7°.

LV-N1 alone does not reach orbit (that needs about 7.8 km/s horizontally), so in a real vehicle an upper stage would take over here. But every model in a full launch simulation is already present.

6.9 Is the simulator right? Verification

A simulator that runs is not yet a simulator that is right. *Verification* means checking that the program solves the equations we *meant* it to solve. (Whether those equations describe the real vehicle well enough is *validation*, Chapter 16.) The best tests switch off parts of the model until an exact answer is known.

Test 1: the rocket equation. Switch off gravity, air and Earth rotation, and keep the nose vertical. Then the only force is the vacuum thrust along the velocity, and Tsiolkovsky's

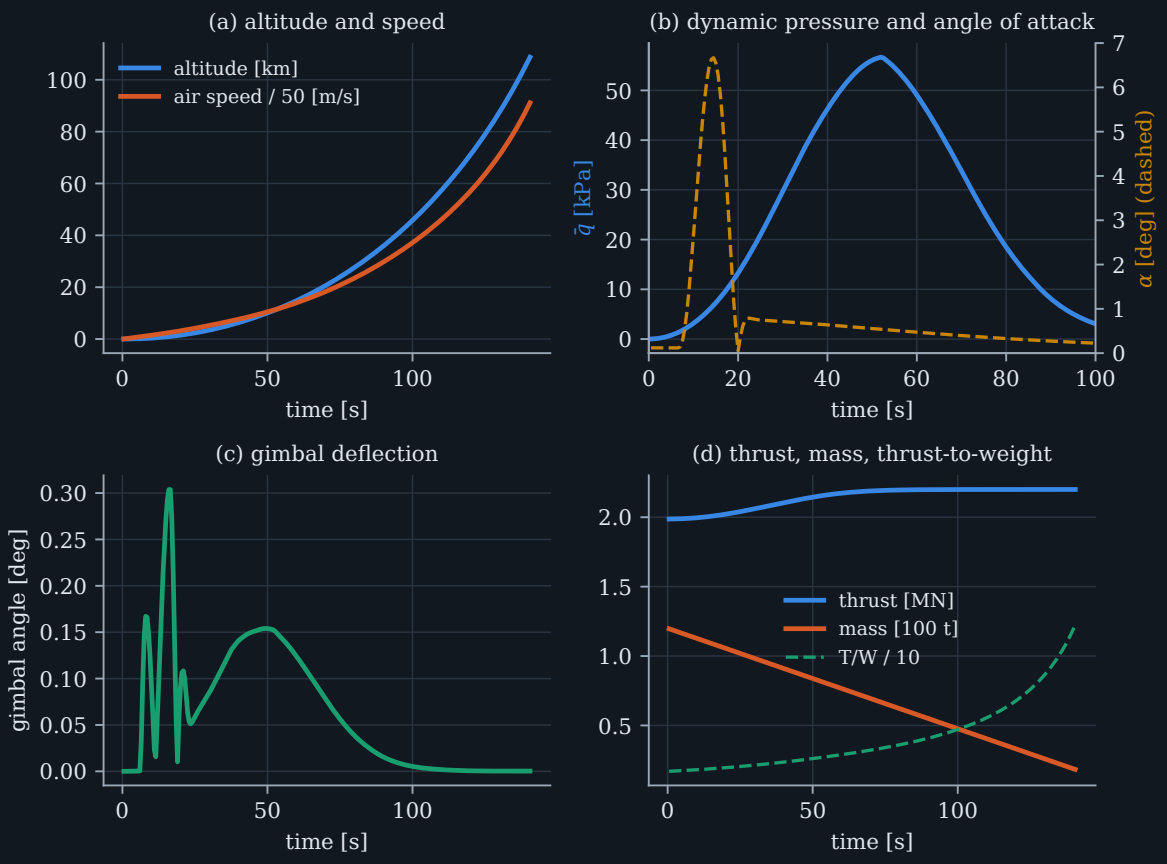


Figure 6.2: The nominal LV-N1 ascent from `lvsim.py`. (a) Altitude and air-relative speed. (b) Dynamic pressure (solid, left axis) and angle of attack (dashed, right axis): α is large only during the pitch kick, when the vehicle is slow and $\bar{q}$ is tiny, and stays near 0.5° through max-Q. (c) The gimbal works hardest during the kick (0.30° at 16.5 s) and around max-Q (0.15°), and almost not at all above the atmosphere. (d) Thrust rises with altitude while the mass falls, so the thrust-to-weight ratio climbs from 1.7 to over 12.

equation (1.6) gives the exact answer $\Delta v = 310 \times 9.80665 \times \ln(120/18) = 5767.361\,427\,\mathrm{m/s}$. If the simulator disagrees, something in the mass model, the thrust, the frames or the integrator is wrong.

Test 2: step-size convergence. Run the full simulation with smaller and smaller steps and compare with a very small reference step. The differences must shrink.

Test 3: an independent implementation. The research version of this simulator, written separately and with many more options, gives 109.73 km, 4608.5 m/s and 56.71 kPa for the same configuration — the same to every printed digit.

```
import math
import lvsim

# Test 1: no air, no gravity, no Earth spin, no turning -> rocket
    equation
```

```
class Bare(lvsim.LVSim):
    def gravity(self, r): return [0.0, 0.0, 0.0]
    def wanted_x(self, t, v_air): return self.up
lvsim_atmos, lvsim_WE = lvsim.atmos, lvsim.WE
lvsim.atmos = lambda h: (288.15, 0.0, 0.0, 340.3) # vacuum
lvsim.WE = 0.0 # Earth not turning
res = Bare().run(dt=0.01)
c = lvsim.V['Isp'] * lvsim.G0
exact = c * math.log(120000 / 18000)
print(f"Test 1 simulated dv = {res['V_inertial']:.6f} m/s")
print(f" rocket eq. = {exact:.6f} m/s")
lvsim.atmos, lvsim.WE = lvsim_atmos, lvsim_WE # restore

# Test 2: step-size study of the full simulation (RK4 should be 4th
    order)
ref = lvsim.LVSim().run(dt=0.0025)
print("Test 2 dt [s] altitude error [m]")
for dt in (0.04, 0.02, 0.01, 0.005):
    r = lvsim.LVSim().run(dt=dt)
    print(f" {dt:6.3f} {abs(r['h'] - ref['h']):.3e}")
```

```
Test 1 simulated dv = 5767.361427 m/s
      rocket eq. = 5767.361427 m/s
Test 2 dt [s] altitude error [m]
       0.040 2.253e-04
       0.020 1.945e-04
       0.010 1.055e-04
       0.005 1.777e-05
```

Test 1 agrees to all ten printed digits. In Test 2 the errors are all below a quarter of a *millimetre* over a 110 km climb, so 0.01 s is more than accurate enough. But notice they do not fall by 16 each time, as the simple test above did. Why not? Our f has corners: the rate limit and the gimbal clamp switch on and off, the guidance phases change at 6, 16 and 22 s. RK4's fourth order is a promise for *smooth* functions. Near a corner it behaves more like a lower-order method. That is normal, and a good reason to always run a step-size study rather than assume the textbook order.

6.10 Monte Carlo analysis

6.10.1 Why one flight is not enough

The run above used the *nominal* vehicle on a calm, standard day. The real LV-N1 will have an engine slightly better or worse than the data sheet, aerodynamic coefficients that are only known to about ±10%, a dry mass that grew during construction, and a wind that nobody can predict exactly. The questions that matter are about all those possibilities at once:

- What burnout altitude should we *expect*, and how sure are we?
- What is the *probability* that the $\bar{q}\,\alpha$ load exceeds what the structure can take?
- How rare is a really bad outcome — one in a hundred, or one in a million?

Each of these is an *expected value*, and each expected value is an *integral*. Monte Carlo analysis computes those integrals by simulating. This section follows Robert and Casella, *Monte Carlo Statistical Methods*, Chapter 3 (Monte Carlo integration) and Sections 4.1 and 4.4 (controlling the variance) — the STAT 3585 core for this chapter.

6.10.2 Classical Monte Carlo integration

Let X be the random inputs (with density f) and $h(X)$ the output we care about. We want

$$\mu = \mathbb{E}[h(X)] = \int h(x)\, f(x)\, \mathrm{d}x. \tag{6.2}$$

> **Key idea**: The Monte Carlo estimator. Draw $X_1, \ldots, X_n$ independently from f, and average:
>
> $$\hat{\mu}_n = \frac{1}{n} \sum_{i=1}^{n} h(X_i). \tag{6.3}$$

Why does this work? Two theorems from probability.

- **The law of large numbers**: $\hat{\mu}_n \to \mu$ as $n \to \infty$. The average of many independent draws settles at the expected value.
- **The central limit theorem**: for large n, $\hat{\mu}_n$ is approximately normal with mean μ and standard deviation $\sigma/\sqrt{n}$, where $\sigma^2 = \mathrm{Var}[h(X)]$.

The second one is the useful one, so let us see where the $\sqrt{n}$ comes from. The variance of a sum of independent terms is the sum of the variances, and a constant factor comes out squared:

$$\mathrm{Var}[\hat{\mu}_n] = \mathrm{Var}\left[\frac{1}{n}\sum h(X_i)\right] = \frac{1}{n^2}\sum_{i=1}^{n} \mathrm{Var}[h(X_i)] = \frac{1}{n^2}\, n\sigma^2 = \frac{\sigma^2}{n}.$$

Taking the square root, the *standard error* is $\sigma/\sqrt{n}$. We do not know σ, but we can estimate it by the sample standard deviation s of the $h(X_i)$. A 95% confidence interval is then

$$\hat{\mu}_n \pm 1.96\, \frac{s}{\sqrt{n}}. \tag{6.4}$$

> **Key idea**: The square-root law. The error of Monte Carlo falls like $1/\sqrt{n}$, *whatever the dimension of* X. To halve the error you need four times as many runs; to gain one more digit, a hundred times as many. The good news is in the small print: the rate does not care whether X has 1 component or 1000. For a launch vehicle with dozens of uncertain inputs, that is why Monte Carlo beats every grid-based method.

A first example. A classic test function used throughout Robert and Casella's books is $h(x) = \left(\cos 50x + \sin 20x\right)^2$ on $[0,1]$: wiggly enough that a coarse grid would struggle. With $X \sim \mathcal{U}(0,1)$, $f(x) = 1$ on $[0,1]$ and (6.2) is just $\int_0^1 h(x)\, \mathrm{d}x$.

R

```r
# Classical Monte Carlo integration of h(x) on [0, 1]
h <- function(x) (cos(50 * x) + sin(20 * x))^2
exact <- integrate(h, 0, 1)$value # numerical check
set.seed(3); n <- 10000
x <- runif(n) # X ~ U(0,1)
est <- mean(h(x)) # sample mean
se <- sd(h(x)) / sqrt(n) # CLT error
round(c(exact = exact, estimate = est, se = se,
      lower = est - 1.96 * se, upper = est + 1.96 * se), 4)
```

```
  exact estimate     se  lower  upper
 0.9652   0.9655 0.0104 0.9451 0.9858
```

Python

```python
# Classical Monte Carlo integration of h(x) on [0, 1]
import numpy as np
from scipy.integrate import quad
h = lambda x: (np.cos(50 * x) + np.sin(20 * x))**2
exact = quad(h, 0, 1, limit=200)[0] # numerical check
rng = np.random.default_rng(3); n = 10000
x = rng.uniform(size=n) # X ~ U(0,1)
est = h(x).mean() # sample mean
se = h(x).std(ddof=1) / np.sqrt(n) # CLT error
print(np.round([exact, est, se, est - 1.96*se, est + 1.96*se], 4))
```

```
[0.9652 0.967 0.0105 0.9465 0.9875]
```

Both estimates (0.9655 and 0.9670) are within one standard error (≈ 0.0104) of the true value 0.9652, and both 95% intervals contain it. With 10 000 samples we have about two correct digits: the square-root law at work.

6.10.3 Monitoring convergence

How do you know n is large enough? Robert and Casella's Section 4.1 recommends watching the estimate *as it grows*: plot $\hat{\mu}_k$ against k together with the band $\hat{\mu}_k \pm 2 s_k/\sqrt{k}$. Early on the estimate jumps around and the band is wide; as k grows, the curve settles and the band narrows like $1/\sqrt{k}$ (Fig. 6.3(a)). If the curve is still drifting when the run ends, you have not run enough. This takes one line in either language: `cumsum(y) / (1:n)` in R, `np.cumsum(y) / np.arange(1, n+1)` in Python.

6.10.4 Probabilities are expectations too

A probability is the expected value of an *indicator*: a function that is 1 when the event happens and 0 otherwise,

$$P(A) = \mathbb{E}[\mathbb{1}_A(X)], \qquad \hat{p} = \frac{\text{number of runs in which } A \text{ happened}}{n}.$$

Each indicator has variance $p(1-p)$, so the standard error is $\sqrt{\hat{p}(1-\hat{p})/n}$.

> **Worked example**: How likely is LV-N1 to fall short?. The ideal velocity budget is $\Delta v = I_{sp} g_0 \ln\big((m_{\text{dry}} + m_p)/m_{\text{dry}}\big)$. Suppose the specific impulse is known to ± 1.5 s, the dry mass to ± 150 kg and the loaded propellant to ± 300 kg (one standard deviation, normal). How likely is $\Delta v < 5700$ m/s?
>
> *A rough answer by hand first.* The slopes of Δv with respect to the three inputs, times their standard deviations, are
>
> $$1.5\, g_0 \ln\tfrac{120}{18} = 27.9\,\text{m/s}, \quad 150\, c\left(\tfrac{1}{120\,000} - \tfrac{1}{18\,000}\right) = -21.5\,\text{m/s}, \quad 300\,\frac{c}{120\,000} = 7.6\,\text{m/s},$$

with $c = 310 g_0$. Independent errors add in squares: $\sigma_{\Delta v} \approx \sqrt{27.9^2 + 21.5^2 + 7.6^2} = 36.1\,\mathrm{m/s}$. 5700 is $(5700 - 5767.4)/36.1 = -1.87$ standard deviations from the mean, and a normal table gives $P \approx 0.031$.

Monte Carlo (the first line of each output below, 10 000 samples): R gives 0.0276 ± 0.0016 and Python 0.0329 ± 0.0018. Both are within two standard errors of 0.031.

6.10.5 Importance sampling: estimating rare events

The problem. Suppose the true probability is $p = 10^{-6}$. With 10 000 plain runs we expect 0.01 failures — almost certainly none — and the estimate is $\hat{p} = 0$. To see even ten failures we would need ten million runs. For a two-second simulation, that is seven months of computing.

The trick. Robert and Casella's Section 3.3. Take *any* other density g that is positive wherever $h(x)f(x) \neq 0$. Multiply and divide by it inside the integral:

$$\mathbb{E}_f[h(X)] = \int h(x)\, f(x)\, \mathrm{d}x = \int h(x)\, \frac{f(x)}{g(x)}\, g(x)\, \mathrm{d}x = \mathbb{E}_g\left[h(Y)\, \frac{f(Y)}{g(Y)}\right]. \tag{6.5}$$

Nothing has changed mathematically, but the right-hand side is an expectation under g. So draw $Y_1, \ldots, Y_n$ from g instead, and average $h(Y_i)\, w(Y_i)$, with *importance weights* $w = f/g$:

$$\hat{\mu}_{IS} = \frac{1}{n} \sum_{i=1}^{n} h(Y_i)\, \frac{f(Y_i)}{g(Y_i)}.$$

Choose g to put many samples *where the failures are*. Each failure then counts for less (its weight f/g is small), but we see thousands of them instead of none.

> **Key idea**: Importance sampling in plain words. Cheat on purpose, then correct for the cheating. Sample from a world where failure is common, and give each sample a weight equal to how much more likely it was in that world than in the real one.

Example (a): a normal tail. $P(Z > 4.5)$ for $Z \sim \mathcal{N}(0,1)$ is exactly 3.398×10^{-6}. Plain Monte Carlo with $n = 10\,000$ sees no samples beyond 4.5. As proposal take $Y = 4.5 + E$ with E exponential: every sample lands in the tail. The weight is $\varphi(y)/e^{-(y-4.5)}$, where φ is the normal density.

Example (b): LV-N1 falling far short. What is $P(\Delta v < 5600\,\mathrm{m/s})$? That is -4.65 standard deviations. Write the three inputs as standard normals $\boldsymbol{z}$. The failure region is roughly the half-space "$\boldsymbol{a} \cdot \boldsymbol{z} < -4.65\,\|\boldsymbol{a}\|$" with the slopes $\boldsymbol{a} = (27.9, -21.5, 7.6)$ from the last example. Its most likely point is $\boldsymbol{z}^\star = -4.65\,\boldsymbol{a}/\,\|\boldsymbol{a}\|$. We sample from the same normal shifted to $\boldsymbol{z}^\star$. The weight is the ratio of the two normal densities, which simplifies beautifully:

$$w(\boldsymbol{z}) = \frac{\varphi(\boldsymbol{z})}{\varphi(\boldsymbol{z} - \boldsymbol{z}^\star)} = \frac{e^{-\|\boldsymbol{z}\|^2/2}}{e^{-\|\boldsymbol{z}-\boldsymbol{z}^\star\|^2/2}} = \exp\!\left(-\boldsymbol{z} \cdot \boldsymbol{z}^\star + \tfrac{1}{2}\,\|\boldsymbol{z}^\star\|^2\right).$$

R

```r
# Rare events by importance sampling
set.seed(7); n <- 10000
# (a) P(Z > 4.5) for Z ~ N(0,1): exact
    value from pnorm
naive <- mean(rnorm(n) > 4.5)
y <- 4.5 + rexp(n) # proposal: exponential
     beyond 4.5
w <- dnorm(y) / dexp(y - 4.5) # weights f/
    g
c(exact = pnorm(4.5, lower.tail = FALSE),
    naive = naive,
  IS = mean(w), se = sd(w) / sqrt(n))

# (b) P(dv < 5600 m/s) for LV-N1: shift
    the inputs towards failure
dv <- function(z) (310 + 1.5 * z[, 1]) *
    9.80665 *
  log((18000 + 150 * z[, 2] + 102000 + 300
      * z[, 3]) /
     (18000 + 150 * z[, 2]))
a <- c(27.91, -21.53, 7.60) # slope of dv
    in z (sec. 6.9)
zs <- -4.65 * a / sqrt(sum(a^2)) # most
    likely failure point
Z <- matrix(rnorm(3 * n), n, 3) + matrix(
    zs, n, 3, byrow = TRUE)
w <- exp(-Z %*% zs + sum(zs^2) / 2) # phi(
    z) / phi(z - zs)
hit <- (dv(Z) < 5600) * w
c(naive_hits = sum(dv(matrix(rnorm(3 * n),
     n, 3)) < 5600),
  IS = mean(hit), se = sd(hit) / sqrt(n))
```

```
     exact naive IS se
3.397673e-06 0.000000e+00 3.433440e-06
    4.420341e-08
  naive_hits IS se
0.000000e+00 1.306306e-06 3.085023e-08
```

Python

```python
# Rare events by importance sampling
import numpy as np
from scipy.stats import norm, expon
rng = np.random.default_rng(7); n = 10000
# (a) P(Z > 4.5) for Z ~ N(0,1): exact
    value from norm.sf
naive = np.mean(rng.normal(size=n) > 4.5)
y = 4.5 + rng.exponential(size=n) #
    proposal: exponential beyond 4.5
w = norm.pdf(y) / expon.pdf(y - 4.5) #
    weights f/g
print("exact", norm.sf(4.5), "naive",
    naive,
    "IS", w.mean(), "se", w.std(ddof=1) /
        n**0.5)

# (b) P(dv < 5600 m/s) for LV-N1: shift
    the inputs towards failure
def dv(z):
   md = 18000 + 150 * z[:, 1]
   return (310 + 1.5 * z[:, 0]) * 9.80665 *
        np.log(
      (md + 102000 + 300 * z[:, 2]) / md)
a = np.array([27.91, -21.53, 7.60]) #
    slope of dv in z (sec. 6.9)
zs = -4.65 * a / np.linalg.norm(a) # most
    likely failure point
Z = rng.normal(size=(n, 3)) + zs
w = np.exp(-Z @ zs + zs @ zs / 2) # phi(z)
     / phi(z - zs)
hit = (dv(Z) < 5600) * w
print("naive hits", np.sum(dv(rng.normal(
    size=(n, 3))) < 5600),
    "IS", hit.mean(), "se", hit.std(ddof
        =1) / n**0.5)
```

```
exact 3.3976731247300535e-06 naive 0.0 IS
    3.337807245991495e-06 se
    4.360545653833701e-08
naive hits 0 IS 1.3202415074817527e-06 se
    3.090109669043727e-08
```

Reading the results. For the normal tail both languages are within 2% of the exact 3.398×10^{-6}, with a standard error near 4.4×10^{-8} — from 10 000 samples that plain Monte Carlo would have reported as "zero". For LV-N1, importance sampling gives $P(\Delta v < 5600) \approx 1.31 \times 10^{-6}$ (standard error 0.03×10^{-6}). As an independent check we ran *two hundred million* plain samples: 278 failures, $\hat{p} = (1.39 \pm 0.08) \times 10^{-6}$. Importance sampling reached the same answer with 1/20 000 of the work, and a smaller error bar. (The straight-line estimate $\Phi(-4.65) = 1.7 \times 10^{-6}$ is a little too high, because Δv is curved: the logarithm in the rocket equation bends the failure boundary.)

> **Reality check.** A bad proposal can make things worse: if g is too narrow, or put in the wrong place, a few rare samples get enormous weights and the variance explodes. Always look at the standard error and the largest weights. The "shift to the most likely

> failure point" used here is the standard first choice in reliability engineering.

6.10.6 Getting more from fewer runs: antithetic variables and control variates

Robert and Casella's Section 4.4 describes two ways to shrink σ itself, instead of increasing n.

Antithetic variables. If $Z \sim \mathcal{N}(0, \mathbf{I})$, so is $-Z$. Run the simulation in pairs, at Z and at $-Z$, and average each pair: $\bar{Y} = \frac{1}{2}\big(h(Z) + h(-Z)\big)$. For a pair of variables with the same variance σ^2 and correlation ρ,

$$\mathrm{Var}[\bar{Y}] = \tfrac{1}{4}\big(\sigma^2 + \sigma^2 + 2\rho\sigma^2\big) = \frac{\sigma^2}{2}(1+\rho).$$

With n runs we have $n/2$ pairs, so the variance of the estimate is $\sigma^2(1+\rho)/n$ against σ^2/n for plain sampling. If h is increasing in the inputs, $h(Z)$ and $h(-Z)$ move in opposite directions, ρ is negative, and we win. If h is nearly linear, $\rho \approx -1$ and the win is enormous.

Control variates. Suppose we have a second quantity X, computed from the same random inputs, whose mean $\mathbb{E}[X]$ we know *exactly*, and which is correlated with $Y = h(Z)$. Use the known error of X to correct Y:

$$\hat{\mu}_{CV} = \frac{1}{n}\sum_{i=1}^{n}\Big(Y_i - b\,\big(X_i - \mathbb{E}[X]\big)\Big).$$

For any b this still has mean μ, since the correction has mean zero. Its variance is $\sigma_Y^2 - 2b\,\mathrm{Cov}(X,Y) + b^2\sigma_X^2$, a parabola in b. Setting its derivative to zero gives the best multiplier and the remaining variance:

$$b^\star = \frac{\mathrm{Cov}(X,Y)}{\mathrm{Var}(X)}, \qquad \mathrm{Var}[Y - b^\star X] = \sigma_Y^2\big(1 - \rho_{XY}^2\big).$$

A correlation of 0.99 cuts the variance by a factor of 50; 0.9999, by 5000. For LV-N1 a natural control variate is the straight-line (first-order Taylor) model of Δv, $X = \Delta v_0 + \boldsymbol{a}\cdot\boldsymbol{Z}$, whose mean is exactly Δv_0.

R

```r
# Delta-v of LV-N1 with uncertain Isp, dry mass and propellant
dv <- function(z) { # z: n x 3 standard normals
  isp <- 310 + 1.5 * z[, 1]; md <- 18000 + 150 * z[, 2]
  mp <- 102000 + 300 * z[, 3]
  isp * 9.80665 * log((md + mp) / md)
}
set.seed(6); n <- 10000
Z <- matrix(rnorm(3 * n), n, 3); y <- dv(Z)

p <- mean(y < 5700)
cat("plain: mean", round(mean(y), 2), " se ", round(sd(y) / sqrt(n), 3),
    " P(dv<5700)", p, " se", round(sqrt(p * (1 - p) / n), 4), "\n")

# antithetic pairs (z, -z): same n = 10000 evaluations
Za <- Z[1:(n / 2), ]; ya <- (dv(Za) + dv(-Za)) / 2
cat("anti: mean", round(mean(ya), 2), " se ", round(sd(ya) / sqrt(n/2), 4), "\n")

# control variate: the straight-line (Taylor) model, exact mean known
g0 <- dv(matrix(0, 1, 3)); c0 <- 310 * 9.80665
grad <- c(1.5 * 9.80665 * log(120000 / 18000),
          150 * c0 * (1 / 120000 - 1 / 18000), 300 * c0 / 120000)
x <- drop(g0 + Z %*% grad) # E[x] = g0 exactly
b <- cov(y, x) / var(x) # best multiplier
ycv <- y - b * (x - g0)
cat("control: mean", round(mean(ycv), 2), " se", round(sd(ycv) / sqrt(n), 4),
    " b =", round(b, 4), "\n")
```

```
plain: mean 5768.13 se 0.363 P(dv<5700) 0.0276 se 0.0016
anti: mean 5767.46 se 0.0026
control: mean 5767.46 se 0.0018 b = 1.0002
```

Python

```python
# Delta-v of LV-N1 with uncertain Isp, dry mass and propellant
import numpy as np
def dv(z): # z: n x 3 standard normals
    isp = 310 + 1.5 * z[:, 0]; md = 18000 + 150 * z[:, 1]
    mp = 102000 + 300 * z[:, 2]
    return isp * 9.80665 * np.log((md + mp) / md)
rng = np.random.default_rng(6); n = 10000
Z = rng.normal(size=(n, 3)); y = dv(Z)
p = np.mean(y < 5700)
print("plain: mean", round(y.mean(), 2), " se", round(y.std(ddof=1) / n**0.5, 3),
      " P(dv<5700)", p, " se", round(np.sqrt(p * (1 - p) / n), 4))

# antithetic pairs (z, -z): same n = 10000 evaluations
Za = Z[: n // 2]; ya = (dv(Za) + dv(-Za)) / 2
print("anti: mean", round(ya.mean(), 2), " se", round(ya.std(ddof=1) / (n/2)**0.5, 4))

# control variate: the straight-line (Taylor) model, exact mean known
g0 = dv(np.zeros((1, 3)))[0]; c0 = 310 * 9.80665
grad = np.array([1.5 * 9.80665 * np.log(120000 / 18000),
                 150 * c0 * (1 / 120000 - 1 / 18000), 300 * c0 / 120000])
x = g0 + Z @ grad # E[x] = g0 exactly
b = np.cov(y, x)[0, 1] / x.var(ddof=1) # best multiplier
ycv = y - b * (x - g0)
print("control: mean", round(ycv.mean(), 2), " se", round(ycv.std(ddof=1) / n**0.5, 4),
      " b =", round(b, 4))
```

```
plain: mean 5767.72 se 0.362 P(dv<5700) 0.0329 se 0.0018
anti: mean 5767.46 se 0.0026
control: mean 5767.45 se 0.0018 b = 1.0
```

Reading the results. All three methods use exactly 10 000 evaluations of Δv. Plain Monte Carlo gives the mean to about $\pm 0.36\,\mathrm{m/s}$. Antithetic pairs give $\pm 0.0026\,\mathrm{m/s}$ and the control variate $\pm 0.0018\,\mathrm{m/s}$: standard errors 140 and 200 times smaller, which would have needed $140^2 \approx 20\,000$ and 40 000 times more plain runs. The control-variate multiplier comes out as $b \approx 1.000$: the straight-line model is almost perfect, so the method only has to correct its small curvature. And the mean, $5767.46\,\mathrm{m/s}$, is $0.10\,\mathrm{m/s}$ *above* the nominal $\Delta v_0 = 5767.36\,\mathrm{m/s}$ — a real effect, invisible to plain Monte Carlo at this n: because ln is curved, a lighter-than-nominal vehicle gains more than a heavier one loses.

6.10.7 A dispersion study of the full simulator

Now the real thing: fly the complete 6-DOF simulator 400 times, each time with every uncertain input drawn at random, and turn the 400 results into answers. Each input is modelled with a distribution from Section 2.11:

Input	Distribution	Meaning
specific impulse factor	$\mathcal{N}(1,\ 0.005^2)$	engine efficiency $\pm 1.5\,\mathrm{s}$
mass-flow factor	$\mathcal{N}(1,\ 0.01^2)$	thrust level and burn time $\pm 1\%$
C_A and C_{N_α} factors	$\mathcal{N}(1,\ 0.10^2)$ each	aerodynamic tables $\pm 10\%$
centre-of-pressure shift	$\mathcal{N}(0,\ 0.3\,\mathrm{m}^2)$	where the air pushes
dry-mass factor	$\mathcal{N}(1,\ 0.01^2)$	structure $\pm 180\,\mathrm{kg}$
peak wind speed	Rayleigh, $\sigma = 12\,\mathrm{m/s}$	inverse transform, Section 2.11.2
wind direction	$\mathcal{U}(0, 2\pi)$	from any direction
engine misalignment	$\mathcal{N}(0,\ (0.05^\circ)^2)$	a fixed tilt of the nozzle in pitch

The Python program below draws the inputs, runs `lvsim.py` 400 times (about six minutes) and writes one row per flight to a file, `mc6dof.csv`:

```
# Monte Carlo dispersion study of the full 6-DOF simulation (writes
    mc6dof.csv)
import csv, math
import numpy as np
from lvsim import LVSim

rng = np.random.default_rng(2026)
N = 400
rows = []
for i in range(N):
    W = 12.0 * math.sqrt(-2 * math.log(1 - rng.uniform())) # Rayleigh
        wind (Ch. 2)
    psi = rng.uniform(0, 2 * math.pi) # uniform direction
    disp = dict(k_isp=rng.normal(1, 0.005), k_mdot=rng.normal(1, 0.01),
                k_CA=rng.normal(1, 0.10), k_CNa=rng.normal(1, 0.10),
                d_scp=rng.normal(0, 0.3), k_mdry=rng.normal(1, 0.01),
                wind_n=W * math.cos(psi), wind_e=W * math.sin(psi),
                gimbal_bias=math.radians(rng.normal(0, 0.05)))
    r = LVSim(**disp).run(dt=0.02)
    rows.append(dict(disp, wind=W, h_km=r['h'] / 1000, V=r['V'], gamma=r[
        'gamma'],
                     qbar_kPa=r['qbar'] / 1000, qalpha=r['qalpha'],
                     delta_deg=math.degrees(r['delta'])))
with open('mc6dof.csv', 'w', newline='') as fh:
    w = csv.DictWriter(fh, fieldnames=list(rows[0]))
    w.writeheader(); w.writerows(rows)
print("wrote", len(rows), "runs")
```

This is how real dispersion studies are organised: the simulator is run by one program, and the results are analysed separately — often in a different language. Here the analysis is done in both R and Python, from the same file, so the numbers agree exactly:

R

```r
d <- read.csv("mc6dof.csv"); n <- nrow(d)
    # 400 runs from Python
ci <- function(x) round(mean(x) + c(-1.96,
     0, 1.96) * sd(x) / sqrt(n), 2)
ci(d$h_km) # burnout altitude [km]
ci(d$V) # burnout speed [m/s]
pr <- function(a) round(c(p = mean(a), se
    = sqrt(mean(a) * (1 - mean(a)) / n)),
    4)
pr(d$qbar_kPa > 60) # P(max-Q above 60 kPa)

pr(d$h_km < 95) # P(burnout below 95 km)
quantile(d$h_km, c(0.05, 0.95)) # 90% of
    flights between
ins <- c("k_isp", "k_mdot", "k_CA", "k_CNa
    ", "d_scp", "k_mdry",
      "wind_n", "wind_e", "gimbal_bias")
sort(round(cor(d[ins], d$h_km)[, 1], 2)) #
     what drives the altitude?
```

```
[1] 108.67 109.57 110.47
[1] 4607.08 4611.64 4616.20
    p se
0.1050 0.0153
    p se
0.0750 0.0132
     5% 95%
 92.95474 123.93760
gimbal_bias k_mdry d_scp k_CNa wind_n k_CA
     -0.50 -0.15 -0.05 -0.03 -0.03 0.03
     k_isp k_mdot wind_e
      0.34 0.55 0.60
```

Python

```python
import numpy as np, pandas as pd
d = pd.read_csv("mc6dof.csv"); n = len(d)
    # 400 runs
ci = lambda x: np.round(x.mean() + np.
    array([-1.96, 0, 1.96])
                  * x.std() / np.sqrt(n),
                      2)
print("h_km", ci(d.h_km)) # burnout
    altitude [km]
print("V ", ci(d.V)) # burnout speed [m/s]
def pr(a):
   p = float(a.mean()); return round(p, 4),
        round(float(np.sqrt(p * (1 - p) /
       n)), 4)
print("P(qbar > 60 kPa)", pr(d.qbar_kPa >
    60))
print("P(h < 95 km) ", pr(d.h_km < 95))
print("5%, 95%:", d.h_km.quantile([0.05,
    0.95]).round(2).tolist())
ins = ["k_isp", "k_mdot", "k_CA", "k_CNa",
     "d_scp", "k_mdry",
     "wind_n", "wind_e", "gimbal_bias"]
print(d[ins].corrwith(d.h_km).round(2).
    sort_values().to_string())
```

```
h_km [108.67 109.57 110.47]
V [4607.08 4611.64 4616.2 ]
P(qbar > 60 kPa) (0.105, 0.0153)
P(h < 95 km) (0.075, 0.0132)
5%, 95%: [92.95, 123.94]
gimbal_bias -0.50
k_mdry -0.15
d_scp -0.05
k_CNa -0.03
wind_n -0.03
k_CA 0.03
k_isp 0.34
k_mdot 0.55
wind_e 0.60
```

Reading the results, one line at a time.

- **The mean is known well.** The mean burnout altitude is 109.57 km with a 95% confidence interval of [108.67, 110.47], and the mean speed is 4611.6 ± 4.6 m/s. Both intervals contain the nominal flight (109.73 km, 4608.5 m/s). Figure 6.3(b) shows that the running mean had settled to within a kilometre after about 100 runs.
- **But single flights vary a lot.** The standard deviation of the 400 altitudes is 9.2 km, and 90% of the flights end between 93.0 and 123.9 km. Do not confuse the two numbers: $s = 9.2$ km describes *the vehicle* (how different one flight is from the next); $s/\sqrt{n} = 0.46$ km describes *our knowledge* (how well 400 runs pin down the average). More runs shrink the second, never the first.
- **Probabilities come with error bars.** $P(\bar{q}_{\max} > 60\,\text{kPa}) = 0.105 \pm 0.015$ and $P(h_{\text{burnout}} < 95\,\text{km}) = 0.075 \pm 0.013$. With 400 runs we know a 10% probability to about ± 3 percentage points (two standard errors). For a one-in-a-thousand requirement, plain sampling

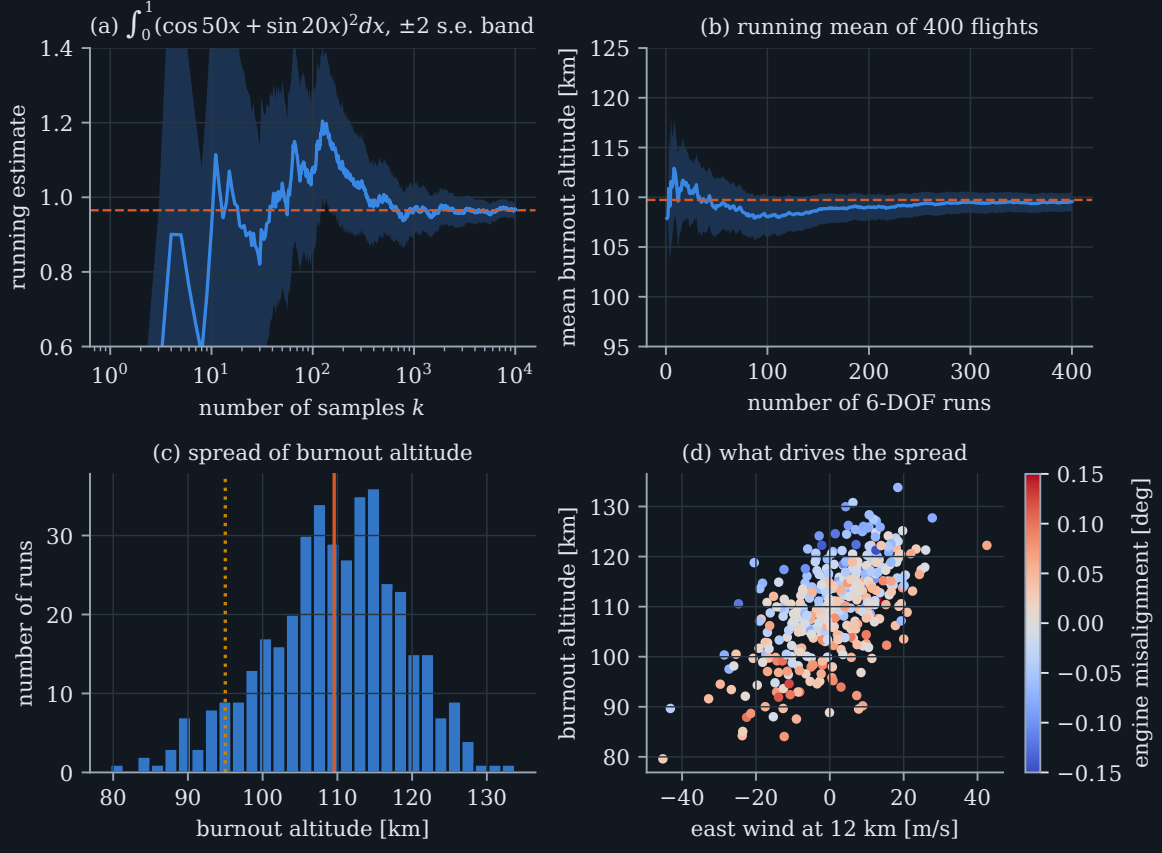


Figure 6.3: Monte Carlo in practice. (a) Monitoring convergence (Robert and Casella, Section 4.1): the running estimate of $\int_0^1(\cos 50x + \sin 20x)^2\,\mathrm{d}x$ with its ±2 standard-error band, on a logarithmic axis; the dashed line is the true value 0.9652. (b) The same for the mean burnout altitude of the 400 dispersed flights; the dashed line is the nominal flight. (c) The spread of the 400 burnout altitudes; the solid line is the mean and the dotted line the 95 km threshold. (d) Burnout altitude against the east wind at 12 km, coloured by the engine misalignment: the two inputs together explain most of the spread.

would need tens of thousands of runs; that is where importance sampling (Section 6.10.5) earns its keep.

- **What drives the spread?** The correlations rank the inputs. The east wind (+0.60), the mass flow (+0.55), the engine misalignment (−0.50) and the specific impulse (+0.34) matter; the aerodynamic coefficients hardly do ($|r| \leq 0.05$). The reason is the gravity turn: the vehicle flies at nearly zero angle of attack, so the normal-force and centre-of-pressure errors have almost nothing to act on, while anything that changes the *direction* of flight in the first twenty seconds — a crosswind, a slightly tilted nozzle, a stronger engine that climbs more steeply before the turn — is carried all the way to burnout.

> **Key idea**: What the dispersion study tells the designer. The open-loop gravity turn is *sensitive*: small errors early become kilometres at burnout. That is not a flaw of the simulator but the first real engineering result it has produced, and it is the reason

launch vehicles fly closed-loop guidance that measures where the vehicle is and corrects the path in flight. Building that guidance — classically in Chapter 9 and with hybrid classical–quantum methods in Chapter 10 — is the job of Part II. Its success will be measured with exactly the Monte Carlo tools of this chapter.

Summary

- A simulator is a state $\boldsymbol{x}$, a derivative function $\boldsymbol{f}(t, \boldsymbol{x})$ that contains all the physics, and an integrator. LV-N1's state has 15 numbers: $\boldsymbol{r}, \boldsymbol{v}, \boldsymbol{q}, \boldsymbol{\omega}, \delta_p, \delta_y$; mass properties are functions of time.
- The derivative function runs 12 steps: mass properties, air, air-relative velocity, thrust, aerodynamics, guidance and control, thrust vector, Newton, Euler, quaternion rates.
- A vertical rise, a small pitch kick and a gravity turn fly the vehicle through the air at almost zero angle of attack. Nominal LV-N1: max-Q 56.7 kPa at 52 s; burnout at 109.7 km, 4608 m/s.
- RK4 at $h = \tau/5 = 0.01$ s is accurate to below a millimetre. Verification compares against exact answers (rocket equation), smaller steps and an independent implementation.
- Monte Carlo estimates an expectation by an average; its error is $s/\sqrt{n}$ whatever the dimension. Probabilities are averages of indicators. Monitor the running mean and its band.
- Importance sampling estimates rare events by sampling where they happen and weighting by f/g.
- Antithetic pairs (Var $\propto 1 + \rho$) and control variates (Var $\propto 1 - \rho^2$) can give the same accuracy with thousands of times fewer runs.
- A dispersion study of the full simulator turns hundreds of runs into estimates with confidence intervals and shows which inputs matter.

Exercises

6.1 State. Why is the mass not part of the state vector? Describe a vehicle (or engine) for which it would have to be.

6.2 Starting state. Compute the starting quaternion for a launch from latitude 45°, and the eastward speed of the pad.

6.3 J_2 gravity. Add the J_2 term of Chapter 2 to `gravity`. How much do the burnout altitude and speed change?

6.4 No feed-forward. Remove the term `- M_A[1]` (and `- M_A[2]`) from the control law. Does LV-N1 survive max-Q? Try $\omega_n = 2.5$ instead.

6.5 **Kick angle.** Run the simulator with pitch kicks of 12°, 16° and 20°. Tabulate the burnout altitude, flight-path angle and max-Q. Explain the trends.

6.6 **Step size.** Repeat Test 2 with forward Euler instead of RK4. Which step size gives the same accuracy as RK4 at 0.01 s, and how much longer does it take?

6.7 **Square-root law.** Estimate $\int_0^1 (\cos 50x + \sin 20x)^2 \, \mathrm{d}x$ with $n = 10^2, 10^3, \dots, 10^6$ and plot the absolute error against n on log axes. What slope do you expect?

6.8 **Importance sampling.** Estimate $P(Z > 6)$ with the exponential proposal. Then try the proposal $\mathcal{N}(6, 1)$. Which has the smaller standard error?

6.9 **Antithetic variables.** Use antithetic pairs to estimate $P(\Delta v < 5700)$. Is the gain as large as for the mean? Explain using the correlation of the indicators.

6.10 **Dispersion study.** Add a random engine misalignment in yaw as well as pitch (standard deviation 0.05°) and repeat the 400-run study. Which output is most affected?

Reading for this chapter

- P. H. Zipfel, *Modeling and Simulation of Aerospace Vehicle Dynamics*, 2nd ed.: Section 10.1 (the six-degree-of-freedom equations of motion, flat and elliptical Earth), Section 10.3 (Monte Carlo analysis, accuracy analysis), Section 10.4 and Appendices B–C (the CADAC simulation architecture).
- K. J. Åström and R. M. Murray, *Feedback Systems*: Section 5.4 (linearisation of a simulation model for control design).
- R. R. Labbe, *Kalman and Bayesian Filters in Python*: Chapter 4 (Gaussian probabilities) and Section 7.5 (designing the process-noise matrix) — how random errors are modelled.
- C. P. Robert and G. Casella, *Monte Carlo Statistical Methods*, 2nd ed.: Chapter 3 (3.1 introduction, 3.2 classical Monte Carlo integration, 3.3 importance sampling), Section 4.1 (monitoring variation with the CLT) and Section 4.4 (acceleration methods: antithetic variables, control variates) — **STAT 3585 core**.
- S. Boyd and L. Vandenberghe, *Convex Optimization*: Chapter 3 (convex functions), in preparation for Chapter 9.